



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Diseño e implementación de
una plataforma web para el
procesamiento de imágenes
mediante pipelines de
inteligencia artificial**



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 23 de mayo
de 2026

Tutores: Jose Luis Garrido Labrador y Jose
Miguel Ramirez Sanz

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	23
Apéndice B Especificación de Requisitos	39
B.1. Introducción	39
B.2. Objetivos generales	39
B.3. Catálogo de requisitos	40
B.4. Especificación de requisitos	47
Apéndice C Especificación de diseño	69
C.1. Introducción	69
C.2. Diseño de datos	69
C.3. Diseño arquitectónico	86
C.4. Diseño procedimental	96
C.5. Diseño de interfaces	101
Apéndice D Documentación técnica de programación	119
D.1. Introducción	119
D.2. Estructura de directorios	119

D.3. Manual del programador	119
D.4. Compilación, instalación y ejecución del proyecto	119
D.5. Pruebas del sistema	119
Apéndice E Documentación de usuario	121
E.1. Introducción	121
E.2. Requisitos de usuarios	121
E.3. Instalación	121
E.4. Manual del usuario	121
Apéndice F Anexo de sostenibilización curricular	123
F.1. Introducción	123
Bibliografía	125

Índice de figuras

A.1. Informe de trabajo completado Sprint 1	3
A.2. Informe de trabajo completado Sprint 2	6
A.3. Informe de trabajo completado Sprint 3	8
A.4. Informe de trabajo completado Sprint 4	9
A.5. Informe de trabajo completado Sprint 5	11
A.6. Informe de trabajo completado Sprint 6	12
A.7. Informe de trabajo completado Sprint 7	14
A.8. Informe de trabajo completado Sprint 8	15
A.9. Informe de trabajo completado Sprint 9	16
A.10. Informe de trabajo completado Sprint 10	18
A.11. Informe de trabajo completado Sprint 11	19
A.12. Informe de trabajo completado Sprint 12	21
A.13. Informe de trabajo completado Sprint 13	23
 B.1. Diagrama de casos de uso del sistema	 47
 C.1. Diagrama entidad-relación del sistema	 71
C.2. Diagrama del modelo relacional del sistema	72
C.3. Arquitectura MVC adaptada a Flask con capa de servicios	88
C.4. Vista de módulos y dependencias principales de la aplicación . .	90
C.5. Diagrama de clases y módulos principales de la capa de servicios	93
C.6. Diagrama de secuencia del CU-1 Registrarse.	102
C.7. Diagrama de secuencia del CU-2 Iniciar sesión.	103
C.8. Diagrama de secuencia del CU-3 Gestionar perfil.	104
C.9. Diagrama de secuencia del CU-4 Solicitar baja de cuenta.	105
C.10. Diagrama de secuencia del CU-5 Consultar tienda de servicios. .	105
C.11. Diagrama de secuencia del CU-6 Suscribirse a un plan.	106
C.12. Diagrama de secuencia del CU-7 Alquilar un modelo de IA. . . .	106

C.13.Diagrama de secuencia del CU-8 Consultar compras y servicios activos.	107
C.14.Diagrama de secuencia del CU-9 Consultar guía de compra. . .	108
C.15.Diagrama de secuencia del CU-10 Consultar pipelines disponibles.	109
C.16.Diagrama de secuencia del CU-11 Cargar y modificar configuración del pipeline.	110
C.17.Diagrama de secuencia del CU-12 Ejecutar pipeline sobre una imagen.	111
C.18.Diagrama de secuencia del CU-13 Visualizar y descargar resultados.	112
C.19.Diagrama de secuencia del CU-14 Consultar historial de ejecuciones.	113
C.20.Diagrama de secuencia del CU-15 Acceder al panel de administración.	114
C.21.Diagrama de secuencia del CU-16 Gestionar usuarios desde administración.	115
C.22.Diagrama de secuencia del CU-17 Gestionar pipelines desde administración.	116
C.23.Diagrama de secuencia del CU-18 Consultar ejecuciones globales.	117
C.24.Diagrama de secuencia del CU-19 Ejecutar tareas de mantenimiento.	118

Índice de tablas

A.1. Sprint 1 – Resumen	3
A.2. Sprint 2 – Resumen	6
A.3. Sprint 3 – Resumen	8
A.4. Sprint 4 – Resumen	9
A.5. Sprint 5 – Resumen	10
A.6. Sprint 6 – Resumen	12
A.7. Sprint 7 – Resumen	13
A.8. Sprint 8 – Resumen	15
A.9. Sprint 9 – Resumen	16
A.10.Sprint 10 – Resumen	18
A.11.Sprint 11 – Resumen	19
A.12.Sprint 12 – Resumen	21
A.13.Sprint 13 – Resumen	22
A.14.Costes de personal	25
A.15.Costes de hardware	26
A.16.Costes de software	26
A.17.Costes varios	27
A.18.Costes totales del proyecto	27
A.19.Modelo de ingresos previsto	28
A.20.Estimación de recuperación de la inversión	28
A.21.Dependencias del proyecto y licencias	30
A.22.Modelos y pesos utilizados	32
A.23.Compatibilidad de licencias	33
A.24.Resumen de la licencia AGPL-3.0	34
A.25.Resumen de la licencia CC-BY-4.0	34
A.26.Resumen de licencias del proyecto	37
B.1. CU-1 Registrarse.	49

B.2. CU-2 Iniciar sesión.	50
B.3. CU-3 Gestionar perfil.	51
B.4. CU-4 Solicitar baja de cuenta.	52
B.5. CU-5 Consultar tienda de servicios.	53
B.6. CU-6 Suscribirse a un plan.	54
B.7. CU-7 Alquilar un modelo de IA.	55
B.8. CU-8 Consultar compras y servicios activos.	56
B.9. CU-9 Consultar guía de compra.	57
B.10.CU-10 Consultar pipelines disponibles.	58
B.11.CU-11 Cargar y modificar configuración del pipeline.	59
B.12.CU-12 Ejecutar pipeline sobre una imagen.	60
B.13.CU-13 Visualizar y descargar resultados.	61
B.14.CU-14 Consultar historial de ejecuciones.	62
B.15.CU-15 Acceder al panel de administración.	63
B.16.CU-16 Gestionar usuarios desde administración.	64
B.17.CU-17 Gestionar pipelines desde administración.	65
B.18.CU-18 Consultar ejecuciones globales.	66
B.19.CU-19 Ejecutar tareas de mantenimiento.	67
C.1. Atributos de la tabla USUARIO	73
C.2. Atributos de la tabla ROL	74
C.3. Atributos de la tabla USUARIO_ROL	75
C.4. Atributos de la tabla TIPO_PLAN	76
C.5. Atributos de la tabla SUSCRIPCION_PLAN	77
C.6. Atributos de la tabla IA_MODELO	78
C.7. Atributos de la tabla IA_MODO	79
C.8. Atributos de la tabla ALQUILA	80
C.9. Atributos de la tabla PIPELINE	81
C.10.Atributos de la tabla EJECUCION	82
C.11.Atributos de la tabla TEMPORAL_ARCHIVO	83
C.12.Atributos de la tabla PIPELINE_ETAPA	84

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este Trabajo Fin de Grado se ha gestionado con **Scrum**, un marco de trabajo ágil orientado a la entrega iterativa e incremental de valor. Scrum se basa en el *control empírico de procesos* (transparencia, inspección y adaptación) y estructura el desarrollo en iteraciones cortas llamadas *sprints*, con artefactos y eventos bien definidos que favorecen el enfoque, la priorización y la mejora continua.

Scrum

Roles. En este proyecto los roles se han adaptado de forma pragmática:

- **Product Owner (PO):** el propio estudiante, responsable de priorizar el *Product Backlog* alineado con los objetivos del TFG y las indicaciones de los tutores.
- **Scrum Master (SM):** rol asumido también por el estudiante, velando por la correcta aplicación del marco y la eliminación de impedimentos.
- **Equipo de desarrollo:** el estudiante (desarrollo, pruebas y documentación). *Stakeholders:* tutores y tribunal.

Artefactos.

- **Product Backlog:** lista priorizada de épicas e historias (documentación, investigación de modelos de IA de procesamiento de imágenes, pruebas comparativas, etc.).
- **Sprint Backlog:** subconjunto comprometido para cada sprint, desglosado en tareas técnicas.
- **Incremento:** entregables verificables al final de cada sprint (p.ej., secciones de la memoria, scripts de prueba, comparativas).

Eventos.

- **Sprint Planning:** definición del objetivo del sprint y selección de trabajo.
- **Daily Scrum:** chequeo diario (*qué hice/haré/impedimentos*) para mantener foco.
- **Sprint Review:** revisión de avances con evidencias (código, resultados, documentación).
- **Sprint Retrospective:** lecciones aprendidas y acciones de mejora para el siguiente sprint.

Medición y estimación del trabajo

Para la planificación y seguimiento del progreso se ha empleado un sistema de medición basado en puntos de historia, una unidad habitual en metodologías ágiles como Scrum. Cada tarea o ítem del *Sprint Backlog* se ha valorado con un número de puntos que refleja su esfuerzo y complejidad relativa.

En este proyecto, se ha establecido una equivalencia práctica de 8 puntos por jornada completa de trabajo, lo que permite estimar la carga de trabajo de cada sprint y evaluar el avance real frente al planificado. Este sistema facilita mantener una visión objetiva del esfuerzo invertido y ajustar la planificación de los siguientes sprints en función de la velocidad media alcanzada.

A.2. Planificación temporal

La planificación se organiza en sprints cortos (1–2 semanas).

Visión general del Sprint 1

- **Fechas:** 27/10 – 06/11
- **Objetivo del sprint:** *Investigación y pruebas iniciales de IAs de procesado de imágenes.*
- **Criterio de aceptación global:** disponer de dos IAs instaladas y probadas con evidencias, y una comparativa preliminar documentada.

Tal y como se muestra en la Tabla A.1 y en la Figura A.1, este sprint se centra en la fase inicial de investigación y preparación del proyecto.

Tarea	Pts
Organización inicial del proyecto	8
Aprendizaje básico de L ^A T _E X	8
Estructura base del documento TFG	8
Investigar modelos IA de imágenes	8
Instalar y probar 1ª IA	8
Instalar y probar 2ª IA	8
Comparativa entre IAs seleccionadas	8
Cierre Sprint	8

Tabla A.1: Sprint 1 – Resumen



Figura A.1: Informe de trabajo completado Sprint 1

Descripción de las tareas del Sprint 1

A continuación se describen brevemente las tareas que componen el Sprint 1:

- **Organización inicial del proyecto.** Se configuró el entorno de trabajo creando la estructura del repositorio, las carpetas para código, experimentos, documentación y resultados, así como los archivos necesarios para el control de versiones y la vinculación con Jira. Esta tarea permitió establecer la base técnica sobre la que se desarrollará el resto del proyecto.
- **Aprendizaje básico de $\text{\LaTeX}$.** Se dedicó tiempo a adquirir soltura con la herramienta de edición científica $\text{\LaTeX}$, aprendiendo a crear capítulos, incluir figuras, tablas, referencias y bibliografía. Este conocimiento asegura una correcta elaboración y mantenimiento de la memoria del TFG.
- **Estructura base del documento TFG.** Se definió la organización general de la memoria, creando los capítulos principales, anexos y secciones iniciales. De este modo quedó preparada la estructura en la que se irá incorporando el contenido teórico y técnico conforme avance el proyecto.
- **Investigar modelos IA de procesamiento de imágenes.** Se llevó a cabo una investigación de diferentes alternativas de inteligencia artificial aplicadas al procesamiento de imágenes, evaluando su documentación, facilidad de uso, licencia y compatibilidad. Tras este estudio se seleccionaron dos herramientas destacadas: **YOLO** y **MediaPipe**, por su relevancia en el ámbito de la visión por computador.
- **Instalar y probar la primera IA seleccionada (YOLO).** En esta tarea se instaló y configuró el modelo YOLO (You Only Look Once), mediante la librería `ultralytics` en Python. Se realizaron pruebas de detección de objetos sobre imágenes propias para verificar la correcta ejecución del modelo, su velocidad de inferencia y la claridad de sus resultados. Esta fase permitió comprobar la idoneidad de YOLO para el reconocimiento y localización de elementos dentro de una escena.
- **Instalar y probar la segunda IA seleccionada (MediaPipe).** Posteriormente se instaló el framework MediaPipe de Google, que ofrece soluciones preentrenadas orientadas al análisis de características humanas como el rostro, las manos o la pose corporal. Se probaron

distintos módulos, especialmente los de detección de manos y pose, verificando su funcionamiento en tiempo real y su facilidad de integración en proyectos Python.

- **Comparativa entre las IAs seleccionadas.** Una vez probadas ambas herramientas, se elaboró una comparativa que analiza sus principales diferencias: finalidad, precisión, velocidad, facilidad de uso, requerimientos y tipo de tareas para las que son más adecuadas. En esta comparativa se concluye que **MediaPipe** destaca en la detección y seguimiento de características humanas con bajo coste computacional, mientras que **YOLO** ofrece mayor versatilidad para la detección general de objetos y escenarios más complejos. La documentación y análisis detallados de esta comparativa se incluyen en el capítulo 4 de la memoria.
- **Cierre del Sprint.** Finalmente, se revisaron todas las tareas completadas y se documentaron los avances obtenidos, preparando el tablero y el backlog para el siguiente sprint. Esta revisión permitió valorar el cumplimiento de los objetivos y establecer las líneas de trabajo futuras.

Visión general del Sprint 2

- **Fechas:** 09/11 – 20/11
- **Objetivo del sprint:** *Construir un servidor funcional en Flask capaz de recibir imágenes, ejecutar detecciones simuladas y ofrecer una interfaz mínima de usuario.*
- **Criterio de aceptación global:** disponer de un servidor operativo, con endpoints funcionales, selección de modelos simulada y una UI básica que permita validar el flujo completo de subida y procesado.

Tal y como se recoge en la Tabla A.2 y en la Figura A.2, este sprint introduce la construcción del servidor y la primera interfaz funcional.

Descripción de las tareas del Sprint 2

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Definir alcance del sprint y crear historias.** Se concretaron los objetivos del sprint y se desglosaron en historias de usuario.

Tarea	Pts
Definir alcance del sprint y crear historias	8
Elegir framework servidor Flask	8
Elección librería para simulación de detecciones	6
Configurar entorno desarrollo y estructura base	6
Probar funcionamiento servidor	8
Desarrollar endpoint en servidor para subir imágenes	10
Desarrollar endpoint en servidor para detección simulada	12
Crear interfaz mínima de aplicación	8
Validación final y documentación del sprint	6

Tabla A.2: Sprint 2 – Resumen

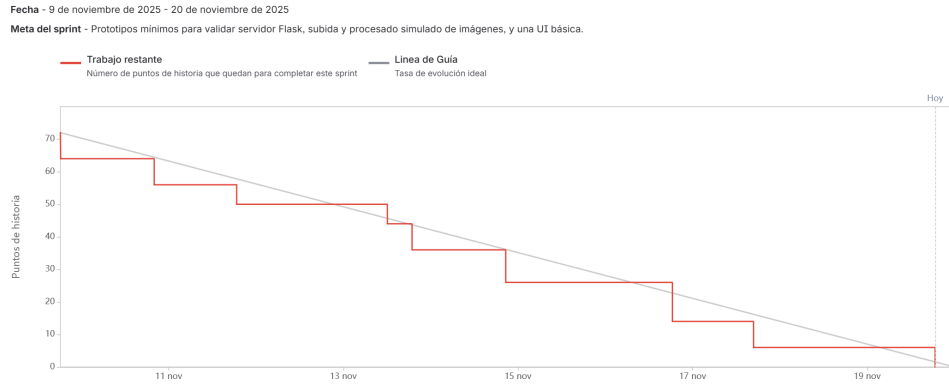


Figura A.2: Informe de trabajo completado Sprint 2

- **Elegir framework servidor.** Tras evaluar distintas opciones, se seleccionó **Flask** como framework principal por su ligereza, modularidad y facilidad para integrar endpoints REST orientados al procesado de imágenes.
- **Elección librería para simulación de detecciones.** Para poder validar el flujo completo antes de integrar YOLO o MediaPipe reales, se creó una librería propia que genera detecciones simuladas (clases, cajas y confianza).
- **Configurar entorno de desarrollo y estructura base.** Se construyó una arquitectura inicial organizada en módulos y se configuró un servidor Flask estable listo para ampliaciones.

- **Probar funcionamiento del servidor.** Se verificó la ejecución del backend, comprobando rutas, carga de configuraciones, manejo de errores y logs. Esta fase permitió asegurar que el servidor respondía correctamente antes de construir los endpoints principales.
- **Desarrollar endpoint para subir imágenes.** Se creó un endpoint capaz de recibir imágenes vía POST, almacenarlas temporalmente y devolver su ruta accesible desde la interfaz.
- **Desarrollar endpoint para detección.** Se implementó un endpoint `/detect` que recibe la imagen subida y devuelve detecciones generadas mediante la librería simulada y posteriormente los endpoint para el procesamiento de imágenes con yolo y mediapipe aplicando patrón factory.
- **Crear interfaz mínima de aplicación.** Se desarrolló una pequeña UI con HTML y JavaScript para permitir al usuario subir imágenes, elegir un modelo disponible y visualizar el resultado generado.
- **Validación final y documentación del sprint.** Se revisaron todas las historias completadas, se generaron capturas de evidencia y se documentó el sprint.

Visión general del Sprint 3

- **Fechas:** 14/02 – 19/02
- **Objetivo del sprint:** *Definir un borrador de los requisitos funcionales, requisitos no funcionales y casos de uso del proyecto.*
- **Criterio de aceptación global:** tener suficientes requisitos y casos de uso para cubrir el funcionamiento total de la aplicación.

Como se observa en la Tabla A.3 y la Figura A.3, este sprint se centra en la definición de requisitos y casos de uso.

Descripción de las tareas del Sprint 3

Nada importante a destacar en este sprint.

Tarea	Pts
Definición alcance sprint	2
Plantear alcance de requisitos y casos de uso	6
Definir requisitos funcionales	8
Definir requisitos no funcionales	8
Definir casos de uso	16

Tabla A.3: Sprint 3 – Resumen

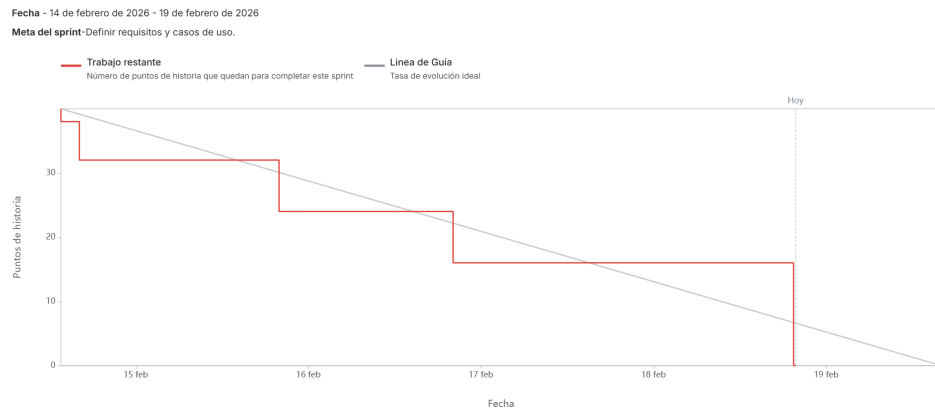


Figura A.3: Informe de trabajo completado Sprint 3

Visión general del Sprint 4

- **Fechas:** 19/02 – 25/02
- **Objetivo del sprint:** *Completar la documentación del MVP (RF/RNF/CU) y diseñar el modelo E-R de la aplicación (entidades, atributos, relaciones, normalización y reglas).*
- **Criterio de aceptación global:** disponer de la especificación de requisitos del MVP documentada y un modelo E-R completo y coherente (incluyendo normalización, integridad y restricciones de negocio).

Tal y como se muestra en la Tabla A.4 y la Figura A.4, este sprint se enfoca en el diseño del modelo de datos y la documentación del MVP.

Tarea	Pts
Corregir y subir documento “RF_RNF_CU_aplicación_completa”	2
Documentar los RF/RNF/CU para el producto mínimo viable	6
Modelo E-R (identificación de entidades y alcance)	8
Modelo E-R (atributos y claves)	8
Modelo E-R (relaciones y cardinalidades)	8
Modelo E-R (normalización, reglas de integridad y restricciones de negocio)	8

Tabla A.4: Sprint 4 – Resumen

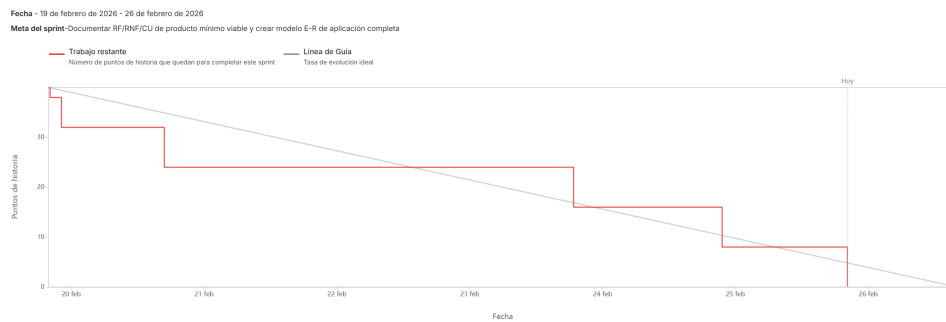


Figura A.4: Informe de trabajo completado Sprint 4

Descripción de las tareas del Sprint 4

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Corregir y subir documento “RF_RNF_CU_aplicación_completa”.**
Se revisó el documento general de requisitos y casos de uso de la aplicación completa, corrigiendo inconsistencias de redacción y estructura.
- **Documentar los RF/RNF/CU para el producto mínimo viable.**
Se seleccionaron y redactaron los requisitos funcionales y no funcionales junto con los casos de uso estrictamente necesarios para el MVP, priorizando el flujo básico de acceso, selección de IA, envío de imagen y obtención de resultados.
- **Modelo E-R (identificación de entidades y alcance).** Se identificaron las entidades completas de la aplicación y se delimitó el alcance del modelo para cubrir tanto el MVP como la aplicación completa.

- **Modelo E-R (atributos y claves).** Se definieron los atributos relevantes de cada entidad y sus claves, estableciendo claves primarias, restricciones de unicidad y una estructura consistente para su posterior representación en el diagrama.
- **Modelo E-R (relaciones y cardinalidades).** Se diseñó el diagrama E-R utilizando las entidades con sus atributos creados previamente junto definiendo relaciones y cardinalidades que unirían estas.
- **Modelo E-R (normalización, reglas de integridad y restricciones de negocio).** Se verificó el modelo respecto a formas normales (hasta 3FN) y se definieron reglas de integridad (entidad, referencial y coherencia semántica), además de restricciones de negocio (dominios de estado, coherencia temporal, unicidades y reglas específicas como renovaciones y consistencia IA-modo).

Visión general del Sprint 5

- **Fechas:** 03/03 – 05/03
- **Objetivo del sprint:** *Retroalimentación de diagramas E-R y Relacional.*
- **Criterio de aceptación global:** Tener correcciones hechas sobre los modelos E-R y Relacional de acuerdo a lo comentado en el Sprint Review.

Como se refleja en la Tabla A.5 y la Figura A.5, este sprint se centra en la mejora de los modelos E-R y relacional.

Tarea	Pts
Modificar modelo relacional	8
Modelo E-R	8
Repasar si E-R y relacional son válidos para casos de uso	8

Tabla A.5: Sprint 5 – Resumen

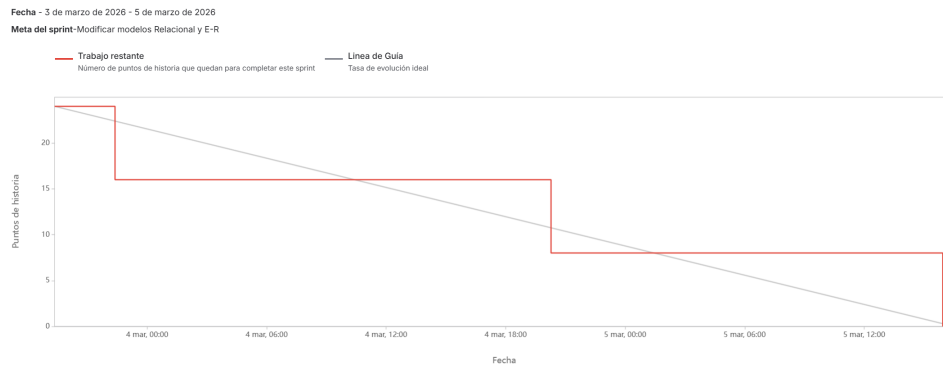


Figura A.5: Informe de trabajo completado Sprint 5

Descripción de las tareas del Sprint 5

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Modificar modelo relacional.** Hacer modificaciones en el modelo relacional para poder estar totalmente normalizado y tener una mejor estructura en cuanto a crecimiento de aplicación.
- **Modelo E-R.** Realizar el modelo Entidad Relación con la retrospectiva del Sprint Review.
- **Repasar si E-R y relacional son válidos para casos de uso.** Verificar si con ambos diagramas se podrían llegar a realizar correctamente todos los casos de usos planteados en anteriores sprints.

Visión general del Sprint 6

- **Fechas:** 07/03 – 12/03
- **Objetivo del sprint:** *Definir y ajustar el modelo de configuración y ejecución de pipelines.*
- **Criterio de aceptación global:** Tener definidos los criterios de configuración, creación de pipelines y ejecuciones, y reflejarlo en el modelo de datos.

Tal y como se muestra en la Tabla A.6 y la Figura A.6, este sprint introduce el modelo de pipelines y su configuración.

Tarea	Pts
Modificación E-R	4
Plantear si habrá configuración elegible por el usuario o no	6
Plantear si los pipelines los crea el administrador o usuarios	2
Plantear si se pueden repetir ejecuciones	2
Actualizar modelo respecto a los planteamientos tomados	10
Revisar si los requisitos y casos de uso cuadran con respecto a los planteamientos	8

Tabla A.6: Sprint 6 – Resumen

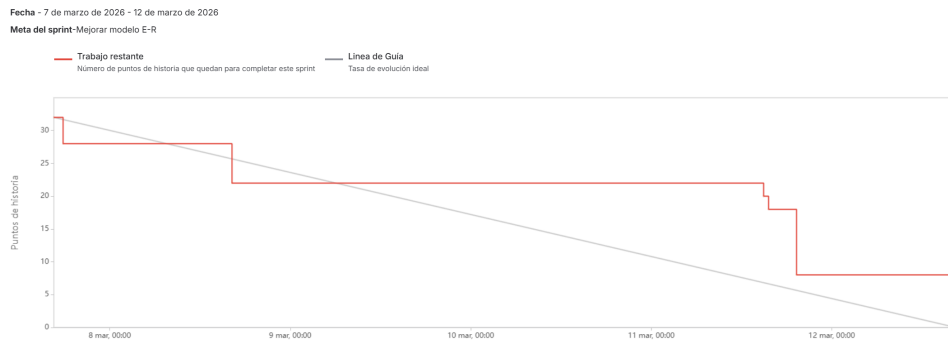


Figura A.6: Informe de trabajo completado Sprint 6

Descripción de las tareas del Sprint 6

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Modificación E-R.** Ajustar el diagrama E-R según los nuevos planteamientos.
- **Plantear si habrá configuración elegible por el usuario o no.** Analizar si los usuarios pueden modificar configuraciones.
- **Plantear si los pipelines los crea el administrador o usuarios.** Determinar si los pipelines los crea el administrador o los usuarios.
- **Plantear si se pueden repetir ejecuciones.** Estudiar si las ejecuciones pueden repetirse.
- **Actualizar modelo respecto a los planteamientos tomados.** Reflejar las decisiones tomadas en el modelo de datos.

- **Revisar si los requisitos y casos de uso cuadran con respecto a los planteamientos.** Comprobar coherencia entre requisitos, casos de uso y modelo.

Visión general del Sprint 7

- **Fechas:** 16/03 – 19/03
- **Objetivo del sprint:** *Preparar el entorno de desarrollo definitivo en Debian e iniciar la implementación real de la base de datos del sistema.*
- **Criterio de aceptación global:** Disponer de un entorno funcional en Debian con MariaDB operativo y haber comenzado la implementación física de la base de datos a partir de los diagramas diseñados.

Tal y como se muestra en la Tabla A.7 y la Figura A.7, este sprint se centra en la transición desde un entorno de desarrollo inicial hacia un entorno más realista y alineado con el despliegue final, así como en el inicio de la materialización del modelo de datos definido en fases anteriores.

Tarea	Pts
Crear Debian en dual boot con Windows	8
Preparar Debian con configuración mínima	4
Instalar MariaDB	4
Pruebas MariaDB	8
Empezar base de datos TFG	8

Tabla A.7: Sprint 7 – Resumen

Descripción de las tareas del Sprint 7

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Crear Debian en dual boot con Windows** Se realizó la instalación de Debian en configuración de arranque dual con Windows con el objetivo de disponer de un entorno de desarrollo más estable, controlado y alineado con el entorno de despliegue final en servidores Linux.

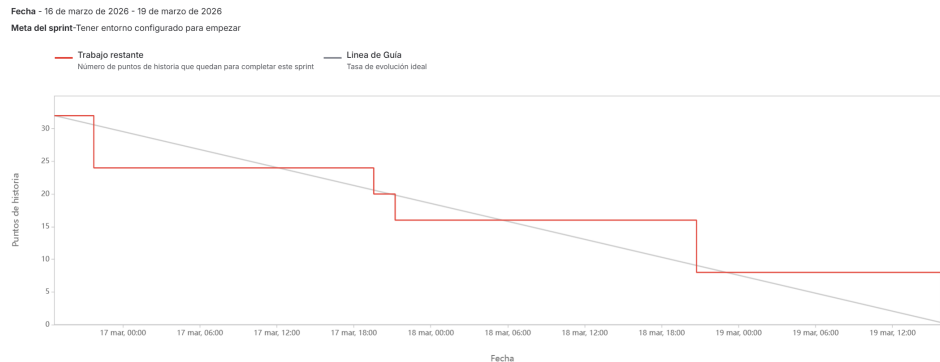


Figura A.7: Informe de trabajo completado Sprint 7

- **Preparar Debian con configuración mínima** Configuración inicial del sistema con las herramientas básicas necesarias para el desarrollo.
- **Instalar MariaDB** Instalación del sistema gestor de bases de datos que será utilizado en el proyecto.
- **Pruebas MariaDB** Verificación del correcto funcionamiento de MariaDB mediante pruebas básicas de conexión y ejecución de consultas.
- **Empezar base de datos TFG** Inicio de la implementación física de la base de datos, trasladando los diagramas entidad-relación y el modelo relacional definidos previamente a tablas reales en MariaDB.

Visión general del Sprint 8

- **Fechas:** 20/03 – 26/03
- **Objetivo del sprint:** *Ajustar el diseño del modelo de datos e implementar completamente la base de datos del sistema.*
- **Criterio de aceptación global:** Disponer de una base de datos completamente implementada en MariaDB y coherente con el modelo relacional actualizado.
- **Explicación sprint retrasado:** El sprint se vio retrasado por una baja médica, por lo que parte del trabajo previsto se desplazó temporalmente..

Tal y como se muestra en la Tabla A.8 y la Figura A.8, este sprint se centra en consolidar el diseño de datos mediante su ajuste durante la

implementación real, así como en completar la construcción de la base de datos del sistema.

Tarea	Pts
Actualizar modelo relacional	4
Implementar base de datos	12
Documentar base de datos	10
Documentar aspectos relevantes memoria	10
Añadir referencias a imágenes y tablas en anexos	4

Tabla A.8: Sprint 8 – Resumen

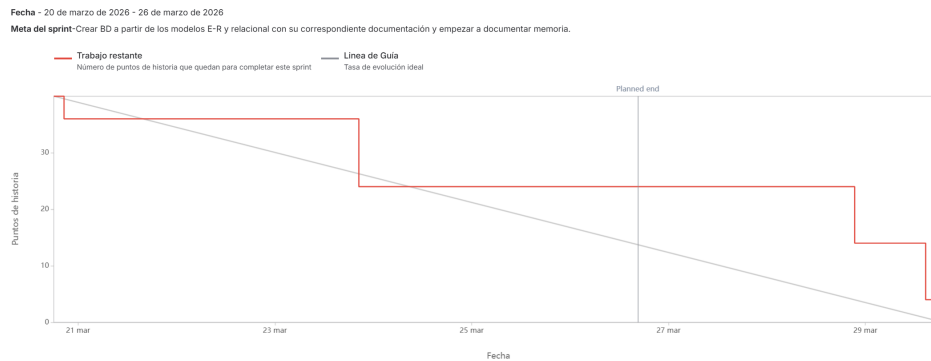


Figura A.8: Informe de trabajo completado Sprint 8

Descripción de las tareas del Sprint 8

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Actualizar modelo relacional** Se realizaron ajustes en el modelo relacional a partir de los cambios detectados durante la implementación de la base de datos.
- **Implementar base de datos** Construcción completa de la base de datos en MariaDB a partir del modelo relacional definido.
- **Documentar base de datos** Elaboración de la documentación asociada al diseño e implementación de la base de datos.

- **Documentar aspectos relevantes memoria** Redacción de los aspectos clave del desarrollo para su inclusión en la memoria del TFG.
- **Añadir referencias a imágenes y tablas en anexos** Incorporación de referencias cruzadas a figuras y tablas dentro de los anexos.

Visión general del Sprint 9

- **Fechas:** 06/04 – 09/04
- **Objetivo del sprint:** *Definir la estructura base del sistema bajo el patrón MVC e integrar los primeros componentes funcionales del backend y la interfaz.*
- **Criterio de aceptación global:** Disponer de una arquitectura MVC funcional con modelos de IA operativos, autenticación básica de usuarios y una interfaz mínima de interacción.

Tal y como se muestra en la Tabla A.9 y la Figura A.9, este sprint marca el inicio de la construcción de la aplicación como sistema estructurado, pasando de componentes aislados a una arquitectura organizada basada en el patrón Modelo-Vista-Controlador.

Tarea	Pts
Crear primera estructura modelo vista controlador	12
Poner en funcionamiento los modelos en el modelo	8
Poner en funcionamiento usuarios en el modelo	6
Crear interfaz mínima	6

Tabla A.9: Sprint 9 – Resumen

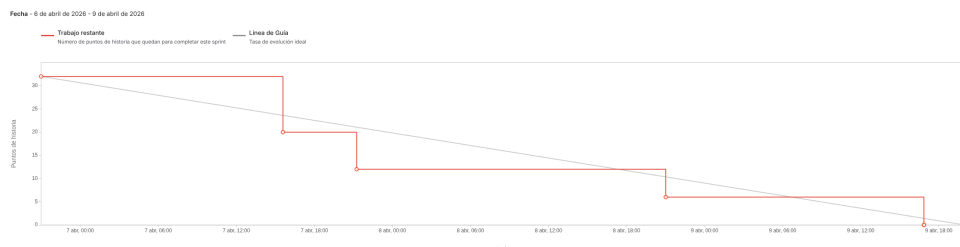


Figura A.9: Informe de trabajo completado Sprint 9

Descripción de las tareas del Sprint 9

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Crear primera estructura modelo vista controlador** Definición de la estructura base del sistema siguiendo el patrón MVC, estableciendo la organización inicial de carpetas, módulos y responsabilidades.
- **Poner en funcionamiento los modelos en el modelo** Integración de los modelos de IA dentro de la arquitectura MVC para su ejecución desde la lógica de negocio.
- **Poner en funcionamiento usuarios en el modelo** Implementación de un sistema básico de autenticación que permite el inicio de sesión de usuarios.
- **Crear interfaz mínima** Desarrollo de una interfaz sencilla que permite visualizar y probar el funcionamiento básico de la aplicación de forma gráfica.

Visión general del Sprint 10

- **Fechas:** 13/04 – 16/04
- **Objetivo del sprint:** *Mejorar la calidad de la documentación y avanzar en la implementación del sistema de autenticación en la interfaz.*
- **Criterio de aceptación global:** Disponer de una documentación más refinada y de un sistema de inicio de sesión funcional desde la interfaz gráfica con diferenciación básica de roles.

Tal y como se muestra en la Tabla A.10 y la Figura A.10, este sprint se centra en mejorar la calidad de la documentación del proyecto y en avanzar hacia una interacción más completa del usuario con la aplicación mediante autenticación desde la interfaz.

Descripción de las tareas del Sprint 10

A continuación se describen brevemente las tareas realizadas durante este sprint:

Tarea	Pts
Corregir problemas con documentación	3
Mejorar documentación técnicas y herramientas	5
Reducir aspectos relevantes de tamaño	8
Empezar con inicio sesión de usuarios	16

Tabla A.10: Sprint 10 – Resumen

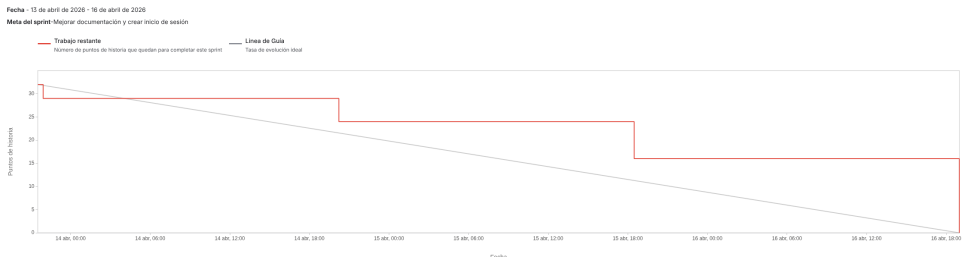


Figura A.10: Informe de trabajo completado Sprint 10

- **Corregir problemas con documentación** Corrección de errores de formato y estructura detectados en la documentación del proyecto.
- **Mejorar documentación técnicas y herramientas** Refinamiento de la sección de técnicas y herramientas utilizadas en el TFG.
- **Reducir aspectos relevantes de tamaño** Ajuste del contenido de la memoria para mejorar su claridad y adecuación al formato requerido.
- **Empezar con inicio sesión de usuarios** Implementación del inicio de sesión desde la interfaz gráfica, incluyendo la separación de vistas entre usuarios normales y administradores.

Visión general del Sprint 11

- **Fechas:** 24/04 – 29/04
- **Objetivo del sprint:** *Mejorar la documentación de sprints anteriores y comenzar a detallar la aplicación básica, incorporando persistencia de usuarios, ejecuciones y archivos.*
- **Criterio de aceptación global:** Disponer de una documentación de planificación más completa y de una base funcional capaz de registrar

usuarios, guardar ejecuciones en base de datos y gestionar los archivos generados durante el procesamiento.

Tal y como se muestra en la Tabla A.11 y la Figura A.11, este sprint se centra en completar partes pendientes de documentación y en avanzar en la aplicación básica, especialmente en la persistencia de usuarios, ejecuciones y archivos temporales. Durante el desarrollo se añadió además una nueva tarea, SCRUM-69 (Refactorización del orquestador para soporte Multi-Imagen), al detectarse que el guardado de ejecuciones en base de datos podía mejorarse si el orquestador soportaba correctamente salidas con varias imágenes.

Tarea	Pts
Mejorar documentación sprints pasados	5
Actualizar documentación base de datos	5
Hacer registro de usuarios	8
Crear script semilla para base de datos	6
Implementar guardado de ejecuciones en BD	8
Implementar gestión de archivos y limpieza	8
Refactorización del orquestador para soporte Multi-Imagen	8

Tabla A.11: Sprint 11 – Resumen

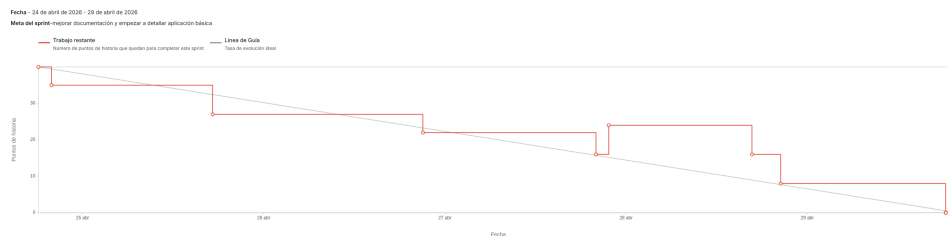


Figura A.11: Informe de trabajo completado Sprint 11

Descripción de las tareas del Sprint 11

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Mejorar documentación sprints pasados** Revisión de la documentación de los sprints anteriores para corregir descripciones incompletas, mejorar la coherencia entre tareas y dejar mejor reflejado el avance real del proyecto.

- **Actualizar documentación base de datos** Actualización de la documentación asociada al modelo de datos, incorporando los cambios realizados durante la implementación y ajustando la explicación de las tablas, relaciones y restricciones.
- **Hacer registro de usuarios** Implementación del alta de usuarios en la aplicación, permitiendo crear nuevas cuentas y almacenar sus datos de forma persistente en la base de datos.
- **Crear script semilla para base de datos** Creación de un script de carga inicial para insertar datos base en el sistema, como usuarios de prueba, roles, planes, modelos de IA, modos y pipelines iniciales.
- **Implementar guardado de ejecuciones en BD** Incorporación del registro de ejecuciones en base de datos, de forma que cada procesamiento realizado pueda quedar asociado a un usuario, un pipeline, su estado y la configuración aplicada.
- **Implementar gestión de archivos y limpieza** Implementación de la gestión de archivos generados durante las ejecuciones, incluyendo el registro de rutas temporales y la lógica necesaria para poder limpiar resultados caducados.
- **Refactorización del orquestador para soporte Multi-Imagen** Esta tarea se añadió durante el sprint al implementar el guardado de ejecuciones en base de datos. Al revisar cómo debían almacenarse los resultados, se detectó que algunos pipelines podían generar más de una imagen de salida, por ejemplo en procesos de recorte o análisis por sujetos. Por ello se refactorizó el orquestador para soportar correctamente salidas multi-imagen desde este punto del desarrollo, evitando dejar una estructura limitada que después habría sido más difícil de corregir.

Visión general del Sprint 12

- **Fechas:** 04/05 – 07/05
- **Objetivo del sprint:** *Continuar con el desarrollo de la aplicación, incorporando funcionalidades de usuario, gestión comercial e historial de ejecuciones.*
- **Criterio de aceptación global:** Disponer de una aplicación más completa, permitiendo al usuario modificar sus datos, alquilar modelos, cambiar de plan y consultar el historial de ejecuciones realizadas.

Tal y como se muestra en la Tabla A.12 y la Figura A.12, este sprint se centra en ampliar las funcionalidades disponibles para el usuario dentro de la aplicación. Tras haber avanzado en la autenticación, el registro de usuarios y el guardado de ejecuciones, en este sprint se incorporan opciones necesarias para que el sistema empiece a comportarse como una aplicación funcional y no sólo como una demo técnica.

Tarea	Pts
Crear opción para modificar datos de usuario	8
Crear opción para alquilar modelos	8
Crear opción para ver historial de ejecuciones	8
Crear opción para cambiar de plan	8

Tabla A.12: Sprint 12 – Resumen

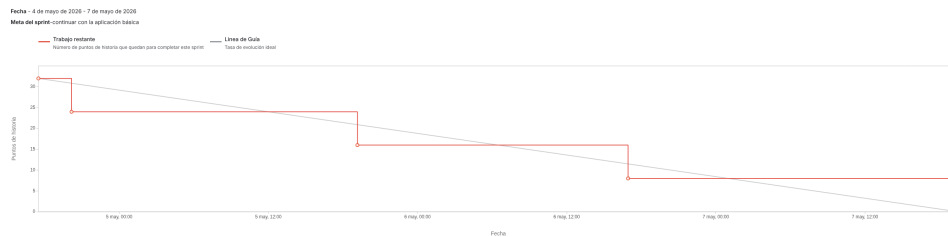


Figura A.12: Informe de trabajo completado Sprint 12

Descripción de las tareas del Sprint 12

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Crear opción para modificar datos de usuario** Implementación de la funcionalidad que permite al usuario consultar y modificar sus datos personales desde la aplicación, avanzando hacia una gestión básica de perfil.
- **Crear opción para alquilar modelos** Desarrollo de la opción para contratar modelos de IA de forma individual, permitiendo que los usuarios del plan básico puedan acceder a modelos concretos durante un periodo determinado.

- **Crear opción para ver historial de ejecuciones** Incorporación de una vista de historial donde el usuario puede consultar las ejecuciones realizadas, aprovechando el trabajo previo de persistencia de ejecuciones y gestión de archivos.
- **Crear opción para cambiar de plan** Implementación de la funcionalidad para cambiar el plan del usuario comprando una mejora, permitiendo diferenciar entre el acceso básico y el acceso Pro dentro del sistema.

Visión general del Sprint 13

- **Fechas:** 10/05 – 14/05
- **Objetivo del sprint:** *Corregir detalles de la aplicación básica, mejorar la estructura del código y comenzar con la creación de pruebas.*
- **Criterio de aceptación global:** Disponer de una aplicación básica más estable, con mejoras en la organización del código y con una primera batería de tests que permita comprobar el correcto funcionamiento de partes importantes del sistema.

Tal y como se muestra en la Tabla A.13 y la Figura A.13, este sprint se centra en estabilizar la aplicación tras la incorporación de las funcionalidades principales de usuario, alquiler de modelos, planes e historial. Además, se comienza a dar más peso a la calidad interna del proyecto mediante mejoras de código y creación de pruebas.

Tarea	Pts
Mejoras comentarios y estructura de código	8
Corregir detalles aplicación básica	16
Creación de tests	16

Tabla A.13: Sprint 13 – Resumen

Descripción de las tareas del Sprint 13

A continuación se describen brevemente las tareas realizadas durante este sprint:

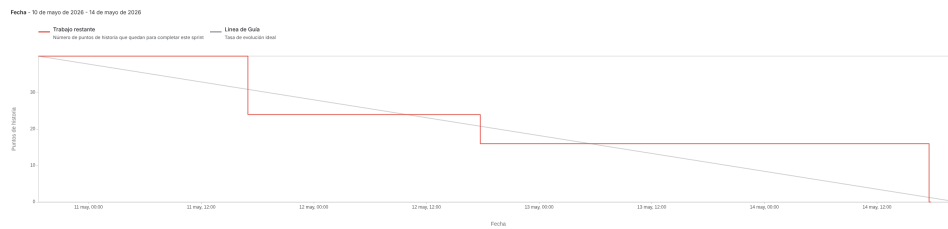


Figura A.13: Informe de trabajo completado Sprint 13

- **Mejoras comentarios y estructura de código** Revisión de distintas partes del código para mejorar su organización, aclarar comentarios y dejar una estructura más limpia de cara a futuras ampliaciones y mantenimiento.
- **Corregir detalles aplicación básica** Corrección de fallos y pequeños problemas detectados durante el uso de la aplicación básica, especialmente en las funcionalidades incorporadas en los sprints anteriores.
- **Creación de tests** Creación de pruebas y corrección de sus correspondientes fallos encontrados para validar partes importantes del sistema. Esta tarea permite comprobar de forma más controlada que los cambios realizados no rompen funcionalidades ya implementadas.

A.3. Estudio de viabilidad

Viabilidad económica

En este apartado se analiza el coste que habría tenido el desarrollo del proyecto si se hubiera realizado en un entorno empresarial real. Aunque el trabajo ha sido desarrollado como Trabajo de Fin de Grado, esta estimación permite valorar económicamente el esfuerzo invertido, los recursos utilizados y la posible recuperación de la inversión mediante el modelo de negocio planteado.

La aplicación se ha diseñado con una orientación comercial basada en el acceso a modelos de inteligencia artificial y a planes de servicio. En concreto, se plantea un precio de **9,99 €/mes** para el alquiler individual de cada modelo de IA y de **19,99 €/mes** para el plan Pro, que permite un acceso más amplio al catálogo disponible.

Costes

La estructura de costes se ha dividido en costes de personal, hardware, software y otros gastos asociados al desarrollo.

Costes de personal Para estimar el coste de personal se ha tomado como referencia el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública. En concreto, se ha considerado el perfil correspondiente al área de programación, grupo D, nivel 1, por ajustarse al tipo de trabajo realizado: desarrollo backend, diseño de base de datos, integración de modelos de inteligencia artificial, autenticación, control de acceso, pruebas y documentación técnica [10].

El salario anual bruto de referencia utilizado es de **20.436,01 €**. A partir de una jornada anual de **1.800 horas**, se obtiene el coste bruto por hora mediante una regla de tres:

$$\frac{20,436,01}{1,800} = 11,35$$

Por tanto, el coste bruto por hora es de **11,35 €/h**. Dado que se han dedicado aproximadamente **300 horas** al desarrollo del proyecto, el coste salarial bruto proporcional sería:

$$11,35 \times 300 = 3,406,00$$

El coste salarial bruto proporcional asciende, por tanto, a **3.406,00 €**.

Para calcular el coste real para la empresa no se utiliza el salario neto, sino el salario bruto más las cotizaciones empresariales correspondientes. El IRPF no se añade como coste empresarial adicional, ya que se trata de una retención practicada al trabajador sobre su salario bruto. En cambio, sí se tienen en cuenta las cotizaciones a cargo de la empresa.

En esta estimación se han considerado los siguientes conceptos empresariales: contingencias comunes, contingencias profesionales, desempleo, Fondo de Garantía Salarial, formación profesional y Mecanismo de Equidad Inter-generacional. Los porcentajes generales de cotización se han tomado de la información oficial de la Seguridad Social [9]. Para las contingencias profesionales se ha tomado como referencia la actividad de programación, consultoría y otras actividades relacionadas con la informática, correspondiente al CNAE 62 [12].

Concepto	Porcentaje	Coste
Salario bruto proporcional a 300 horas	–	3.406,00 €
Contingencias comunes	23,60 %	803,82 €
Contingencias profesionales	1,50 %	51,09 €
Desempleo	5,50 %	187,33 €
Fondo de Garantía Salarial	0,20 %	6,81 €
Formación profesional	0,60 %	20,44 €
Mecanismo de Equidad Intergeneracional	0,75 %	25,55 €
Total cotizaciones empresariales	32,15 %	1.095,03 €
Coste total de personal		4.501,03 €

Tabla A.14: Costes de personal

El desglose del coste de personal se muestra en la Tabla A.14.

De este modo, el coste de personal no se limita al salario bruto proporcional, sino que refleja el coste aproximado que asumiría una empresa para desarrollar el proyecto durante las 300 horas estimadas.

Costes de hardware Para el desarrollo se ha utilizado un ordenador portátil, dos pantallas externas, teclado y ratón. Estos dispositivos no se consumen íntegramente durante el proyecto, por lo que se ha calculado su amortización proporcional. Se ha considerado una vida útil de **4 años** y un periodo de uso de **9 meses**.

$$\text{Coste amortizado} = \text{Coste total} \times \frac{9}{48}$$

El desglose del hardware utilizado y su coste amortizado se muestra en la Tabla A.15.

Costes de software La mayor parte de las herramientas utilizadas durante el desarrollo son gratuitas o de código abierto. Es el caso de Python, Flask, MariaDB, Visual Studio Code, GitHub, Draw.io, DBeaver, L^AT_EX y las principales librerías empleadas en el proyecto.

No obstante, durante el desarrollo también se han utilizado herramientas de inteligencia artificial generativa como apoyo para la depuración de errores, revisión de código, contraste de decisiones técnicas y mejora de la

Concepto	Coste total	Coste amortizado
Ordenador portátil	1.000,00 €	187,50 €
Pantalla externa principal	300,00 €	56,25 €
Pantalla externa secundaria	200,00 €	37,50 €
Teclado	60,00 €	11,25 €
Ratón	70,00 €	13,13 €
Total	1.630,00 €	305,63 €

Tabla A.15: Costes de hardware

Concepto	Coste mensual	Coste total
ChatGPT Plus	21,99 €	197,91 €
Google AI Pro / Gemini	21,99 €	197,91 €
Herramientas libres y gratuitas	0,00 €	0,00 €
Total		395,82 €

Tabla A.16: Costes de software

documentación. Para esta estimación se han considerado las suscripciones utilizadas durante los nueve meses principales de trabajo.

El desglose de costes de software se recoge en la Tabla [A.16](#).

Costes varios También se han tenido en cuenta los costes derivados del uso de internet y electricidad durante el desarrollo. Para internet se ha considerado una tarifa mensual aproximada de **30 €** durante nueve meses. En el caso de la electricidad, se ha estimado un consumo medio de **180 W** para el equipo de trabajo completo, incluyendo portátil, pantallas y periféricos, durante las 300 horas de desarrollo.

$$0,18 \times 300 \times 0,20 = 10,80$$

El coste eléctrico estimado durante el desarrollo es de **10,80 €**. El resumen de estos gastos aparece en la Tabla [A.17](#).

No se ha incluido un coste específico de alojamiento en producción, ya que la entrega del proyecto se plantea como una aplicación reproducible en entorno local o de demostración. En caso de explotación real, habría que

Concepto	Coste
Internet	270,00 €
Electricidad	10,80 €
Total	280,80 €

Tabla A.17: Costes varios

Concepto	Coste
Personal	4.501,03 €
Hardware amortizado	305,63 €
Software	395,82 €
Varios	280,80 €
Total	5.483,28 €

Tabla A.18: Costes totales del proyecto

añadir costes de servidor, dominio, almacenamiento y mantenimiento, que dependerían del número de usuarios y del volumen de imágenes procesadas.

Costes totales

El coste total estimado del proyecto se obtiene sumando los costes de personal, hardware amortizado, software y gastos varios. Este resumen se muestra en la Tabla A.18.

Por tanto, el coste económico aproximado del desarrollo asciende a **5.483,28 €**. Esta cifra no representa un gasto real desembolsado íntegramente durante el TFG, sino una estimación del coste que tendría desarrollar una aplicación de estas características en un contexto profesional.

Beneficios

La aplicación desarrollada está pensada para poder generar ingresos mediante un modelo de acceso por servicios. Por un lado, los usuarios pueden alquilar modelos de inteligencia artificial de forma individual. Por otro, pueden contratar un plan Pro que habilite un acceso más completo a los modelos y pipelines disponibles.

La Tabla A.19 muestra el modelo de ingresos previsto para la aplicación.

Concepto	Precio
Alquiler mensual de un modelo IA	9,99 €/mes
Plan Pro	19,99 €/mes

Tabla A.19: Modelo de ingresos previsto

Vía de ingresos	Precio mensual	Mensualidades necesarias
Plan Pro	19,99 €	275
Alquiler individual de modelo IA	9,99 €	549

Tabla A.20: Estimación de recuperación de la inversión

A partir del coste total estimado, puede calcularse el número de mensualidades necesarias para recuperar la inversión inicial. Si únicamente se tuvieran en cuenta suscripciones Pro, sería necesario alcanzar aproximadamente:

$$\frac{5,483,28}{19,99} = 274,30$$

Es decir, serían necesarias unas **275 mensualidades Pro**. Esto podría equivaler, por ejemplo, a 55 usuarios Pro durante cinco meses o a unos 31 usuarios Pro durante nueve meses.

En el caso de que la recuperación se realizara únicamente mediante alquileres individuales de modelos, el cálculo sería:

$$\frac{5,483,28}{9,99} = 548,88$$

Por tanto, serían necesarias aproximadamente **549 mensualidades de alquiler individual** para cubrir el coste estimado del desarrollo.

La Tabla A.20 resume las mensualidades necesarias en cada una de las dos vías de ingresos planteadas.

Estos cálculos muestran que el proyecto podría ser económicamente viable si consiguiera una base moderada de usuarios recurrentes. Además, el diseño basado en modelos, modos y pipelines permite ampliar el catálogo sin rehacer la aplicación desde cero. Esta característica es importante desde

el punto de vista económico, ya que el coste de añadir nuevos servicios sería menor que el coste de desarrollo inicial.

Por ello, la rentabilidad potencial del proyecto no depende únicamente de la versión actual, sino también de la posibilidad de incorporar nuevos modelos de inteligencia artificial, nuevos modos de análisis y nuevos pipelines configurables que aumenten el valor ofrecido al usuario.

Viabilidad legal

En esta sección se analizan los aspectos legales más relevantes del proyecto. En concreto, se estudian las licencias de las dependencias utilizadas, la compatibilidad entre ellas, la licencia más adecuada para el código fuente desarrollado, la licencia de la documentación y las consideraciones relacionadas con la protección de datos personales.

Al tratarse de una aplicación web que integra librerías externas y modelos de inteligencia artificial, no basta con escoger una licencia de forma independiente. La licencia final del proyecto debe ser compatible con las licencias de las dependencias utilizadas y, especialmente, con aquellas que imponen obligaciones más restrictivas.

Software

En primer lugar, se analizan las dependencias principales utilizadas en el proyecto. Para ello se ha partido del entorno de ejecución empleado durante el desarrollo y se ha revisado el código fuente, ya que algunas dependencias se utilizan de forma directa aunque no aparezcan inicialmente como dependencia principal del entorno.

En particular, se ha añadido `SQLAlchemy` al análisis porque el proyecto lo utiliza directamente para ejecutar sentencias SQL auxiliares en el proceso de inicialización de la base de datos. Además, se ha separado el análisis de MediaPipe en dos partes: por un lado la biblioteca Python `mediapipe`, y por otro los modelos `.task` descargados y almacenados localmente en `models/mp`.

Las licencias se han consultado a partir de las fichas oficiales de PyPI, de la documentación de los proyectos y de las páginas de referencia sobre licencias libres [22, 23, 2, 5, 20, 24, 8, 1, 13, 21, 7, 3, 26, 27, 28, 6, 25, 18, 17]. La Tabla A.21 recoge las dependencias más relevantes del proyecto.

Como puede observarse en la Tabla A.21, la mayoría de dependencias utilizan licencias permisivas, como MIT, BSD o Apache-2.0. Estas licen-

Dependencia	Versión	Descripción	Licencia
Flask	3.0.3	Framework web utilizado para construir la aplicación y definir sus rutas.	BSD-3-Clause
Flask-SQLAlchemy	3.1.1	Extensión que integra SQLAlchemy con Flask para la gestión de la base de datos.	BSD License
SQLAlchemy	2.0.31	ORM y capa de abstracción de base de datos utilizada también de forma directa en el proyecto.	MIT
Flask-JWT-Extend	4.6.0	Extensión utilizada para la autenticación mediante JSON Web Tokens.	MIT
python-dotenv	1.0.1	Biblioteca utilizada para cargar variables de entorno desde ficheros <code>.env</code> .	BSD-3-Clause
Werkzeug	3.0.3	Biblioteca WSGI utilizada por Flask y empleada también para funciones de seguridad.	BSD-3-Clause
PyMySQL	1.1.1	Conector Python para bases de datos MySQL/MariaDB.	MIT
cryptography	42.0.8	Biblioteca criptográfica utilizada como soporte de seguridad y conexión.	Apache-2.0 / BSD-3-Clause
numpy	1.26.4	Biblioteca de cálculo numérico utilizada como soporte del procesamiento de imágenes.	BSD-3-Clause
opencv-python-headless	4.10.0.84	Versión de OpenCV para procesamiento de imágenes sin dependencias gráficas.	MIT / Apache-2.0 / LGPL-2.1
Pillow	10.4.0	Biblioteca de soporte para apertura y manipulación de imágenes.	HPND
mediapipe	0.10.14	Framework de Google utilizado para detección de manos y estimación de pose.	Apache-2.0
torch	2.3.1	Biblioteca de aprendizaje profundo utilizada como parte del ecosistema de visión artificial.	BSD-3-Clause
torchvision	0.18.1	Biblioteca complementaria de PyTorch para tareas de visión por computador.	BSD-3-Clause
ultralytics	8.2.50	Biblioteca utilizada para integrar YOLOv8 en la detección de objetos y recorte de personas.	AGPL-3.0 / empresarial
gunicorn	22.0.0	Servidor WSGI utilizado para ejecutar aplicaciones Python en despliegue.	MIT
pytest	8.2.2	Framework utilizado para la ejecución de pruebas automatizadas.	MIT

Tabla A.21: Dependencias del proyecto y licencias

cias permiten el uso, modificación y distribución del software con pocas restricciones, siempre que se mantengan los avisos de copyright y licencia correspondientes.

Sin embargo, la dependencia `ultralytics`, utilizada para integrar YOLOv8, introduce una restricción importante. Esta biblioteca se distribuye bajo licencia `AGPL-3.0`, salvo que se utilice una licencia empresarial específica. Por tanto, mientras el proyecto mantenga esta dependencia, no sería correcto licenciar todo el código bajo una licencia más permisiva como MIT o Apache-2.0 sin revisar previamente las obligaciones de `AGPL-3.0`.

Modelos y pesos utilizados

Además de las bibliotecas Python, el proyecto utiliza modelos preentrenados. Esta parte debe tratarse de forma separada porque los modelos no son exactamente código fuente propio, pero sí forman parte del funcionamiento real de la aplicación.

En el caso de MediaPipe, el código garantiza que los modelos utilizados de `hand_landmarker.task` y `pose_landmarker_full.task` estén disponibles en local. Si no existen en la carpeta `models/mp`, se descargan la primera vez desde los repositorios de Google y después se reutilizan localmente. Esta decisión evita depender de una descarga externa en cada ejecución y hace más controlable el comportamiento del sistema.

La Tabla [A.22](#) recoge los modelos y pesos principales considerados en el proyecto.

Tal y como se muestra en la Tabla [A.22](#), los modelos de MediaPipe no son el elemento que condiciona la licencia final del proyecto, ya que pertenecen al ecosistema MediaPipe y se documentan bajo condiciones compatibles con Apache-2.0 [[14](#), [15](#), [19](#)]. En cambio, los pesos y el uso de YOLOv8 mediante `ultralytics` sí deben tratarse junto con la licencia `AGPL-3.0` de dicha biblioteca [[28](#), [29](#)].

Compatibilidad de licencias

A la hora de escoger una licencia para el proyecto se ha buscado que sea lo menos restrictiva posible, pero siempre respetando las condiciones de las dependencias utilizadas.

Las licencias MIT, BSD, HPND y Apache-2.0 son licencias permisivas y compatibles con un proyecto que finalmente se distribuya bajo una licencia copyleft fuerte. Por tanto, no impiden utilizar `AGPL-3.0` como licencia final

Recurso	Uso en el proyecto	Licencia / condiciones
yolov8n.pt	Pesos de YOLOv8 utilizados por la biblioteca ultralytics para detección de objetos y recorte de personas.	AGPL-3.0 / licencia empresarial
hand_landmarker.task	Modelo de MediaPipe Tasks utilizado para detectar landmarks de manos.	Apache-2.0
pose_landmarker_full.task	Modelo de MediaPipe Tasks utilizado para estimar la pose humana.	Apache-2.0
Configuraciones JSON propias	Ficheros mp_manos.json, mp_pose.json, yolo_deteccion.json y yolo_recortes.json.	Propias del proyecto

Tabla A.22: Modelos y pesos utilizados

del código fuente. En cambio, la presencia de **ultralytics** sí condiciona la decisión, ya que AGPL-3.0 exige que el código fuente de las obras derivadas o integradas se ponga a disposición bajo la misma licencia cuando se distribuye el software o se ofrece como servicio de red [18, 17].

La Tabla A.23 resume el papel de cada familia de licencias dentro del proyecto.

Como se recoge en la Tabla A.23, la licencia que condiciona la decisión final es AGPL-3.0. Si en el futuro se quisiera publicar el proyecto bajo una licencia más permisiva, habría que sustituir **ultralytics** por una alternativa compatible con esa licencia o adquirir una licencia empresarial que permita su integración en productos cerrados o con licencias diferentes.

Licencia del código fuente

Teniendo en cuenta las dependencias utilizadas, la licencia más adecuada para el código fuente del proyecto es la **GNU Affero General Public License v3.0** (AGPL-3.0). Esta licencia permite el uso, modificación y distribución del software, pero exige que las modificaciones y trabajos derivados se mantengan bajo la misma licencia. Además, está especialmente pensada

Licencia	Uso en el proyecto	Compatibilidad
MIT	Presente en dependencias como <code>SQLAlchemy</code> , <code>PyMySQL</code> , <code>gunicorn</code> y <code>pytest</code> .	Compatible
BSD / BSD-3-Clause	Presente en <code>Flask</code> , <code>Werkzeug</code> , <code>NumPy</code> , <code>PyTorch</code> y otras dependencias.	Compatible
Apache-2.0	Presente en <code>MediaPipe</code> y en componentes del ecosistema <code>OpenCV</code> .	Compatible
HPND	Presente en <code>Pillow</code> .	Compatible
LGPL-2.1	Presente en componentes binarios asociados a <code>FFmpeg</code> dentro de las ruedas de <code>OpenCV</code> .	Compatible con uso como librería
AGPL-3.0	Presente en <code>Ultralytics/YOLOv8</code> . Condiciona la licencia final del código fuente.	Licencia dominante

Tabla A.23: Compatibilidad de licencias

para aplicaciones que se ofrecen a través de red, por lo que encaja con una aplicación web.

La elección de AGPL-3.0 resulta coherente con el proyecto por dos motivos. En primer lugar, permite cumplir con la dependencia más restrictiva utilizada, que es `ultralytics`. En segundo lugar, protege el carácter abierto del proyecto si un tercero modifica la aplicación y la ofrece como servicio web.

La Tabla A.24 resume los aspectos principales de la licencia AGPL-3.0.

Por todo ello, el código fuente de la aplicación se licenciaría bajo AGPL-3.0. Esta opción no impide una posible explotación económica del proyecto, ya que el modelo de negocio planteado se basa en el acceso a modelos y planes de servicio. No obstante, si se quisiera desarrollar una versión comercial cerrada, habría que revisar especialmente la dependencia con `ultralytics` y valorar la adquisición de una licencia empresarial.

Documentación

Aunque podría utilizarse la misma licencia que para el código fuente, no resulta lo más adecuado aplicar AGPL-3.0 a la documentación. Esta licencia

Derechos	Condiciones	Limitaciones
Uso comercial	Mantener avisos de copyright y licencia	Sin garantía
Distribución	Liberar el código fuente bajo la misma licencia	Limitación de responsabilidad
Modificación	Indicar los cambios realizados	No concede uso de marcas
Uso como servicio web	Hacer disponible el código fuente de la versión ofrecida por red	Copyleft fuerte
Uso de patentes	Mantener las condiciones de la licencia	Obligaciones sobre obras derivadas

Tabla A.24: Resumen de la licencia AGPL-3.0

Derechos	Condiciones	Limitaciones
Uso comercial	Reconocimiento de autoría	Sin garantía
Distribución	Indicar cambios realizados si los hubiera	No concede derechos sobre marcas
Modificación	Mantener referencia a la licencia	No concede derechos sobre patentes
Uso privado	No aplicar restricciones adicionales incompatibles	Limitación de responsabilidad

Tabla A.25: Resumen de la licencia CC-BY-4.0

está pensada principalmente para software y contiene obligaciones que no encajan tan bien con textos, diagramas o material explicativo.

Por ello, para la documentación del proyecto se propone utilizar una licencia **Creative Commons Attribution 4.0 International** (CC-BY-4.0). Esta licencia permite copiar, distribuir y modificar la documentación, incluso con fines comerciales, siempre que se reconozca la autoría original [4].

La Tabla A.25 resume los aspectos principales de la licencia CC-BY-4.0.

De esta forma, la memoria, los anexos, los diagramas y el material escrito del proyecto pueden distribuirse de manera más natural que si se empleara una licencia diseñada para código fuente.

Imágenes, recursos y resultados generados

Las capturas, diagramas y recursos gráficos creados específicamente para la memoria y los anexos se consideran material propio del proyecto. Por coherencia con la documentación, se propone licenciarlos también bajo CC-BY-4.0, salvo aquellos recursos que procedan de herramientas o fuentes externas y que mantengan su propia licencia.

En cuanto a las imágenes subidas por los usuarios a la aplicación, no se consideran parte del código fuente ni de la documentación del proyecto. Estas imágenes pertenecen al usuario que las aporta y la aplicación solo las trata para ejecutar el procesamiento solicitado. Por este motivo, el sistema debe evitar conservarlas indefinidamente y debe tratarlas como archivos temporales asociados a una ejecución.

Los archivos generados por la aplicación, como imágenes procesadas o ficheros JSON de resultados, son salidas derivadas del uso del sistema. En una explotación real, las condiciones de uso deberían indicar expresamente si estos resultados pertenecen al usuario, al proveedor del servicio o si se someten a alguna licencia concreta. Para este TFG se considera que dichos resultados se generan únicamente como evidencia del procesamiento y no se redistribuyen como un conjunto de datos propio.

Protección de datos personales

Además de las licencias de software, el proyecto debe tener en cuenta la normativa sobre protección de datos personales, especialmente el Reglamento General de Protección de Datos y la Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales [16, 11].

La aplicación gestiona cuentas de usuario, correos electrónicos, nombres de usuario, contraseñas cifradas mediante hash, suscripciones, alquileres de modelos y registros de ejecuciones. Por tanto, estos datos deben tratarse de acuerdo con los principios de minimización, limitación del plazo de conservación, confidencialidad y control por parte del usuario.

Desde el punto de vista del diseño, se han incorporado varias decisiones orientadas a reducir riesgos:

- Las contraseñas no se almacenan en texto plano, sino mediante hash.
- Las imágenes y resultados se gestionan como archivos temporales.
- El acceso a resultados se realiza mediante tokens de descarga.

- Las ejecuciones quedan asociadas al usuario autenticado, evitando accesos cruzados entre usuarios.
- Se contempla la baja de cuenta y la posterior anonimización de datos personales.

En relación con el derecho de supresión o derecho al olvido, el sistema contempla la posibilidad de que un usuario solicite la baja de su cuenta. Al hacerlo, se marca la cuenta como borrada, se revocan sus servicios activos y se registra la fecha de baja. Posteriormente, el proceso de mantenimiento puede anonimizar la identidad del usuario, manteniendo únicamente aquellos registros que deban conservarse por razones técnicas, históricas o de integridad del sistema.

Esta decisión permite aproximar el comportamiento de la aplicación a los principios de minimización, limitación del plazo de conservación y control del usuario sobre sus datos. En una explotación real, sería necesario completar esta parte con una política de privacidad, condiciones de uso, información sobre el responsable del tratamiento, finalidad del tratamiento, base jurídica, plazo de conservación y procedimiento formal para ejercer derechos.

Resumen de licencias del proyecto

La Tabla [A.26](#) recoge la propuesta final de licenciamiento del proyecto.

En conclusión, la viabilidad legal del proyecto es positiva siempre que se respeten las condiciones de las dependencias empleadas. La presencia de **ultralitics** obliga a adoptar una postura conservadora y licenciar el código bajo **AGPL-3.0**, salvo que se sustituya dicha dependencia o se obtenga una licencia empresarial. Para la documentación y recursos propios no software se propone **CC-BY-4.0**, por ser una licencia más adecuada para textos, diagramas e imágenes.

Esta licencia no impide monetizar el servicio mediante alquileres de modelos o planes de suscripción, pero sí evita una explotación cerrada del código. Si en el futuro se quisiera distribuir o explotar la aplicación como producto propietario, habría que sustituir la dependencia **ultralitics** por otra alternativa compatible o adquirir una licencia empresarial.

Recurso	Licencia propuesta
Código fuente de la aplicación	AGPL-3.0
Código fuente de pruebas	AGPL-3.0
Configuraciones propias del proyecto	AGPL-3.0, al formar parte del funcionamiento de la aplicación
Modelos de MediaPipe utilizados localmente	Apache-2.0, según condiciones del ecosistema MediaPipe
Modelo YOLOv8 utilizado mediante Ultralytics	AGPL-3.0 / licencia empresarial
Documentación del proyecto	CC-BY-4.0
Diagramas e imágenes propias de la documentación	CC-BY-4.0
Imágenes subidas por usuarios	Propiedad del usuario que las aporta
Resultados generados por la aplicación	Según condiciones de uso del servicio

Tabla A.26: Resumen de licencias del proyecto

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

Este apéndice recoge los requisitos y casos de uso del sistema desarrollado. La aplicación se centra en permitir a usuarios autenticados acceder a una plataforma web de procesamiento de imágenes mediante modelos de inteligencia artificial, organizados en forma de *pipelines* configurables.

El sistema contempla distintos perfiles de acceso: usuarios con plan Básico, usuarios con plan Pro y usuarios administradores. Los usuarios pueden seleccionar un pipeline disponible según sus permisos, subir una imagen, ejecutar el procesamiento, visualizar los resultados generados y acceder temporalmente a los archivos de salida. Además, la aplicación incorpora funcionalidades de gestión de cuenta, contratación de planes o modelos individuales, consulta de ejecuciones realizadas y administración básica del sistema.

Los objetivos generales del sistema son los siguientes:

B.2. Objetivos generales

- Permitir el acceso de usuarios autenticados a una plataforma web de procesamiento de imágenes mediante modelos de inteligencia artificial.
- Diferenciar entre usuarios de diferentes planes y usuarios con rol de Administrador.

- Ofrecer un flujo completo de uso: registro o inicio de sesión, consulta de servicios, selección de pipeline, configuración de la ejecución, subida de imagen, procesamiento, visualización de resultados y descarga temporal de archivos.
- Aplicar control de acceso en el servidor para impedir que un usuario ejecute pipelines que requieran modelos no contratados o no disponibles para su plan.
- Permitir la contratación de planes y modelos individuales, diferenciando entre acceso completo mediante plan Pro y acceso concreto mediante alquiler de modelos.
- Registrar las ejecuciones realizadas, manteniendo trazabilidad básica sobre el usuario, el pipeline ejecutado, el estado de la operación, la duración, los posibles errores y la configuración aplicada.
- Gestionar los archivos generados como recursos temporales, evitando su conservación indefinida y controlando su acceso mediante identificadores de descarga.
- Facilitar la administración del sistema mediante un panel específico para revisar usuarios, estados, ejecuciones y pipelines.
- Diseñar una base funcional ampliable, de forma que puedan añadirse nuevos modelos, modos o pipelines sin rehacer la estructura general de la aplicación.

B.3. Catálogo de requisitos

Requisitos funcionales (MVP)

RF-1 Gestión de cuentas de usuario

La aplicación debe permitir la creación, acceso, modificación y baja lógica de cuentas de usuario.

- RF-1.1 Registrar un nuevo usuario.
- RF-1.2 Iniciar sesión mediante credenciales válidas.
- RF-1.3 Cerrar sesión desde la interfaz.
- RF-1.4 Consultar los datos del perfil.

- RF-1.5 Modificar los datos básicos de la cuenta.
- RF-1.6 Cambiar la contraseña verificando previamente la contraseña actual.
- RF-1.7 Solicitar la baja de la cuenta.

RF-2 Autenticación, roles y permisos

La aplicación debe proteger las operaciones principales mediante autenticación y aplicar permisos según el rol y el estado del usuario.

- RF-2.1 Generar un token de acceso tras un inicio de sesión correcto.
- RF-2.2 Proteger las rutas privadas para impedir accesos no autenticados.
- RF-2.3 Distinguir entre usuario normal y usuario administrador.
- RF-2.4 Impedir el acceso a cuentas suspendidas o dadas de baja.
- RF-2.5 Restringir las acciones administrativas a usuarios con rol de Administrador.

RF-3 Planes, suscripciones y contratación individual

La aplicación debe permitir consultar y contratar servicios que determinan el acceso a modelos y pipelines.

- RF-3.1 Consultar los planes disponibles.
- RF-3.2 Suscribirse a un plan.
- RF-3.3 Consultar los modelos de IA disponibles para contratación individual.
- RF-3.4 Alquilar un modelo de IA de forma individual.
- RF-3.5 Consultar compras, alquileres y suscripciones activas.
- RF-3.6 Activar de forma inmediata un servicio programado cuando proceda.
- RF-3.7 Modificar la renovación automática de planes o modelos contratados.

RF-4 Catálogo de modelos, modos y pipelines

La aplicación debe ofrecer al usuario un catálogo de pipelines ejecutables según los modelos disponibles para su cuenta.

- RF-4.1 Registrar modelos de IA disponibles en el sistema.
- RF-4.2 Asociar a cada modelo distintos modos de ejecución.
- RF-4.3 Definir pipelines formados por una o varias etapas ordenadas.
- RF-4.4 Mostrar al usuario únicamente los pipelines que puede ejecutar.
- RF-4.5 Informar al usuario cuando no tenga acceso a ningún pipeline.
- RF-4.6 Mostrar una guía de compra que ayude a identificar qué servicios son necesarios para ejecutar determinados pipelines.

RF-5 Configuración de pipelines

La aplicación debe permitir ejecutar pipelines con una configuración predefinida y, cuando sea necesario, modificar parámetros antes del procesamiento.

- RF-5.1 Consultar la configuración predeterminada de un pipeline.
- RF-5.2 Mostrar la configuración de forma editable en la interfaz.
- RF-5.3 Permitir al usuario modificar parámetros de ejecución en formato JSON.
- RF-5.4 Validar que la configuración personalizada tenga un formato correcto.
- RF-5.5 Registrar la configuración aplicada a la ejecución cuando sea distinta a la predeterminada.

RF-6 Procesamiento de imágenes mediante pipelines

La aplicación debe ejecutar el procesamiento de una imagen siguiendo las etapas del pipeline seleccionado.

- RF-6.1 Subir una imagen desde el panel principal.
- RF-6.2 Validar el archivo recibido antes de procesarlo.

- RF-6.3 Comprobar que el usuario tiene permiso para ejecutar el pipeline seleccionado.
- RF-6.4 Ejecutar las etapas del pipeline en el orden definido.
- RF-6.5 Encadenar la salida de una etapa como entrada de la siguiente cuando proceda.
- RF-6.6 Permitir pipelines con una única etapa o con varias etapas.
- RF-6.7 Generar resultados visuales y datos estructurados de cada etapa.

RF-7 Visualización y descarga de resultados

La aplicación debe mostrar los resultados de una ejecución de forma visual además de permitir su descarga mientras estén disponibles.

- RF-7.1 Mostrar las imágenes generadas por cada etapa del pipeline.
- RF-7.2 Mostrar los datos JSON asociados a cada etapa.
- RF-7.3 Permitir la descarga de imágenes procesadas.
- RF-7.4 Permitir la descarga de resultados JSON.
- RF-7.5 Impedir la descarga de archivos inexistentes, caducados o no autorizados.

RF-8 Historial de ejecuciones

La aplicación debe permitir al usuario consultar las ejecuciones realizadas desde su cuenta.

- RF-8.1 Consultar el historial de ejecuciones del usuario autenticado.
- RF-8.2 Mostrar el pipeline ejecutado, fecha, estado, duración y posibles errores.
- RF-8.3 Indicar si una ejecución conserva archivos temporales disponibles.
- RF-8.4 Consultar el detalle de una ejecución concreta.
- RF-8.5 Impedir que un usuario acceda a ejecuciones de otros usuarios.

RF-9 Gestión de archivos temporales

La aplicación debe gestionar los archivos de entrada, salida e intermedios como recursos temporales asociados a una ejecución.

- RF-9.1 Registrar los archivos temporales asociados a una ejecución.
- RF-9.2 Asociar tokens de descarga a los archivos que deban ser accesibles desde la interfaz.
- RF-9.3 Definir una fecha de expiración para los archivos temporales.
- RF-9.4 Eliminar o invalidar archivos temporales caducados.
- RF-9.5 Evitar que la aplicación acumule indefinidamente imágenes procesadas.

RF-10 Administración de usuarios

La aplicación debe permitir al administrador consultar y gestionar el estado de las cuentas de usuario.

- RF-10.1 Acceder a un panel de administración.
- RF-10.2 Consultar el listado de usuarios.
- RF-10.3 Ver información básica de cada usuario.
- RF-10.4 Cambiar el estado de una cuenta cuando sea necesario.
- RF-10.5 Impedir que un administrador modifique accidentalmente el estado de su propia cuenta.

RF-11 Administración de pipelines

La aplicación debe permitir al administrador gestionar los pipelines disponibles en el sistema sin modificar directamente el código fuente.

- RF-11.1 Consultar los pipelines existentes.
- RF-11.2 Crear nuevos pipelines.
- RF-11.3 Definir las etapas de un pipeline.
- RF-11.4 Modificar la información y composición de un pipeline.
- RF-11.5 Deshabilitar o eliminar pipelines cuando sea posible.

- RF-11.6 Evitar la eliminación física de pipelines con ejecuciones asociadas, conservando la coherencia del historial.

RF-12 Auditoría y mantenimiento del sistema

La aplicación debe incorporar funciones de revisión y mantenimiento que permitan conservar un estado coherente.

- RF-12.1 Consultar ejecuciones globales desde el panel de administración.
- RF-12.2 Renovar servicios activos cuando tengan renovación automática.
- RF-12.3 Desactivar servicios caducados cuando no deban renovarse.
- RF-12.4 Anonimizar cuentas dadas de baja tras el periodo previsto.
- RF-12.5 Registrar errores de ejecución de forma comprensible para el usuario.

RF-13 Interfaz web

La aplicación debe disponer de una interfaz web que permita utilizar las funciones principales sin acceder directamente a la API.

- RF-13.1 Disponer de página pública de inicio.
- RF-13.2 Disponer de pantallas de registro e inicio de sesión.
- RF-13.3 Disponer de panel principal de usuario.
- RF-13.4 Disponer de página de perfil.
- RF-13.5 Disponer de tienda y página de compras.
- RF-13.6 Disponer de guía de compra.
- RF-13.7 Disponer de historial y vista de resultados.
- RF-13.8 Disponer de panel de administración.

Requisitos no funcionales (MVP)

RNF-1 Usabilidad

La interfaz debe ser clara y permitir que un usuario pueda ejecutar el flujo principal sin conocer la estructura interna del sistema.

RNF-2 Seguridad

Las operaciones privadas deben requerir autenticación y el servidor debe verificar los permisos antes de ejecutar acciones sensibles.

RNF-3 Privacidad

Las imágenes y resultados generados deben tratarse como recursos temporales, evitando su almacenamiento indefinido y controlando el acceso mediante tokens.

RNF-4 Rendimiento

El procesamiento debe ejecutarse en tiempos razonables para imágenes de tamaño habitual, evitando operaciones innecesarias cuando el usuario no tenga permisos o el archivo no sea válido.

RNF-5 Mantenibilidad

El sistema debe organizarse de forma modular, separando controladores, modelos, servicios, plantillas y lógica específica de inteligencia artificial.

RNF-6 Escalabilidad funcional

La arquitectura debe permitir añadir nuevos modelos, modos y pipelines con cambios reducidos sobre la estructura existente.

RNF-7 Trazabilidad

El sistema debe conservar información básica sobre ejecuciones y cambios relevantes, de forma que sea posible reconstruir el uso principal de la aplicación.

RNF-8 Portabilidad

La aplicación debe poder ejecutarse en un entorno de desarrollo o despliegue documentado, con configuración centralizada y dependencias identificadas.

RNF-9 Robustez

El sistema debe responder de forma controlada ante errores de entrada, configuraciones incorrectas, archivos no válidos, servicios caducados o fallos internos.

RNF-10 Comprobabilidad

Las partes principales del sistema deben poder verificarse mediante pruebas automatizadas, especialmente autenticación, permisos, tienda, ejecución de pipelines y procesamiento de imágenes.

B.4. Especificación de requisitos

En esta sección se recogen los casos de uso principales del sistema. Con el fin de ofrecer una visión general previa a su descripción detallada, en la Figura B.1 se muestra el diagrama de casos de uso de la aplicación. Este diagrama permite identificar de forma visual los actores que intervienen en el sistema y las funcionalidades más relevantes asociadas a cada uno de ellos.

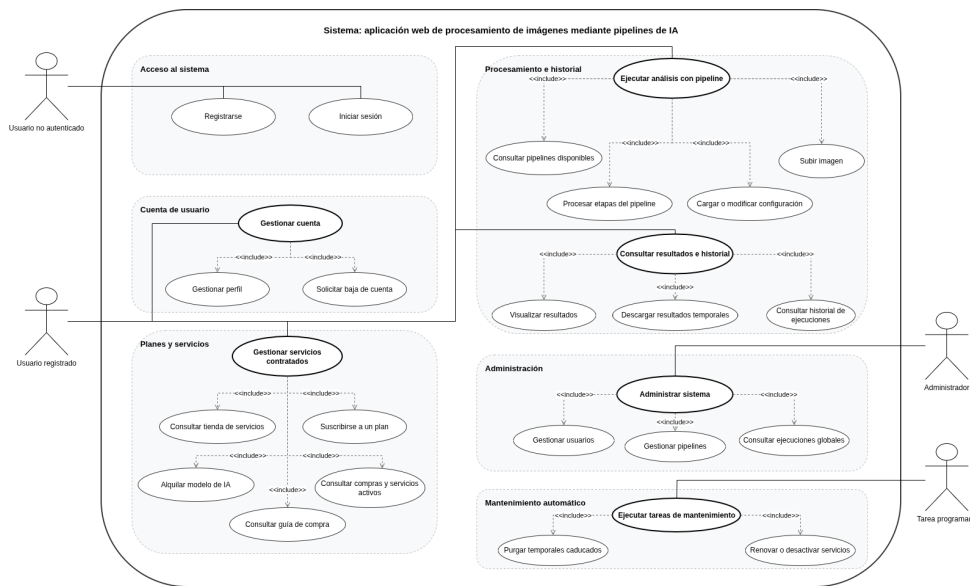


Figura B.1: Diagrama de casos de uso del sistema

El diagrama diferencia cuatro actores principales: el usuario no autenticado, el usuario registrado, el administrador y las tareas programadas del sistema. El usuario no autenticado puede registrarse e iniciar sesión. Una vez autenticado, el usuario registrado puede gestionar su cuenta, consultar los servicios disponibles, contratar planes o modelos de inteligencia artificial, ejecutar pipelines sobre imágenes, visualizar resultados y consultar su historial de ejecuciones.

Por otro lado, el administrador dispone de funcionalidades específicas de gestión, entre las que se encuentran la administración de usuarios, la gestión de pipelines y la consulta global de ejecuciones. Finalmente, el actor correspondiente a las tareas programadas representa los procesos automáticos de mantenimiento, encargados de depurar archivos temporales caducados y revisar la renovación o desactivación de servicios.

A continuación, se describen de forma individual los casos de uso principales del sistema..

CU-1	Registrarse
Versión	1.0
Actor	Usuario no autenticado
Requisitos asociados	RF-1.1, RF-13.2
Descripción	Creación de una cuenta de usuario para acceder a las funciones privadas de la aplicación.
Precondición	El usuario no dispone de una cuenta registrada con el mismo correo o nombre de usuario.
Acciones	1. El usuario accede a la pantalla de registro. 2. Introduce nombre de usuario, correo, contraseña y datos requeridos. 3. El sistema valida que los campos sean correctos. 4. El sistema comprueba que no exista una cuenta con los mismos datos únicos. 5. El sistema crea la cuenta y asigna el rol de usuario. 6. El sistema informa de que el registro se ha completado correctamente.
Postcondición	La cuenta queda creada y puede utilizarse para iniciar sesión.
Excepciones	Correo duplicado, nombre de usuario duplicado, contraseña no válida o datos incompletos.
Importancia	Alta

Tabla B.1: CU-1 Registrarse.

CU-2	Iniciar sesión
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-1.2, RF-2.1, RF-2.2, RF-13.2
Descripción	Acceso del usuario a la aplicación mediante credenciales válidas.
Precondición	El usuario dispone de una cuenta activa.
Acciones	1. El usuario accede a la pantalla de inicio de sesión. 2. Introduce su identificador y contraseña. 3. El sistema valida las credenciales. 4. El sistema comprueba que la cuenta esté activa. 5. El sistema genera el token de acceso y guarda la información necesaria para la sesión. 6. El usuario accede al panel correspondiente según su rol.
Postcondición	El usuario queda autenticado en el sistema.
Excepciones	Credenciales incorrectas, usuario inexistente o cuenta suspendida.
Importancia	Alta

Tabla B.2: CU-2 Iniciar sesión.

CU-3	Gestionar perfil
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-1.4, RF-1.5, RF-1.6, RF-13.4
Descripción	Consulta y modificación de los datos básicos de la cuenta.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario accede a la página de perfil. 2. El sistema muestra los datos asociados a la cuenta. 3. El usuario modifica los campos permitidos. 4. Si desea cambiar la contraseña, introduce también la contraseña actual. 5. El sistema valida los datos y aplica los cambios. 6. El sistema informa del resultado de la operación.
Postcondición	Los datos de la cuenta quedan actualizados.
Excepciones	Contraseña actual incorrecta, nueva contraseña no válida o error de persistencia.
Importancia	Media

Tabla B.3: CU-3 Gestionar perfil.

CU-4	Solicitar baja de cuenta
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-1.7, RF-12.4
Descripción	Baja lógica de una cuenta de usuario y desactivación de sus servicios activos.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario solicita la baja de su cuenta. 2. El sistema localiza la cuenta autenticada. 3. El sistema marca la cuenta como borrada. 4. El sistema desactiva sus suscripciones y alquileres activos. 5. El sistema registra la fecha de baja para su posterior tratamiento.
Postcondición	La cuenta queda dada de baja y sus servicios dejan de estar activos.
Excepciones	Identidad no localizada o error al aplicar la baja.
Importancia	Media

Tabla B.4: CU-4 Solicitar baja de cuenta.

CU-5	Consultar tienda de servicios
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-3.1, RF-3.3, RF-13.5
Descripción	Consulta de planes y modelos disponibles para contratación.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario accede a la tienda. 2. El sistema consulta los planes disponibles. 3. El sistema consulta los modelos de IA habilitados. 4. El sistema muestra el estado de cada servicio respecto al usuario.
Postcondición	El usuario puede decidir qué plan o modelo contratar.
Excepciones	Error al recuperar el catálogo de servicios.
Importancia	Alta

Tabla B.5: CU-5 Consultar tienda de servicios.

CU-6	Suscribirse a un plan
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-3.2, RF-3.6, RF-3.7
Descripción	Contratación de un plan de servicio, como el plan Pro, para ampliar el acceso a modelos y pipelines.
Precondición	El usuario ha iniciado sesión y el plan está disponible.
Acciones	1. El usuario selecciona un plan. 2. El sistema comprueba que el plan existe y está habilitado. 3. El usuario confirma la suscripción. 4. El sistema registra la suscripción y sus fechas de vigencia. 5. El sistema actualiza el estado de acceso del usuario.
Postcondición	El usuario dispone del plan contratado.
Excepciones	Plan inexistente, plan no disponible o error al registrar la suscripción.
Importancia	Alta

Tabla B.6: CU-6 Suscribirse a un plan.

CU-7	Alquilar un modelo de IA
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-3.3, RF-3.4, RF-3.6, RF-3.7
Descripción	Contratación individual de un modelo de IA para permitir la ejecución de pipelines que dependan de él.
Precondición	El usuario ha iniciado sesión y el modelo está habilitado.
Acciones	1. El usuario selecciona un modelo de IA disponible. 2. El sistema comprueba que el modelo existe y está habilitado. 3. El sistema comprueba si el usuario ya tiene un acceso activo. 4. El usuario confirma la contratación. 5. El sistema registra el alquiler y su periodo de vigencia.
Postcondición	El usuario puede acceder a pipelines que utilicen el modelo contratado, siempre que se cumplan el resto de condiciones.
Excepciones	Modelo inexistente, modelo ya contratado, fecha inválida o error al registrar la operación.
Importancia	Alta

Tabla B.7: CU-7 Alquilar un modelo de IA.

CU-8	Consultar compras y servicios activos
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-3.5, RF-3.6, RF-3.7, RF-13.5
Descripción	Consulta de planes, alquileres y servicios activos o programados del usuario.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario accede a la página de compras. 2. El sistema recupera sus suscripciones y alquileres. 3. El sistema muestra fechas, estado y renovación automática. 4. El usuario puede modificar opciones permitidas, como renovación o activación inmediata.
Postcondición	El usuario conoce el estado de sus servicios.
Excepciones	Error al recuperar la información o intento de modificar un servicio inexistente.
Importancia	Media Alta

Tabla B.8: CU-8 Consultar compras y servicios activos.

CU-9	Consultar guía de compra
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-4.6, RF-13.6
Descripción	Consulta orientativa de qué modelos o planes son necesarios para ejecutar los pipelines disponibles.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario accede a la guía de compra. 2. El sistema obtiene los pipelines y sus modelos requeridos. 3. El sistema indica qué requisitos cumple ya el usuario. 4. El sistema muestra qué modelos o planes debería contratar para ampliar su acceso.
Postcondición	El usuario dispone de información para decidir qué servicio contratar.
Excepciones	No existen pipelines disponibles o se produce un error al calcular dependencias.
Importancia	Media

Tabla B.9: CU-9 Consultar guía de compra.

CU-10	Consultar pipelines disponibles
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-4.3, RF-4.4, RF-4.5, RF-13.3
Descripción	Obtención de los pipelines que el usuario puede ejecutar según su rol, plan y modelos contratados.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario accede al panel principal. 2. El sistema identifica al usuario autenticado. 3. El sistema revisa su plan y sus modelos contratados. 4. El sistema obtiene los pipelines compatibles. 5. La interfaz muestra únicamente los pipelines ejecutables.
Postcondición	El usuario puede seleccionar un pipeline permitido.
Excepciones	Si no hay pipelines disponibles, se muestra un aviso y se deshabilita la ejecución.
Importancia	Muy alta

Tabla B.10: CU-10 Consultar pipelines disponibles.

CU-11	Cargar y modificar configuración del pipeline
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-5.1, RF-5.2, RF-5.3, RF-5.4, RF-5.5
Descripción	Consulta de la configuración predeterminada de un pipeline y posible modificación antes de ejecutarlo.
Precondición	El usuario ha seleccionado un pipeline permitido.
Acciones	1. El usuario pulsa la opción de cargar configuración. 2. El sistema recupera la configuración predeterminada de las etapas. 3. La interfaz muestra la configuración en formato editable. 4. El usuario modifica los parámetros que considere necesarios. 5. El sistema validará la configuración al lanzar la ejecución.
Postcondición	La configuración queda preparada para utilizarse en la ejecución.
Excepciones	Configuración no encontrada, formato JSON incorrecto o pipeline no permitido.
Importancia	Alta

Tabla B.11: CU-11 Cargar y modificar configuración del pipeline.

CU-12	Ejecutar pipeline sobre una imagen
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-6.1, RF-6.2, RF-6.3, RF-6.4, RF-6.5, RF-6.6, RF-6.7
Descripción	Flujo principal de procesamiento: el usuario sube una imagen, selecciona un pipeline y obtiene resultados generados por sus etapas.
Precondición	El usuario ha iniciado sesión y dispone de acceso al pipeline seleccionado.
Acciones	1. El usuario selecciona un pipeline disponible. 2. El usuario adjunta una imagen. 3. Opcionalmente, el usuario ajusta la configuración. 4. El usuario pulsa la opción de ejecutar análisis. 5. El sistema valida la imagen y la configuración. 6. El sistema comprueba los permisos del usuario. 7. El sistema ejecuta las etapas del pipeline. 8. El sistema registra la ejecución y sus resultados. 9. El sistema devuelve los resultados a la interfaz.
Postcondición	La ejecución queda registrada y el usuario puede visualizar los resultados generados.
Excepciones	Archivo inválido, pipeline no permitido, configuración incorrecta, error de procesamiento o ausencia de resultados útiles.
Importancia	Muy alta

Tabla B.12: CU-12 Ejecutar pipeline sobre una imagen.

CU-13	Visualizar y descargar resultados
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-7.1, RF-7.2, RF-7.3, RF-7.4, RF-7.5, RF-9.2
Descripción	Visualización de imágenes y datos generados por una ejecución, con posibilidad de descarga temporal.
Precondición	Existe una ejecución completada con archivos disponibles.
Acciones	1. El sistema muestra las etapas ejecutadas. 2. El sistema muestra las imágenes generadas por cada etapa. 3. El usuario puede desplegar los datos JSON. 4. El usuario puede descargar imágenes o archivos JSON. 5. El sistema resuelve la descarga mediante el token temporal correspondiente.
Postcondición	El usuario visualiza o descarga los resultados mientras estén disponibles.
Excepciones	Archivo caducado, token inválido, archivo eliminado físicamente o acceso no autorizado.
Importancia	Alta

Tabla B.13: CU-13 Visualizar y descargar resultados.

CU-14	Consultar historial de ejecuciones
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-8.1, RF-8.2, RF-8.3, RF-8.4, RF-8.5, RF-13.7
Descripción	Consulta de las ejecuciones realizadas por el usuario autenticado.
Precondición	El usuario ha iniciado sesión.
Acciones	1. El usuario accede a la página de historial. 2. El sistema obtiene las ejecuciones asociadas a su cuenta. 3. El sistema muestra pipeline, fecha, estado, duración y errores. 4. El usuario selecciona una ejecución concreta. 5. El sistema muestra el detalle y los archivos disponibles, si no han expirado.
Postcondición	El usuario consulta sus ejecuciones y puede revisar las que aún tengan resultados disponibles.
Excepciones	No existen ejecuciones, ejecución no encontrada o intento de acceder a una ejecución ajena.
Importancia	Alta

Tabla B.14: CU-14 Consultar historial de ejecuciones.

CU-15	Acceder al panel de administración
Versión	1.0
Actor	Administrador
Requisitos asociados	RF-2.3, RF-2.5, RF-10.1, RF-13.8
Descripción	Acceso del administrador a la consola de gestión del sistema.
Precondición	El usuario ha iniciado sesión con rol de Administrador.
Acciones	1. El administrador inicia sesión. 2. El sistema detecta su rol. 3. El sistema redirige al panel de administración. 4. El panel muestra las secciones de gestión disponibles.
Postcondición	El administrador puede acceder a funciones de revisión y gestión.
Excepciones	Usuario no administrador o token no válido.
Importancia	Alta

Tabla B.15: CU-15 Acceder al panel de administración.

CU-16	Gestionar usuarios desde administración
Versión	1.0
Actor	Administrador
Requisitos asociados	RF-10.2, RF-10.3, RF-10.4, RF-10.5
Descripción	Consulta de usuarios y modificación controlada de su estado.
Precondición	El administrador ha accedido al panel de administración.
Acciones	1. El administrador consulta el listado de usuarios. 2. El sistema muestra información básica de cada cuenta. 3. El administrador selecciona una cuenta. 4. El administrador solicita un cambio de estado. 5. El sistema valida que la operación sea permitida. 6. El sistema aplica el cambio y confirma la operación.
Postcondición	El estado de la cuenta queda actualizado.
Excepciones	Usuario no encontrado, intento de auto-bloqueo o error al actualizar la base de datos.
Importancia	Alta

Tabla B.16: CU-16 Gestionar usuarios desde administración.

CU-17	Gestionar pipelines desde administración
Versión	1.0
Actor	Administrador
Requisitos asociados	RF-11.1, RF-11.2, RF-11.3, RF-11.4, RF-11.5, RF-11.6
Descripción	Creación, modificación, deshabilitación o eliminación de pipelines desde el panel de administración.
Precondición	El administrador ha iniciado sesión y existen modelos y modos registrados.
Acciones	1. El administrador consulta los pipelines existentes. 2. El administrador crea un nuevo pipeline o selecciona uno existente. 3. El administrador define o modifica sus etapas. 4. El sistema valida que las etapas hagan referencia a modelos y modos válidos. 5. El sistema guarda la configuración del pipeline. 6. Si se solicita eliminar un pipeline, el sistema comprueba si tiene ejecuciones asociadas.
Postcondición	El catálogo de pipelines queda actualizado sin necesidad de modificar el código fuente.
Excepciones	Pipeline inexistente, etapas inválidas, falta de permisos o existencia de ejecuciones asociadas que impidan el borrado físico.
Importancia	Muy alta

Tabla B.17: CU-17 Gestionar pipelines desde administración.

CU-18	Consultar ejecuciones globales
Versión	1.0
Actor	Administrador
Requisitos asociados	RF-12.1, RNF-7
Descripción	Consulta administrativa del histórico general de ejecuciones realizadas en el sistema.
Precondición	El administrador ha accedido al panel de administración.
Acciones	1. El administrador solicita la vista global de ejecuciones. 2. El sistema obtiene las ejecuciones registradas. 3. El sistema muestra usuario, pipeline, fecha, estado, duración y posibles errores. 4. El administrador revisa la información para mantenimiento o supervisión.
Postcondición	El administrador dispone de una visión global del uso del sistema.
Excepciones	Falta de permisos o error al obtener la información.
Importancia	Media Alta

Tabla B.18: CU-18 Consultar ejecuciones globales.

CU-19	Ejecutar tareas de mantenimiento
Versión	1.0
Actor	Sistema
Requisitos asociados	RF-9.4, RF-9.5, RF-12.2, RF-12.3, RF-12.4
Descripción	Revisión periódica de temporales, servicios caducados, renovaciones automáticas y cuentas dadas de baja.
Precondición	Existen ejecuciones, servicios o cuentas susceptibles de revisión.
Acciones	1. El sistema comprueba si hay archivos temporales caducados. 2. El sistema elimina o invalida los recursos que ya no deben estar disponibles. 3. El sistema revisa suscripciones y alquileres vencidos. 4. El sistema renueva o desactiva servicios según su configuración. 5. El sistema revisa cuentas dadas de baja para aplicar anonimización cuando proceda.
Postcondición	El sistema conserva un estado coherente y evita acumulación de recursos innecesarios.
Excepciones	Si una tarea falla, el sistema no debe interrumpir el funcionamiento principal de la aplicación.
Importancia	Alta

Tabla B.19: CU-19 Ejecutar tareas de mantenimiento.

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se describe el diseño técnico del sistema, estructurado en tres apartados: diseño de datos, diseño arquitectónico y diseño procedimental.

El objetivo de este anexo es justificar las decisiones adoptadas durante el diseño de la solución. En particular, en el diseño de datos no solo se presenta la estructura final de la base de datos, sino también el motivo por el que se han definido determinados atributos, restricciones y relaciones. De este modo, el modelo no se plantea como una simple colección de tablas, sino como una propuesta coherente con los requisitos del sistema, la trazabilidad de la información y la posibilidad de ampliación futura.

C.2. Diseño de datos

El modelo de datos se ha diseñado para su implantación en MariaDB utilizando el motor InnoDB, lo que permite trabajar con claves foráneas, integridad referencial y soporte transaccional. Además, se utiliza `utf8mb4` con colación `utf8mb4_unicode_ci` para permitir el almacenamiento correcto de caracteres Unicode.

De manera general, el diseño se ha construido siguiendo varios criterios:

- **Separación de responsabilidades:** cada tabla representa una entidad o una asociación con significado propio.

- **Evitar redundancia innecesaria:** se distinguen claramente conceptos como usuarios, roles, planes, suscripciones, modelos IA, modos, pipelines y ejecuciones.
- **Preservar la integridad:** se emplean claves primarias, claves foráneas, restricciones **UNIQUE** y restricciones **CHECK** para evitar valores incoherentes en importes, fechas, duraciones y orden de ejecución.
- **Conservar la trazabilidad:** se evita el borrado automático en cascada en aquellas tablas cuyo contenido forma parte del histórico del sistema.
- **Facilitar la escalabilidad:** la estructura permite añadir nuevos planes, nuevos modelos IA, nuevos modos o nuevos pipelines sin modificar la base del modelo.

Antes de justificar cada una de las tablas del modelo, se incluyen dos representaciones complementarias del diseño de datos. En primer lugar, se presenta el diagrama entidad-relación, que permite comprender la estructura conceptual del sistema, sus entidades principales y las asociaciones existentes entre ellas. En segundo lugar, se muestra el diagrama del modelo relacional, donde dicha propuesta conceptual queda traducida a tablas, claves y relaciones concretas para su implantación en la base de datos.

Diagrama entidad-relación

La Figura **C.1** recoge el diagrama entidad-relación del sistema. En él se representa la visión conceptual del modelo de datos, identificando las entidades principales, sus atributos más relevantes y las relaciones que se establecen entre ellas.

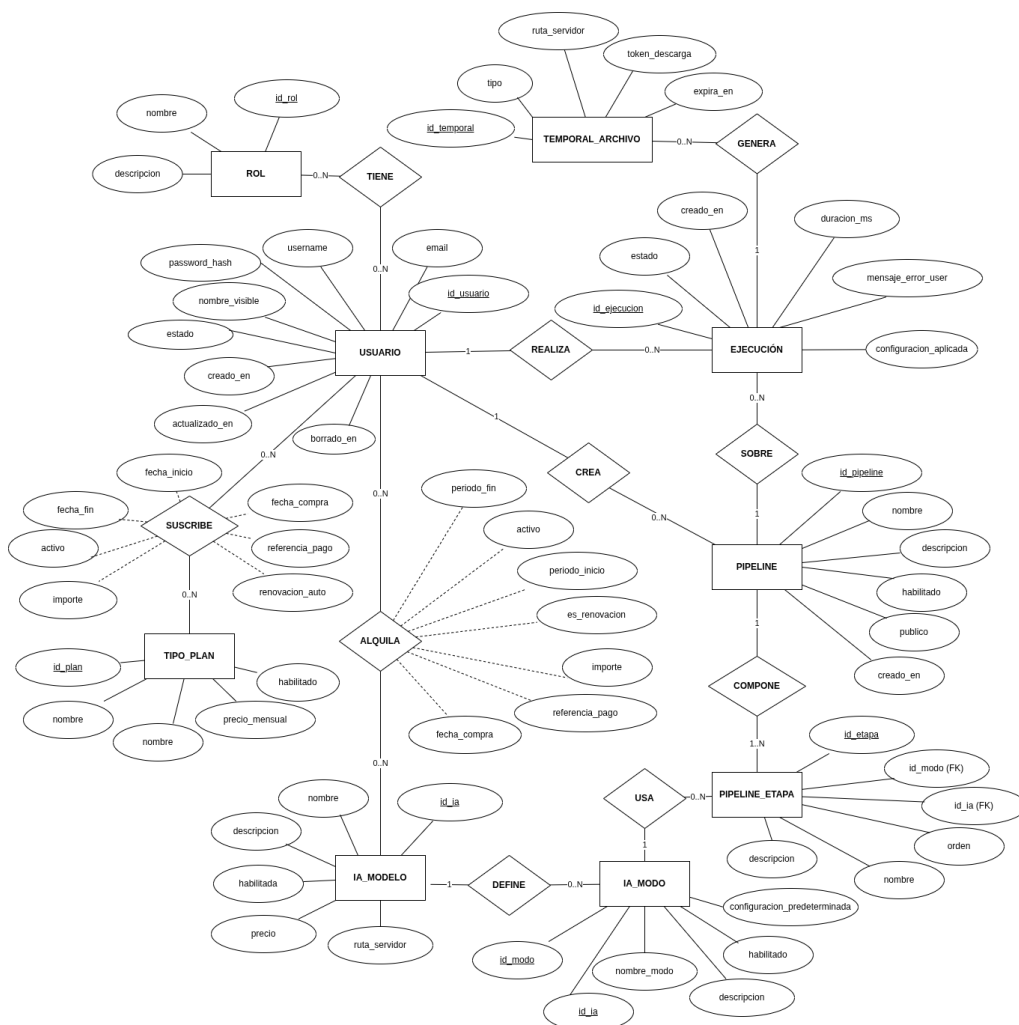


Figura C.1: Diagrama entidad-relación del sistema

Este diagrama permite observar que **USUARIO** actúa como entidad central del sistema, ya que se relaciona con la asignación de roles, la contratación de planes, el acceso individual a modelos de inteligencia artificial, la creación de pipelines y la realización de ejecuciones. Asimismo, se distingue la parte funcional asociada a los modelos IA, sus modos de uso y su composición dentro de pipelines, junto con la gestión de los archivos temporales generados durante las ejecuciones. Esta representación resulta especialmente útil para comprender el significado semántico de cada relación antes de su transformación al esquema relacional definitivo.

Diagrama del modelo relacional

La Figura C.2 muestra la transformación del modelo conceptual al modelo relacional. En este nivel, las entidades y relaciones anteriores pasan a expresarse mediante tablas con sus claves primarias, claves foráneas y restricciones principales.



Figura C.2: Diagrama del modelo relacional del sistema

En este diagrama puede apreciarse con mayor precisión cómo se implementan las relaciones entre las distintas tablas. Por ejemplo, las relaciones de muchos a muchos se materializan mediante tablas intermedias como `USUARIO_ROL`, mientras que otras asociaciones se resuelven a través de claves foráneas directas, como sucede en `SUSCRIPCION_PLAN`, `ALQUILA`, `EJECUCION` o `PIPELINE_ETAPA`. Además, esta representación permite comprobar de forma más clara la estructura final sobre la que se aplican las restricciones de unicidad, integridad referencial y consistencia que se justifican a continuación.

A continuación se justifica cada una de las tablas del modelo..

Tabla USUARIO

La tabla `USUARIO` representa la cuenta de cada persona registrada en el sistema. Se trata de una tabla central, ya que gran parte del resto de

Atributo	Descripción
id_usuario	Identificador interno del usuario. Se define como clave primaria autoincremental para disponer de una referencia estable e independiente de los datos funcionales.
email	Correo electrónico del usuario. Es NOT NULL porque constituye un dato esencial para identificar la cuenta y se declara UNIQUE para impedir duplicidades.
username	Nombre de usuario del sistema. También es NOT NULL y UNIQUE porque debe existir siempre y no puede repetirse sin generar ambigüedad.
password_hash	Contraseña almacenada en formato hash. Es obligatoria porque forma parte de la autenticación y no se guarda en texto plano por seguridad.
nombre_visible	Nombre mostrado en la interfaz. Se permite NULL porque no es imprescindible para el funcionamiento interno del sistema.
estado	Estado lógico de la cuenta. Es obligatorio y tiene el valor por defecto <i>activa</i> para evitar insertar usuarios sin estado definido.
creado_en	Fecha de creación del registro. Se marca como obligatoria y con valor por defecto para asegurar trazabilidad desde el alta.
actualizado_en	Fecha de última modificación. Se actualiza automáticamente para facilitar auditoría y seguimiento de cambios.
borrado_en	Fecha de borrado lógico. Se permite NULL porque solo debe informarse cuando el usuario deja de estar operativo sin eliminarse físicamente.

Tabla C.1: Atributos de la tabla USUARIO

entidades se relacionan directa o indirectamente con ella. Sus atributos se muestran en la Tabla C.1.

La clave primaria sobre **id_usuario** garantiza una identificación interna única y estable. Las restricciones UNIQUE sobre **email** y **username** se introducen porque ambos campos tienen significado funcional dentro del sistema: el primero evita que dos cuentas compartan el mismo correo electrónico y el segundo impide colisiones en el identificador visible del usuario. Por su parte, los campos obligatorios se han limitado a los estrictamente necesarios

Atributo	Descripción
<code>id_rol</code>	Identificador interno del rol. Se utiliza una clave primaria autoincremental para desacoplar la identificación interna del nombre funcional.
<code>nombre</code>	Nombre del rol. Es NOT NULL y UNIQUE para que cada rol quede claramente diferenciado y no existan duplicados lógicos.
<code>descripcion</code>	Texto descriptivo del rol. Se permite NULL porque es útil para documentar, pero no imprescindible para aplicar permisos.

Tabla C.2: Atributos de la tabla ROL

para operar, mientras que otros como `nombre_visible` o `borrado_en` se mantienen opcionales para no forzar información que no siempre debe existir.

Tabla ROL

La tabla `ROL` almacena los distintos perfiles de acceso del sistema, como puede ser usuario administrador, cliente o futuros roles como soporte. Sus atributos se muestran en la Tabla C.2.

La separación de `ROL` respecto de `USUARIO` permite gestionar los perfiles de forma flexible y evita codificar los roles como un simple valor fijo dentro de la cuenta. La unicidad del nombre se justifica porque no tendría sentido disponer de dos roles con la misma denominación, ya que eso introduciría ambigüedad tanto en la lógica del sistema como en la administración.

Tabla USUARIO__ROL

La tabla `USUARIO__ROL` materializa la asignación de roles a usuarios. Sus atributos se resumen en la Tabla C.3. Se trata de una tabla intermedia necesaria para representar una relación de muchos a muchos entre `USUARIO` y `ROL`.

La clave primaria compuesta (`id_usuario`, `id_rol`) evita que el mismo rol se asigne varias veces al mismo usuario. Esta solución es preferible a crear un identificador artificial adicional, ya que la propia combinación de ambos campos ya identifica unívocamente la asociación. En cuanto a las acciones de borrado, se utiliza `ON DELETE CASCADE` hacia `USUARIO` porque,

Atributo	Descripción
<code>id_usuario</code>	Usuario al que se asigna el rol. Es obligatorio porque la asociación no tiene sentido sin un usuario existente.
<code>id_rol</code>	Rol asignado al usuario. También es obligatorio porque la asociación debe apuntar siempre a un rol concreto.

Tabla C.3: Atributos de la tabla `USUARIO_ROL`

si un usuario desaparece, sus asignaciones dejan de tener sentido; en cambio, hacia `ROL` se usa `ON DELETE RESTRICT` para impedir eliminar un rol que todavía está siendo utilizado.

Tabla `TIPO_PLAN`

La tabla `TIPO_PLAN` recoge el catálogo de planes disponibles en el sistema. En la versión actual se contemplan los planes Básico y Pro: el primero sirve como plan de entrada y permite ampliar el acceso mediante alquileres individuales de modelos, mientras que el segundo representa un acceso más completo al catálogo disponible. Sus atributos se presentan en la Tabla C.4. Se separa de `SUSCRIPCION_PLAN` para distinguir entre el tipo abstracto de plan y la contratación concreta realizada por un usuario en un periodo determinado.

La restricción `UNIQUE` sobre `nombre` se justifica porque el catálogo no debe contener dos planes con la misma denominación. El `CHECK` sobre `precio_mensual` impide registrar precios negativos, reforzando la coherencia del dato económico desde la propia base de datos. El campo `habilitado` permite desactivar planes sin necesidad de eliminarlos, lo que conserva referencias históricas y simplifica la administración.

Tabla `SUSCRIPCION_PLAN`

La tabla `SUSCRIPCION_PLAN` registra cada suscripción efectiva de un usuario a un plan concreto. Sus atributos se recogen en la Tabla C.5. Se diferencia de `TIPO_PLAN` porque aquí no se define el plan, sino cada contratación realizada.

Las claves foráneas hacia `USUARIO` y `TIPO_PLAN` usan `RESTRICT` porque las suscripciones forman parte del histórico del sistema. De este modo se evita eliminar usuarios o planes que todavía tienen contratos asociados.

Atributo	Descripción
id_plan	Identificador interno del plan. Se utiliza como clave primaria autoincremental.
nombre	Nombre del plan. Es NOT NULL y UNIQUE porque cada elemento del catálogo debe quedar claramente identificado.
precio_mensual	Precio base mensual del plan. Se permite NULL para no obligar a informar un importe en planes gratuitos o todavía no definidos.
descripcion	Descripción del plan y de sus condiciones generales. Se permite NULL porque no afecta directamente a la lógica de acceso.
habilitado	Indica si el plan está disponible para nuevas contrataciones. Es obligatorio y tiene valor por defecto TRUE para evitar estados indefinidos.

Tabla C.4: Atributos de la tabla TIPO_PLAN

Además, el **CHECK** sobre fechas impide que una suscripción tenga una fecha de finalización anterior a su fecha de inicio, y el **CHECK** sobre **importe** evita registrar cantidades negativas.

Tabla IA__MODELO

La tabla **IA__MODELO** representa cada modelo de inteligencia artificial disponible en la aplicación. Sus atributos se muestran en la Tabla C.6. Esta tabla es una de las entidades principales del dominio, ya que el sistema se centra en permitir el acceso y la ejecución de distintos modelos IA.

La unicidad del nombre se establece para que cada modelo tenga una identidad lógica inequívoca. El **CHECK** sobre **precio** evita importes negativos en el catálogo de modelos. Además, el campo **habilitada** permite activar y desactivar modelos sin eliminarlos físicamente, lo que es útil en mantenimiento, pruebas o retirada temporal del servicio.

Tabla IA__MOD0

La tabla **IA__MOD0** recoge los distintos modos de funcionamiento de cada modelo IA. Sus atributos aparecen en la Tabla C.7. Esta separación permite

Atributo	Descripción
id_suscripcion	Identificador interno de la suscripción. Se define como clave primaria autoincremental.
id_usuario	Usuario suscrito. Es NOT NULL porque la suscripción siempre debe pertenecer a una cuenta concreta.
id_plan	Plan contratado. También es NOT NULL porque toda suscripción debe asociarse a un plan del catálogo.
fecha_compra	Fecha en la que se registra la contratación. Es obligatoria y permite conservar trazabilidad sobre el momento de adquisición del servicio.
fecha_compra	Fecha en la que se registra la contratación. Es obligatoria y permite conservar trazabilidad sobre el momento de adquisición del servicio.
fecha_fin	Fecha de finalización, cuando exista. Se permite NULL porque una suscripción puede estar activa o no tener todavía cierre registrado.
activo	Indica si la suscripción debe considerarse operativa. Es obligatorio y permite resolver de forma directa las consultas de acceso sin depender únicamente del cálculo por fechas.
renovacion_auto	Indica si la renovación es automática. Es obligatorio y se inicia en FALSE por defecto.
importe	Importe cobrado en esa contratación concreta. Se permite NULL porque puede haber promociones o ajustes que no siempre se registren inicialmente.
referencia_pago	Referencia externa del pago asociado. Se deja opcional porque no tiene por qué existir siempre o puede no haberse informado todavía.

Tabla C.5: Atributos de la tabla SUSCRIPCION_PLAN

Atributo	Descripción
<code>id_ia</code>	Identificador interno del modelo. Se define como clave primaria autoincremental.
<code>nombre</code>	Nombre del modelo. Es NOT NULL y UNIQUE para evitar duplicidades semánticas.
<code>descripcion</code>	Descripción funcional del modelo. Se permite NULL porque no siempre es necesaria para la lógica interna.
<code>habilitada</code>	Indica si el modelo puede utilizarse. Es obligatorio y tiene valor por defecto TRUE para mantener un estado definido.
<code>precio</code>	Coste individual del acceso al modelo, si procede. Se permite NULL porque algunos modelos pueden no tener precio independiente.
<code>ruta_servidor</code>	Ruta o referencia interna utilizada por el backend para localizar el modelo. Se deja opcional para no forzar que todos los modelos se resuelvan exactamente igual.

Tabla C.6: Atributos de la tabla IA_MODELO

reutilizar un mismo modelo con varias configuraciones o finalidades sin necesidad de duplicarlo en la tabla principal.

La restricción **UNIQUE** (`id_ia`, `nombre_modo`) evita que un mismo modelo tenga dos modos con el mismo nombre, lo que generaría ambigüedad. Además, se mantiene **UNIQUE** (`id_modo`, `id_ia`) porque esa combinación se usa posteriormente como referencia compuesta desde **PIPELINE_ETAPA**. La clave foránea hacia **IA_MODELO** utiliza **RESTRICT** para impedir eliminar un modelo si todavía conserva modos asociados.

Tabla ALQUILA

La tabla **ALQUILA** registra la adquisición individual de acceso a un modelo IA por parte de un usuario. Sus atributos se detallan en la Tabla C.8. Esta tabla resulta necesaria porque el sistema no solo contempla acceso por suscripción general, sino también compras o activaciones concretas de modelos.

Las claves foráneas hacia **USUARIO** e **IA_MODELO** emplean **RESTRICT** porque estas operaciones forman parte del histórico de negocio y no conviene perderlas automáticamente al eliminar entidades relacionadas. El **CHECK**

Atributo	Descripción
<code>id_modo</code>	Identificador interno del modo. Se define como clave primaria autoincremental.
<code>id_ia</code>	Modelo al que pertenece el modo. Es NOT NULL porque todo modo debe estar asociado a un modelo existente.
<code>nombre_modo</code>	Nombre del modo dentro del modelo. Es obligatorio porque identifica funcionalmente la variante de uso.
<code>descripcion</code>	Explicación opcional del modo. Se permite NULL porque no siempre será necesaria.
<code>habilitado</code>	Indica si el modo está disponible para su uso. Es obligatorio y tiene valor por defecto <code>TRUE</code> .
<code>config_predeterminada</code>	Configuración base del modo. Se almacena como <code>TEXT</code> y se permite NULL para mantener flexibilidad y no imponer una estructura fija.

Tabla C.7: Atributos de la tabla `IA_MODO`

sobre `importe` evita cantidades negativas, mientras que el `CHECK` sobre el periodo obliga a que, si existen fecha de inicio y fecha de fin, la finalización no sea anterior al comienzo del acceso.

Tabla PIPELINE

La tabla `PIPELINE` representa un flujo de procesamiento configurable. Cada pipeline agrupa una o varias etapas que se ejecutan de forma ordenada sobre una imagen. Sus atributos se presentan en la Tabla C.9. Esta tabla es una de las partes más importantes del diseño, ya que permite que el sistema no dependa de flujos escritos de forma fija en el código, sino de configuraciones almacenadas y gestionables desde la propia aplicación.

El campo `id_usuario` usa `ON DELETE SET NULL`. Esta decisión evita que un pipeline desaparezca automáticamente si se elimina la cuenta que lo creó. En este proyecto tiene especial importancia, ya que los pipelines pueden haber sido definidos por un administrador y seguir siendo útiles para el resto del sistema. Además, se define la restricción `UNIQUE (id_usuario, nombre)` para impedir que un mismo usuario tenga dos pipelines con el mismo nombre. Esta restricción resulta especialmente importante de cara a una posible evolución en la que los usuarios puedan crear sus propios

Atributo	Descripción
id_compra	Identificador interno de la operación. Se define como clave primaria autoincremental.
id_usuario	Usuario que realiza la compra. Es NOT NULL porque toda operación debe tener titular.
id_ia	Modelo IA adquirido. También es NOT NULL porque la compra debe referirse a un recurso concreto.
fecha_compra	Fecha de la operación. Se declara obligatoria con valor por defecto para registrar automáticamente el momento del alta.
periodo_inicio	Fecha de inicio de la vigencia del acceso. Se permite NULL porque no todas las operaciones tienen por qué estar acotadas temporalmente del mismo modo.
periodo_fin	Fecha de finalización de la vigencia. También se permite NULL para mantener flexibilidad.
activo	Indica si el acceso debe considerarse operativo. Es obligatorio y permite consultar de forma directa los modelos disponibles para el usuario.
renovacion_auto	Indica si la renovación del acceso es automática. Es obligatorio y se inicia por defecto en FALSE para no asumir renovaciones sin indicación expresa.
importe	Importe cobrado en la operación concreta. Se conserva para mantener el valor pagado en el momento de la compra, aunque posteriormente cambie el precio del modelo. Se permite NULL porque pueden existir casos gratuitos, promocionales o pendientes de registrar.
referencia_pago	Referencia externa del pago. Se deja opcional porque no siempre existirá una referencia única o puede no haberse informado todavía.

Tabla C.8: Atributos de la tabla ALQUILA

Atributo	Descripción
<code>id_pipeline</code>	Identificador interno del pipeline. Se define como clave primaria autoincremental.
<code>id_usuario</code>	Usuario propietario o creador del pipeline. Se permite NULL porque el diseño contempla conservar el pipeline aunque desaparezca su propietario original.
<code>nombre</code>	Nombre del pipeline. Es NOT NULL porque resulta necesario para identificarlo funcionalmente en el catálogo.
<code>descripcion</code>	Descripción del flujo. Se permite NULL porque no siempre es imprescindible documentarlo.
<code>habilitado</code>	Indica si el pipeline puede utilizarse. Es obligatorio y tiene valor por defecto TRUE.
<code>creado_en</code>	Fecha de creación del pipeline. Se registra automáticamente para mantener trazabilidad temporal.
<code>publico</code>	Indica si el pipeline se muestra como disponible para los usuarios o si queda limitado a su propietario. Este campo facilita el filtrado del catálogo sin tener que deducir en cada consulta si el flujo procede de un administrador o de un usuario concreto.

Tabla C.9: Atributos de la tabla PIPELINE

pipelines, ya que permite mantener un catálogo privado ordenado y sin ambigüedades.

Tabla EJECUCION

La tabla **EJECUCION** registra cada procesamiento realizado en el sistema. Sus atributos se muestran en la Tabla C.10. Esta tabla es clave para la trazabilidad operativa, ya que permite saber qué usuario ejecutó qué pipeline, cuándo se lanzó, con qué estado terminó y qué configuración se aplicó.

Las claves foráneas hacia **USUARIO** y **PIPELINE** utilizan **RESTRICT**. En ambos casos interesa conservar la referencia completa de la ejecución: no debe eliminarse un usuario o pipeline si existen registros históricos que dependen de ellos. De esta forma, el historial de ejecuciones mantiene significado y puede consultarse posteriormente desde la interfaz de usuario o desde el panel de administración. Además, el **CHECK** sobre `duracion_ms` evita registrar tiempos negativos.

Atributo	Descripción
<code>id_ejecucion</code>	Identificador interno de la ejecución. Se define como clave primaria autoincremental.
<code>id_usuario</code>	Usuario asociado a la ejecución. Es obligatorio para mantener responsabilidad y trazabilidad sobre el uso del sistema.
<code>id_pipeline</code>	Pipeline ejecutado. Es NOT NULL porque cada ejecución debe quedar vinculada al flujo concreto que se lanzó.
<code>estado</code>	Estado de la ejecución. Es obligatorio y su valor por defecto es pendiente , que representa el estado inicial natural antes de completarse o fallar.
<code>duracion_ms</code>	Duración de la ejecución en milisegundos. Se permite NULL porque puede no conocerse todavía al inicio del proceso o si la ejecución falla antes de completarse.
<code>mensaje_error_user</code>	Mensaje de error orientado al usuario. Es opcional porque solo se utiliza cuando la ejecución no puede completarse correctamente.
<code>creado_en</code>	Fecha de creación de la ejecución. Se registra automáticamente para mantener trazabilidad temporal.
<code>config_aplicada</code>	Configuración real utilizada en la ejecución, tanto si procede de valores predeterminados como si ha sido modificada por el usuario. Se permite NULL porque no siempre será necesario persistirla.

Tabla C.10: Atributos de la tabla EJECUCION

Tabla TEMPORAL__ARCHIVO

La tabla **TEMPORAL__ARCHIVO** gestiona los archivos temporales asociados a una ejecución. Sus atributos aparecen en la Tabla C.11. Estos recursos pueden representar archivos de entrada, de salida o intermedios y su naturaleza es claramente dependiente de la ejecución que los genera.

La restricción **UNIQUE** sobre `token_descarga` garantiza que, si existe, cada token identifique un único recurso. La clave foránea hacia **EJECUCION** emplea **ON DELETE CASCADE** porque aquí sí existe una dependencia total: si desaparece la ejecución, carece de sentido conservar sus archivos temporales. Es precisamente un caso claro y justificado de borrado en cascada.

Atributo	Descripción
<code>id_temporal</code>	Identificador interno del archivo temporal. Se define como clave primaria autoincremental.
<code>id_ejecucion</code>	Ejecución a la que pertenece el archivo. Es obligatorio porque el temporal no tiene sentido de forma independiente.
<code>tipo</code>	Tipo de archivo temporal. Se permite NULL porque puede haber casos en los que no se clasifique expresamente.
<code>ruta_servidor</code>	Ruta interna del archivo en el servidor. Se deja opcional para no imponer una única forma de gestión.
<code>token_descarga</code>	Token utilizado para acceso o descarga controlada. Se permite NULL porque no todos los archivos temporales tienen por qué exponerse.
<code>expira_en</code>	Fecha de expiración del recurso temporal. Es opcional porque puede calcularse o no utilizarse en todos los casos.

Tabla C.11: Atributos de la tabla TEMPORAL_ARCHIVO

Tabla PIPELINE_ETAPA

La tabla PIPELINE_ETAPA descompone cada pipeline en etapas ordenadas. Sus atributos se muestran en la Tabla C.12. Esta estructura permite representar un procesamiento secuencial y modular, en el que cada etapa invoca un modo concreto de un modelo IA.

La clave foránea hacia PIPELINE utiliza ON DELETE CASCADE porque las etapas dependen completamente del pipeline al que pertenecen. En cambio, la clave foránea compuesta (`id_modulo`, `id_ia`) hacia IA_MODALIDAD usa RESTRICT para impedir eliminar un modo que todavía está siendo utilizado. La restricción UNIQUE (`id_pipeline`, `nombre`, `orden`) evita duplicidades problemáticas dentro de un mismo pipeline, y el CHECK (`orden >= 1`) obliga a que el orden sea positivo y coherente con la idea de secuencia.

Relaciones del modelo

Una vez descritas las tablas por separado, conviene explicar de forma conjunta las relaciones del modelo para entender mejor el sentido global del diseño.

Atributo	Descripción
<code>id_etapa</code>	Identificador interno de la etapa. Se define como clave primaria autoincremental.
<code>id_pipeline</code>	Pipeline al que pertenece la etapa. Es obligatorio porque una etapa no existe de forma autónoma.
<code>id_modo</code>	Modo IA utilizado en la etapa. Es NOT NULL porque la etapa debe invocar una operación concreta.
<code>id_ia</code>	Modelo IA asociado al modo referenciado. Es obligatorio para reforzar la coherencia de la referencia compuesta.
<code>orden</code>	Posición de la etapa dentro del pipeline. Es obligatorio porque el flujo debe tener un orden definido.
<code>nombre</code>	Nombre de la etapa. Es obligatorio para identificarla funcionalmente dentro del flujo.
<code>descripcion</code>	Descripción opcional de la finalidad de la etapa. Se permite NULL porque no siempre será necesaria.

Tabla C.12: Atributos de la tabla PIPELINE_ETAPA

- **USUARIO – USUARIO_ROL – ROL**: la asignación de roles se resuelve mediante una tabla intermedia porque la relación es de muchos a muchos. Un usuario puede tener varios roles y un rol puede estar asignado a varios usuarios.
- **USUARIO – SUSCRIPCION_PLAN**: un usuario puede contratar varias suscripciones a lo largo del tiempo, mientras que cada suscripción pertenece a un único usuario.
- **TIPO_PLAN – SUSCRIPCION_PLAN**: un tipo de plan puede estar asociado a muchas suscripciones, mientras que cada suscripción corresponde a un único plan.
- **USUARIO – ALQUILA**: un usuario puede realizar múltiples compras individuales de modelos IA, pero cada operación pertenece a un único usuario.
- **IA_MODELO – ALQUILA**: un modelo IA puede aparecer en muchas compras individuales, mientras que cada compra referencia a un único modelo.
- **IA_MODELO – IA_MODO**: un modelo puede disponer de varios modos, mientras que cada modo pertenece a un único modelo.

- **USUARIO – PIPELINE**: un usuario puede crear varios pipelines. Sin embargo, se permite que el pipeline sobreviva a la eliminación de su propietario mediante `ON DELETE SET NULL`. Esta decisión favorece la conservación de la estructura del flujo sin vincular su existencia estrictamente al usuario original.
- **PIPELINE – PIPELINE_ETAPA**: un pipeline puede contener varias etapas y estas dependen totalmente de él, por eso se utiliza borrado en cascada.
- **IA_MODO – PIPELINE_ETAPA**: cada etapa referencia un modo concreto mediante la pareja (`id_modo`, `id_ia`). Esta referencia compuesta garantiza que el modo utilizado pertenece realmente al modelo indicado.
- **USUARIO – EJECUCION**: un usuario puede generar muchas ejecuciones, mientras que cada ejecución pertenece a un único usuario.
- **PIPELINE – EJECUCION**: un pipeline puede ejecutarse muchas veces. En el diseño final se utiliza `RESTRICT` porque se considera importante que cada ejecución mantenga la referencia completa al pipeline con el que fue lanzada.
- **EJECUCION – TEMPORAL_ARCHIVO**: una ejecución puede generar varios archivos temporales y cada archivo temporal pertenece a una única ejecución. Aquí sí se justifica claramente el borrado en cascada, ya que el temporal depende por completo de la ejecución.

Valoración global del modelo

En conjunto, el modelo de datos final ofrece una estructura coherente con el funcionamiento actual de la aplicación y con su posible evolución. No se limita a almacenar usuarios e imágenes procesadas, sino que representa los elementos principales del dominio: cuentas, roles, planes, alquileres de modelos, modelos de IA, modos de ejecución, pipelines, ejecuciones y archivos temporales.

Una de las decisiones más importantes del diseño es separar claramente el catálogo del sistema de las operaciones realizadas por los usuarios. Por ejemplo, los planes se definen en `TIPO_PLAN`, pero las contrataciones concretas se registran en `SUSCRIPCION_PLAN`; del mismo modo, los modelos de IA se almacenan en `IA_MODELO`, mientras que los accesos individuales se reflejan en `ALQUILA`. Esta separación permite conservar el histórico de uso

y de contratación aunque en el futuro cambien los precios, se deshabiliten modelos o se modifiquen las condiciones de un plan.

También destaca la forma en que se han modelado los pipelines. La existencia de `PIPELINE` y `PIPELINE_ETAPA` permite definir flujos de procesamiento formados por varias etapas ordenadas, sin tener que codificar cada combinación de modelos directamente en la aplicación. Esto favorece la ampliación del sistema, ya que se pueden añadir nuevos flujos combinando modelos y modos ya registrados. Además, la restricción `UNIQUE (id_usuario, nombre)` evita que un mismo usuario pueda tener dos pipelines con el mismo nombre, algo especialmente útil si en el futuro se habilita la creación de pipelines propios por parte de los usuarios.

Las restricciones del modelo refuerzan esta coherencia general. Las claves foráneas mantienen la relación entre las entidades, las restricciones `UNIQUE` evitan duplicidades funcionales y los `CHECK` impiden valores básicos incoherentes, como importes negativos, fechas de fin anteriores a las de inicio, duraciones negativas u órdenes de etapa no válidos.

Por último, las acciones de borrado se han definido según el significado de cada relación. Se utiliza `RESTRICT` en tablas que forman parte del histórico funcional o económico del sistema, como suscripciones, alquileres y ejecuciones; `CASCADE` únicamente en dependencias fuertes, como las etapas de un pipeline o los archivos temporales de una ejecución; y `SET NULL` en la relación entre `USUARIO` y `PIPELINE`, para que un flujo pueda conservarse aunque desaparezca el usuario que lo creó.

Por todo ello, el modelo resultante mantiene un equilibrio entre integridad, trazabilidad y capacidad de ampliación. Esta estructura permite cubrir las funcionalidades actuales del proyecto y deja preparada la base de datos para incorporar nuevos modelos, nuevos modos o nuevas formas de organizar pipelines sin rediseñar el sistema desde cero.

C.3. Diseño arquitectónico

El diseño arquitectónico describe la organización interna de la aplicación y la forma en que se distribuyen las responsabilidades entre sus distintos componentes. Mientras que el diseño de datos se centra en la estructura persistente de la información, este apartado explica cómo se articula la aplicación a nivel de capas, módulos y servicios para permitir la autenticación de usuarios, la gestión de planes y modelos de inteligencia artificial, la configuración de pipelines y la ejecución del procesamiento de imágenes.

La aplicación se ha desarrollado con **Flask**, por lo que no se aplica un patrón MVC clásico de forma estricta, como podría ocurrir en otros frameworks más orientados a dicho patrón. En este caso, se ha adoptado una arquitectura MVC adaptada al funcionamiento de Flask, incorporando además una capa de servicios para evitar que los controladores concentren toda la lógica de negocio y de procesamiento. Esta decisión permite separar mejor las responsabilidades del sistema, facilita las pruebas y favorece la ampliación futura de la aplicación.

De manera general, la arquitectura se organiza en los siguientes bloques:

- **Cliente o navegador:** representa el punto de interacción del usuario con la aplicación. Desde él se realizan peticiones HTTP y se reciben respuestas en forma de páginas HTML o datos JSON.
- **Vista:** está formada por las plantillas HTML y por las respuestas generadas por la API. Su función principal es presentar la información al usuario y permitir la interacción con formularios, paneles y resultados.
- **Controladores:** se implementan mediante módulos **Blueprint** de Flask. Estos módulos reciben las peticiones, validan los datos de entrada, comprueban permisos y delegan las operaciones principales en la capa correspondiente.
- **Capa de servicios:** agrupa la lógica de aplicación que no debe quedar directamente dentro de los controladores. En ella se sitúan componentes como el orquestador de pipelines, la factoría de modelos de inteligencia artificial y los lanzadores concretos de cada familia de modelos.
- **Modelo:** está formado por las clases de persistencia definidas con SQLAlchemy. Estas clases representan las entidades principales del sistema, como usuarios, roles, planes, modelos IA, modos, pipelines, ejecuciones y archivos temporales.
- **Base de datos:** almacena de forma persistente la información del sistema. En el entorno principal se utiliza MariaDB, mientras que en las pruebas automatizadas se emplea SQLite en memoria para aislar los tests y evitar afectar a la base de datos real.

A continuación se presentan tres vistas complementarias de la arquitectura. En primer lugar, se muestra la arquitectura general adaptada a Flask. En segundo lugar, se representa la organización real de módulos y

dependencias principales del proyecto. Por último, se detalla la estructura interna de la capa de servicios y el mecanismo de resolución dinámica de modelos mediante una factoría.

Arquitectura MVC adaptada a Flask con capa de servicios

La Figura C.3 muestra la organización general de la aplicación siguiendo una adaptación del patrón MVC al contexto de Flask. En esta arquitectura, el cliente realiza peticiones HTTP al servidor y recibe como respuesta páginas HTML o datos estructurados en formato JSON. La factoría de aplicación, implementada mediante `create_app()`, se encarga de inicializar la configuración, la base de datos, el sistema JWT y los distintos Blueprints de la aplicación.

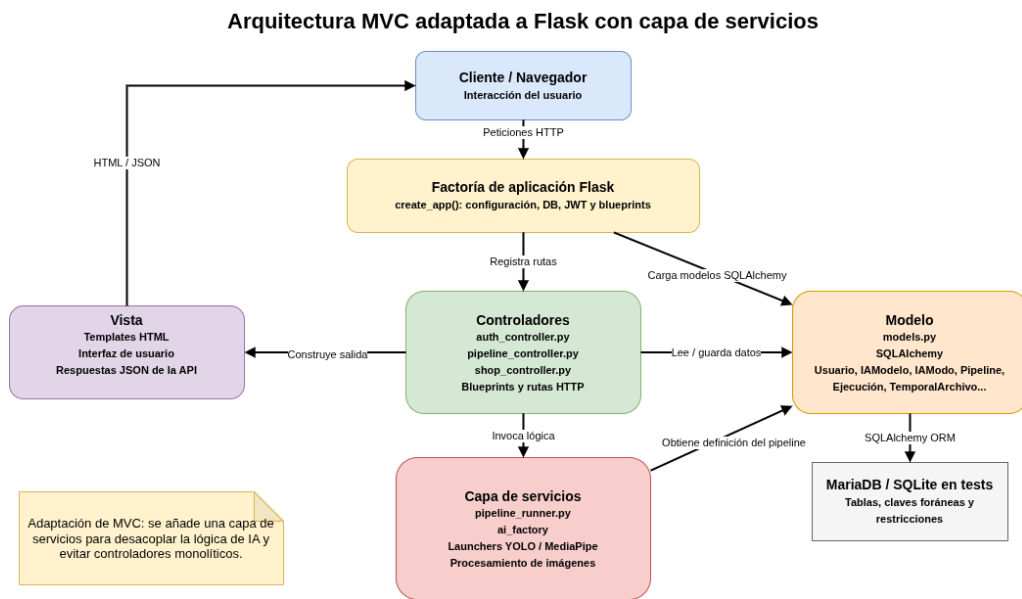


Figura C.3: Arquitectura MVC adaptada a Flask con capa de servicios

En esta estructura, los controladores actúan como punto de entrada de las operaciones de la aplicación. El módulo `auth_controller.py` concentra las operaciones relacionadas con autenticación, registro, inicio de sesión, consulta y modificación del perfil, baja de cuenta y administración de usuarios. El módulo `shop_controller.py` gestiona la parte comercial del sistema, incluyendo la consulta de planes, alquileres de modelos, suscripciones

y servicios activos. Por último, `pipeline_controller.py` se encarga de las operaciones vinculadas a los pipelines, como su consulta, configuración, ejecución, historial y gestión administrativa.

Aunque estos controladores podrían contener directamente toda la lógica del sistema, se ha optado por introducir una capa de servicios. Esta capa permite desacoplar la lógica de procesamiento de imágenes y de ejecución de pipelines respecto de las rutas HTTP. De este modo, los controladores se centran en recibir la petición, validar los datos, comprobar la autenticación y preparar la respuesta, mientras que la ejecución técnica del procesamiento queda delegada en servicios especializados.

La capa de servicios resulta especialmente importante en este proyecto porque el sistema no se limita a ejecutar una única operación fija sobre una imagen. Al contrario, permite definir pipelines formados por varias etapas, donde cada etapa puede utilizar un modelo y un modo de ejecución distinto. Por este motivo, concentrar toda la lógica en el controlador habría dado lugar a componentes demasiado extensos y difíciles de mantener. La separación en servicios facilita que la aplicación pueda incorporar nuevos modelos, modos o flujos de procesamiento sin modificar de forma significativa la estructura general.

El modelo de la aplicación se implementa mediante clases SQLAlchemy, definidas a partir del diseño de datos descrito anteriormente. Estas clases permiten trabajar con entidades persistentes como `Usuario`, `IAModelo`, `IAModo`, `Pipeline`, `PipelineEtapa`, `Ejecucion` o `TemporalArchivo`, ocultando parte de la complejidad de las consultas SQL directas. La comunicación con la base de datos se realiza mediante SQLAlchemy ORM, lo que permite mantener una correspondencia clara entre la lógica de aplicación y el modelo relacional.

Esta arquitectura también favorece la realización de pruebas. Al estar separadas las responsabilidades, es posible probar los controladores mediante el cliente de pruebas de Flask, simular la base de datos mediante SQLite en memoria y aislar determinados servicios mediante técnicas de sustitución o *mocking*. De esta forma, el diseño arquitectónico no solo organiza el código, sino que también contribuye a mejorar la verificabilidad del sistema.

Organización de módulos y dependencias principales

La Figura C.4 complementa la vista arquitectónica anterior mostrando cómo se materializa dicha arquitectura en la estructura real del proyecto. A diferencia del diagrama MVC, que representa las responsabilidades lógicas

del sistema, este diagrama muestra los principales archivos, paquetes y dependencias que intervienen en la aplicación.

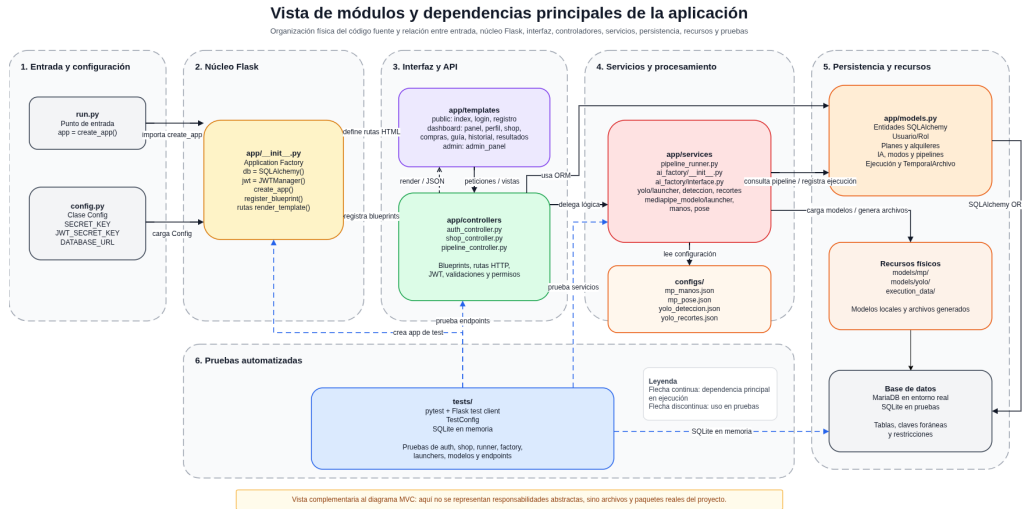


Figura C.4: Vista de módulos y dependencias principales de la aplicación

El punto de entrada de la aplicación se encuentra en `run.py`. Este archivo crea la aplicación mediante la llamada a `create_app()` y arranca el servidor Flask en el entorno de desarrollo. La configuración se centraliza en `config.py`, donde se definen parámetros como la clave de sesión, la clave utilizada para JWT y la URI de conexión a la base de datos. Esta separación evita dispersar la configuración en distintos puntos del código y facilita el cambio entre entornos.

El núcleo de inicialización se encuentra en `app/__init__.py`. En este módulo se crean las extensiones principales de la aplicación, como `SQLAlchemy` y `JWTManager`, se carga la configuración y se registran los `Blueprints` correspondientes a los distintos controladores. De este modo, la aplicación no queda construida directamente en el punto de entrada, sino mediante una factoría de aplicación. Esta solución resulta adecuada para proyectos Flask porque permite crear instancias configuradas de la aplicación tanto en ejecución normal como en entornos de prueba.

Los módulos situados en `app/controllers` concentran las rutas HTTP, las validaciones iniciales y las comprobaciones de permisos. Estos controladores actúan como intermediarios entre las peticiones del usuario y el resto del sistema. Por ejemplo, reciben formularios o datos JSON, comprueban que la solicitud sea válida, verifican el usuario autenticado mediante JWT y

consultan o modifican la información persistente cuando es necesario. Sin embargo, cuando la operación requiere lógica más compleja, delegan en la capa de servicios.

La carpeta **app/templates** contiene las plantillas HTML utilizadas por la interfaz web. Estas plantillas se agrupan según su finalidad: páginas públicas, vistas del panel de usuario y vistas de administración. Esta organización permite separar la presentación de la lógica de control, manteniendo la interfaz en una zona diferenciada respecto a los controladores y servicios.

La capa de servicios se localiza en **app/services**. En ella se encuentra **pipeline_runner.py**, encargado de orquestrar la ejecución de pipelines, y el paquete **ai_factory**, responsable de resolver qué lanzador debe utilizarse para cada modelo de inteligencia artificial. Dentro de esta capa también se agrupan los módulos específicos de YOLO y MediaPipe, que contienen la lógica concreta de procesamiento de imágenes. Esta distribución permite que los controladores no tengan que conocer los detalles internos de cada modelo.

El archivo **app/models.py** concentra las entidades SQLAlchemy de la aplicación. Estas entidades representan usuarios, roles, planes, alquileres, modelos de inteligencia artificial, modos de ejecución, pipelines, ejecuciones y archivos temporales. Desde el punto de vista arquitectónico, este módulo actúa como la capa de modelo de la aplicación y sirve de enlace entre la lógica Python y la base de datos relacional.

La carpeta **configs** almacena las configuraciones predeterminadas de los distintos modos de ejecución. Estas configuraciones permiten que el sistema disponga de valores base para cada modo, pero sin impedir que el usuario pueda modificar determinados parámetros antes de ejecutar un pipeline. Esta decisión refuerza la flexibilidad del sistema, ya que las configuraciones no quedan completamente fijadas en el código fuente.

Además de la persistencia en base de datos, el sistema utiliza recursos físicos en el servidor. En esta categoría se incluyen los modelos locales almacenados en carpetas como **models/mp** y **models/yolo**, así como los archivos temporales o resultados generados durante las ejecuciones. Estos recursos se relacionan con registros de base de datos, especialmente mediante la tabla de archivos temporales, pero no se almacenan directamente como datos binarios dentro del modelo relacional.

Por último, el paquete **tests** reproduce la creación de la aplicación en un entorno controlado. Para ello utiliza una configuración específica de pruebas, un cliente de pruebas de Flask y una base de datos SQLite en memoria.

Esto permite validar controladores, servicios, factorías, lanzadores y modelos sin modificar la base de datos real ni depender de recursos externos. Esta organización hace que las pruebas sean más rápidas, repetibles y seguras.

En conjunto, esta vista de módulos permite comprobar que la arquitectura conceptual no se queda únicamente en una división teórica por capas, sino que se refleja en la estructura real del código fuente. Cada paquete cumple una función clara y las dependencias principales siguen una dirección coherente: entrada y configuración, inicialización de la aplicación, controladores, servicios, modelo, recursos y persistencia.

Diagrama de clases y módulos principales de la capa de servicios

La Figura C.5 muestra una vista más detallada de los elementos que intervienen en la capa de servicios y en la resolución dinámica de modelos de inteligencia artificial. Este diagrama no pretende repetir las entidades persistentes ya representadas en los diagramas entidad-relación y relacional, sino mostrar la estructura lógica de los controladores, servicios, factorías, lanzadores y módulos funcionales implicados en la ejecución de pipelines.

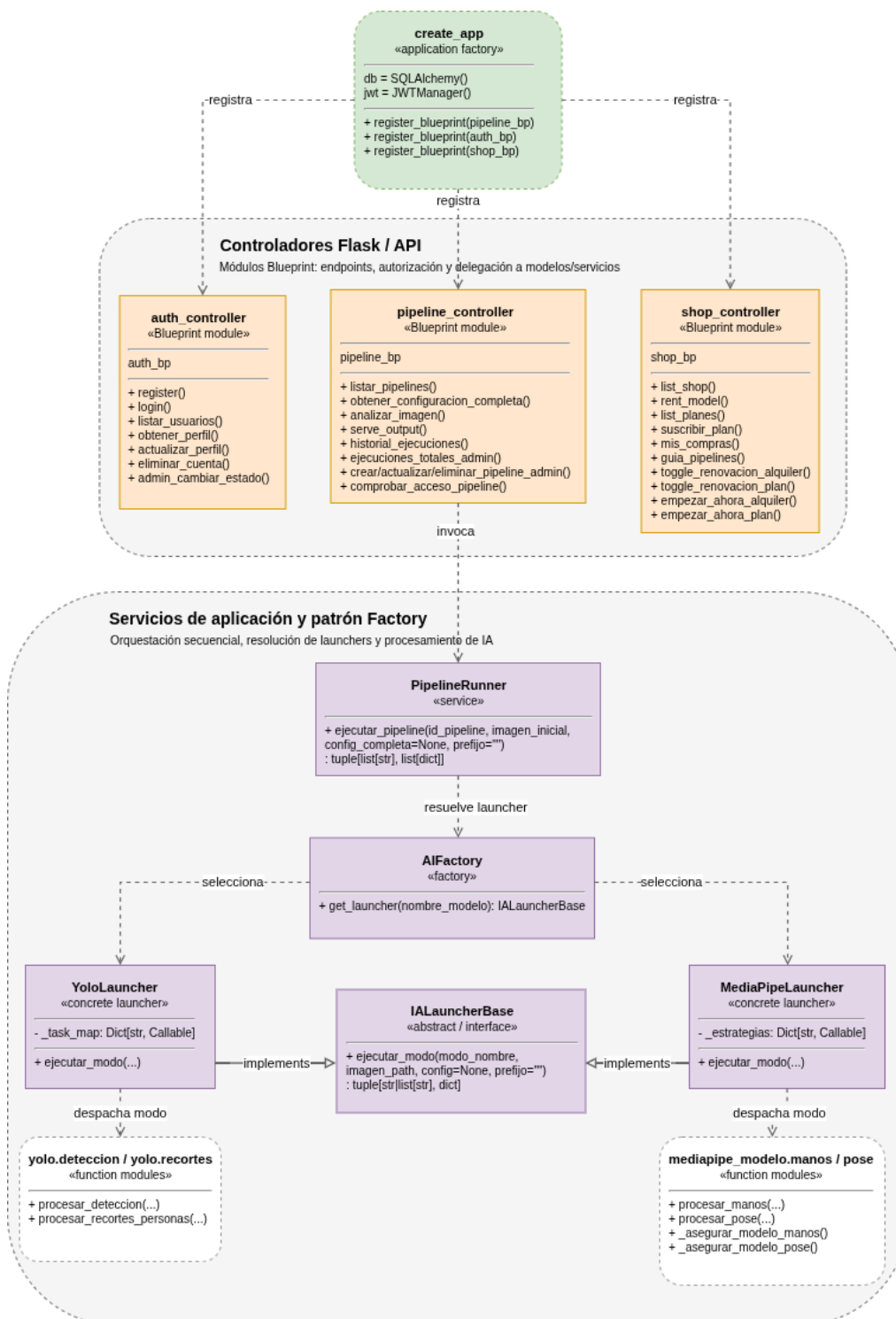


Figura C.5: Diagrama de clases y módulos principales de la capa de servicios

El diagrama combina clases y módulos porque la aplicación está construida sobre Flask y Python. Por un lado, existen componentes que pueden representarse claramente como clases o interfaces, como `PipelineRunner`, `AIFactory`, `IALauncherBase`, `YoloLauncher` y `MediaPipeLauncher`. Por otro lado, existen módulos funcionales, como los controladores Flask o los módulos concretos de procesamiento de YOLO y MediaPipe, que no son clases en sentido estricto, pero sí forman parte de la estructura lógica del sistema.

El flujo principal comienza en `pipeline_controller`, que recibe la solicitud de ejecución de un pipeline. Este controlador valida la imagen recibida, comprueba la configuración enviada por el usuario, verifica los permisos de acceso y delega la ejecución en `PipelineRunner`. De este modo, el controlador no necesita conocer los detalles internos de cada modelo de inteligencia artificial ni la forma concreta en que se procesa cada etapa.

La clase `PipelineRunner` actúa como servicio de orquestación. Su responsabilidad es recorrer de forma ordenada las etapas de un pipeline, obtener la configuración que debe aplicarse en cada una de ellas y solicitar el lanzador adecuado para ejecutar el modelo correspondiente. Esta clase devuelve una colección de rutas de imágenes generadas y una estructura de resultados en formato JSON, lo que permite a la capa de controladores preparar posteriormente la respuesta al usuario.

Para evitar que `PipelineRunner` tenga que instanciar directamente cada tipo de modelo, se introduce `AIFactory`. Esta factoría centraliza la resolución del lanzador adecuado a partir del modelo solicitado. Así, cuando una etapa requiere YOLO, la factoría selecciona `YoloLauncher`; cuando una etapa requiere MediaPipe, selecciona `MediaPipeLauncher`. Esta decisión reduce el acoplamiento entre el orquestador y las implementaciones concretas de inteligencia artificial.

La interfaz `IALauncherBase` define el comportamiento común que deben ofrecer los lanzadores de modelos. A partir de ella, `YoloLauncher` y `MediaPipeLauncher` implementan la operación de ejecución de modo. Cada lanzador se encarga de despachar la operación hacia los módulos funcionales correspondientes. En el caso de YOLO, se utilizan módulos como `yolo.deteccion` y `yolo.recortes`; en el caso de MediaPipe, se emplean módulos como `mediapipe_modelo.manos` y `mediapipe_modelo.pose`.

Esta organización permite que el sistema sea ampliable. Si en el futuro se quisiera incorporar un nuevo modelo de inteligencia artificial, no sería necesario modificar de forma profunda el controlador ni el orquestador principal. Bastaría con añadir el nuevo lanzador, implementar sus modos

de ejecución y registrar su resolución en la factoría. Del mismo modo, si un modelo ya existente incorpora un nuevo modo, este podría añadirse en el lanzador correspondiente sin alterar el funcionamiento general de la aplicación.

Otra ventaja de este diseño es que la definición de los pipelines queda desacoplada del código fuente. Las etapas se almacenan en la base de datos y cada una referencia un modelo y un modo concreto. Durante la ejecución, el sistema interpreta esa definición y resuelve dinámicamente qué componente debe actuar. Esto permite que un administrador pueda modificar la composición de los pipelines desde la aplicación, manteniendo una estructura flexible y preparada para nuevas combinaciones de procesamiento.

Valoración global de la arquitectura

La arquitectura resultante mantiene una separación clara entre presentación, control de peticiones, lógica de aplicación, procesamiento de inteligencia artificial y persistencia. Esta separación es especialmente importante en un sistema que combina autenticación, gestión comercial, administración de recursos, procesamiento de imágenes y generación de resultados temporales.

La adaptación del patrón MVC a Flask permite estructurar la aplicación de forma comprensible sin forzar una correspondencia rígida con un framework MVC tradicional. Los `Blueprints` actúan como controladores, las plantillas HTML y respuestas JSON cumplen la función de vista, y las clases `SQLAlchemy` representan el modelo persistente. Sobre esta base se incorpora una capa de servicios que permite aislar la lógica más compleja del proyecto.

El diagrama de módulos y dependencias muestra que esta arquitectura se refleja en la organización real del código. El punto de entrada, la configuración, la inicialización de Flask, los controladores, las plantillas, los servicios, los modelos, las configuraciones externas, los recursos físicos y las pruebas se encuentran separados en elementos diferenciados. Esta división reduce el acoplamiento entre partes del sistema y facilita localizar cada responsabilidad dentro del proyecto.

El principal beneficio de la capa de servicios se observa en la ejecución de pipelines. En lugar de codificar cada flujo de procesamiento como una función independiente e inamovible, el sistema interpreta una secuencia de etapas almacenada en base de datos y utiliza una factoría para resolver dinámicamente el modelo que debe ejecutarse. Esta solución reduce la

dependencia entre componentes y facilita la incorporación de nuevos modelos, modos y pipelines.

Además, el diseño favorece la mantenibilidad del código. Los controladores quedan más centrados en la gestión de peticiones y respuestas; los servicios contienen la lógica de orquestación; los lanzadores encapsulan las particularidades de cada familia de modelos; y los módulos funcionales realizan el procesamiento concreto de las imágenes. Esta distribución evita controladores monolíticos y permite localizar con mayor facilidad el lugar donde debe realizarse una modificación.

En conjunto, la arquitectura propuesta resulta coherente con los objetivos del proyecto. Permite construir una aplicación web funcional, con autenticación, gestión de usuarios, planes y modelos, ejecución de pipelines configurables y consulta de resultados. Al mismo tiempo, deja preparada la estructura para futuras ampliaciones, como la incorporación de nuevos modelos de inteligencia artificial, nuevos modos de ejecución, nuevas configuraciones de pipeline o mejoras en la administración del sistema.

C.4. Diseño procedimental

El diseño procedimental describe el comportamiento dinámico del sistema a partir de los principales flujos de interacción entre los actores, la interfaz web, los controladores de Flask, los servicios auxiliares y la capa de persistencia. Mientras que el diseño de datos define la estructura de la información y el diseño arquitectónico muestra la organización general de la aplicación, este apartado se centra en el orden en el que se producen las operaciones durante la ejecución de los casos de uso.

Para representar estos flujos se han elaborado diagramas de secuencia simplificados. En ellos se muestran únicamente los componentes que intervienen realmente en cada operación: el actor correspondiente, la vista web asociada, los controladores de la aplicación, la base de datos gestionada mediante SQLAlchemy sobre MariaDB, el servicio JWT cuando interviene en procesos de autenticación y los servicios internos necesarios en los flujos de procesamiento. Además, los diagramas incluyen alternativas de error mediante bloques *alt*, con el objetivo de reflejar el tratamiento de situaciones como credenciales incorrectas, ausencia de permisos, errores de validación, recursos inexistentes o fallos de persistencia.

Flujos de autenticación y gestión de cuenta

Los primeros flujos procedimentales están relacionados con la creación de cuentas, el inicio de sesión y la gestión básica del perfil del usuario. Estos casos de uso son la base del resto de operaciones privadas de la aplicación, ya que permiten identificar al usuario, asociarle roles y aplicar posteriormente las reglas de acceso a planes, modelos y pipelines.

La Figura C.6 muestra el proceso de registro de un usuario no autenticado. En este flujo, la interfaz recoge los datos del formulario y el controlador de autenticación valida el contenido recibido, comprueba la unicidad del correo y del nombre de usuario, crea la cuenta en base de datos, asigna el rol correspondiente y genera un token JWT para permitir el acceso posterior al sistema.

La Figura C.7 representa el inicio de sesión. El usuario introduce sus credenciales desde la interfaz web y el controlador de autenticación consulta la cuenta asociada, valida la contraseña, comprueba el estado de la cuenta y solicita la generación del token de acceso. En caso de credenciales incorrectas, usuario inexistente o cuenta suspendida, el flujo alternativo devuelve el error correspondiente a la interfaz.

La Figura C.8 recoge la consulta y modificación del perfil. En este caso, el usuario autenticado solicita sus datos, la interfaz envía la petición protegida al controlador y este recupera la información de la cuenta desde la base de datos. Cuando el usuario modifica campos permitidos, el controlador valida los datos, comprueba la contraseña actual si procede y actualiza la información persistida.

La Figura C.9 muestra la baja lógica de la cuenta. El usuario solicita la operación desde la interfaz, el controlador identifica la cuenta autenticada y actualiza su estado en base de datos. Este diseño evita una eliminación física inmediata y permite mantener la coherencia con ejecuciones, compras o suscripciones previamente registradas.

Flujos de tienda, planes y servicios

El segundo bloque de diagramas describe los procedimientos asociados a la tienda de servicios. Estos flujos permiten consultar los planes y modelos disponibles, contratar un plan, alquilar modelos de IA y revisar el estado de los servicios activos del usuario.

La Figura C.10 representa la consulta de la tienda. La interfaz solicita al controlador de tienda el catálogo disponible y este recupera de la base de

datos los planes, modelos habilitados y el estado de contratación del usuario. El resultado permite mostrar en la interfaz qué servicios están disponibles y cuáles se encuentran ya activos.

La Figura C.11 muestra el proceso de suscripción a un plan. Tras seleccionar un plan en la interfaz, el controlador comprueba que existe en la base de datos, registra la suscripción asociada al usuario y actualiza su estado de acceso. Las alternativas contemplan situaciones como plan inexistente, usuario no autenticado o error al guardar la operación.

La Figura C.12 representa el alquiler individual de un modelo de inteligencia artificial. Este flujo permite a un usuario con plan básico ampliar temporalmente su acceso a un modelo concreto. El controlador valida la existencia y disponibilidad del modelo, comprueba si el usuario ya dispone de un acceso activo y registra el alquiler con sus fechas de vigencia.

La Figura C.13 recoge la consulta de compras y servicios activos. La interfaz solicita al controlador el estado de suscripciones y alquileres del usuario autenticado, y el sistema devuelve la información necesaria para mostrar fechas, estado de vigencia y opciones asociadas a cada servicio.

La Figura C.14 muestra la guía de compra. En este flujo, el sistema obtiene los pipelines disponibles y sus dependencias de modelos y modos, cruza esta información con los servicios activos del usuario y devuelve una orientación sobre qué servicios debería contratar para ampliar su acceso.

Flujos de consulta, configuración y ejecución de pipelines

El tercer bloque procedimental se centra en la funcionalidad principal de la aplicación: la selección de pipelines, la carga de configuración, la ejecución del análisis y la consulta de resultados. Estos flujos son los más relevantes desde el punto de vista técnico, ya que combinan autenticación, validación de permisos, persistencia, procesamiento de imágenes y generación de resultados temporales.

La Figura C.15 representa la consulta de pipelines disponibles. El controlador identifica al usuario autenticado, revisa su rol, su plan y sus alquileres activos, y filtra los pipelines que puede ejecutar. De esta forma, la interfaz no muestra simplemente todos los pipelines existentes, sino únicamente aquellos compatibles con los permisos reales del usuario.

La Figura C.16 muestra la carga y modificación de la configuración de un pipeline. El sistema recupera la configuración predeterminada de las etapas

y la presenta en la interfaz para que el usuario pueda revisarla o adaptarla antes de lanzar la ejecución. La validación definitiva de la configuración se realiza posteriormente durante el envío de la solicitud de análisis.

La Figura C.17 representa el flujo central de ejecución de un pipeline sobre una imagen. El usuario selecciona un pipeline, adjunta una imagen y envía opcionalmente una configuración personalizada. El controlador valida el archivo, comprueba el formato de la configuración, verifica los permisos de acceso, registra la ejecución, prepara un espacio de trabajo temporal y delega la ejecución en el servicio encargado de recorrer las etapas del pipeline. Al finalizar, se registran los archivos temporales generados y se devuelve a la interfaz la información necesaria para visualizar el resultado.

La Figura C.18 muestra la visualización y descarga de resultados. La interfaz presenta las etapas ejecutadas, las imágenes generadas y los datos estructurados en formato JSON. Cuando el usuario solicita una descarga, el sistema resuelve el archivo mediante un token temporal, comprueba que no haya caducado y entrega el recurso sin exponer directamente la ruta interna del servidor.

La Figura C.19 recoge la consulta del historial de ejecuciones. El controlador recupera únicamente las ejecuciones asociadas al usuario autenticado y devuelve la información necesaria para mostrar el pipeline ejecutado, la fecha, el estado, la duración y los posibles errores. Este flujo mantiene el aislamiento entre usuarios, impidiendo que una cuenta pueda consultar ejecuciones ajenas.

Flujos de administración

El cuarto bloque agrupa los procedimientos propios del administrador. Estos flujos permiten acceder al panel de administración, gestionar cuentas de usuario, modificar pipelines sin tocar el código fuente y consultar ejecuciones globales. Desde el punto de vista procedimental, estos casos de uso reflejan la separación entre las operaciones disponibles para usuarios normales y las operaciones restringidas por rol.

La Figura C.20 muestra el acceso al panel de administración. El sistema valida el token JWT y comprueba que el usuario autenticado dispone del rol de administrador. Si la comprobación es correcta, la interfaz administrativa queda disponible; en caso contrario, se devuelve una respuesta de acceso denegado.

La Figura C.21 representa la gestión de usuarios desde administración. El administrador consulta el listado de cuentas, revisa información básica y

solicita cambios de estado cuando procede. El controlador valida la operación, evita acciones no permitidas y persiste los cambios en la base de datos.

La Figura C.22 muestra la gestión de pipelines desde el panel de administración. Este flujo es especialmente relevante porque permite crear, modificar, deshabilitar o eliminar pipelines desde la propia aplicación. El controlador comprueba que las etapas hagan referencia a modelos y modos válidos, guarda la definición del flujo y, en caso de eliminación, decide entre borrar físicamente el pipeline o deshabilitarlo si existen ejecuciones asociadas.

La Figura C.23 representa la consulta global de ejecuciones. A diferencia del historial de usuario, este flujo está restringido al administrador y permite obtener una visión general de las ejecuciones realizadas en el sistema, incluyendo su estado y posibles errores registrados.

Flujo de mantenimiento automático

Por último, el sistema incorpora tareas de mantenimiento orientadas a conservar la coherencia de los datos y evitar la acumulación de recursos temporales. Este flujo no parte de una interacción directa de un usuario, sino de una ejecución automática del propio sistema.

La Figura C.24 muestra el procedimiento de mantenimiento. El sistema localiza archivos temporales caducados, elimina los recursos físicos cuando procede, actualiza el estado de registros asociados, revisa suscripciones o alquileres expirados y aplica los cambios necesarios en base de datos. Este flujo contribuye a mantener bajo control el ciclo de vida de los archivos generados y de los servicios temporales contratados.

Síntesis del diseño procedimental

En conjunto, los diagramas anteriores muestran cómo la aplicación organiza sus procedimientos principales mediante una separación clara entre interfaz, controladores, servicios internos y persistencia. Los flujos de autenticación establecen la identidad del usuario mediante JWT; los flujos de tienda y servicios determinan qué permisos tiene cada cuenta; los flujos de pipelines materializan el procesamiento de imágenes mediante etapas configurables; y los flujos de administración permiten modificar usuarios y pipelines sin alterar directamente el código fuente.

Esta organización procedimental refuerza la escalabilidad del sistema. La lógica de negocio queda concentrada en los controladores y servicios corres-

pondientes, mientras que la interfaz actúa como punto de interacción con el usuario y la base de datos mantiene el estado persistente de cuentas, planes, modelos, pipelines, ejecuciones y archivos temporales. Como resultado, el sistema puede ampliarse con nuevos modelos, modos de ejecución o flujos de procesamiento manteniendo una estructura coherente y trazable.

C.5. Diseño de interfaces

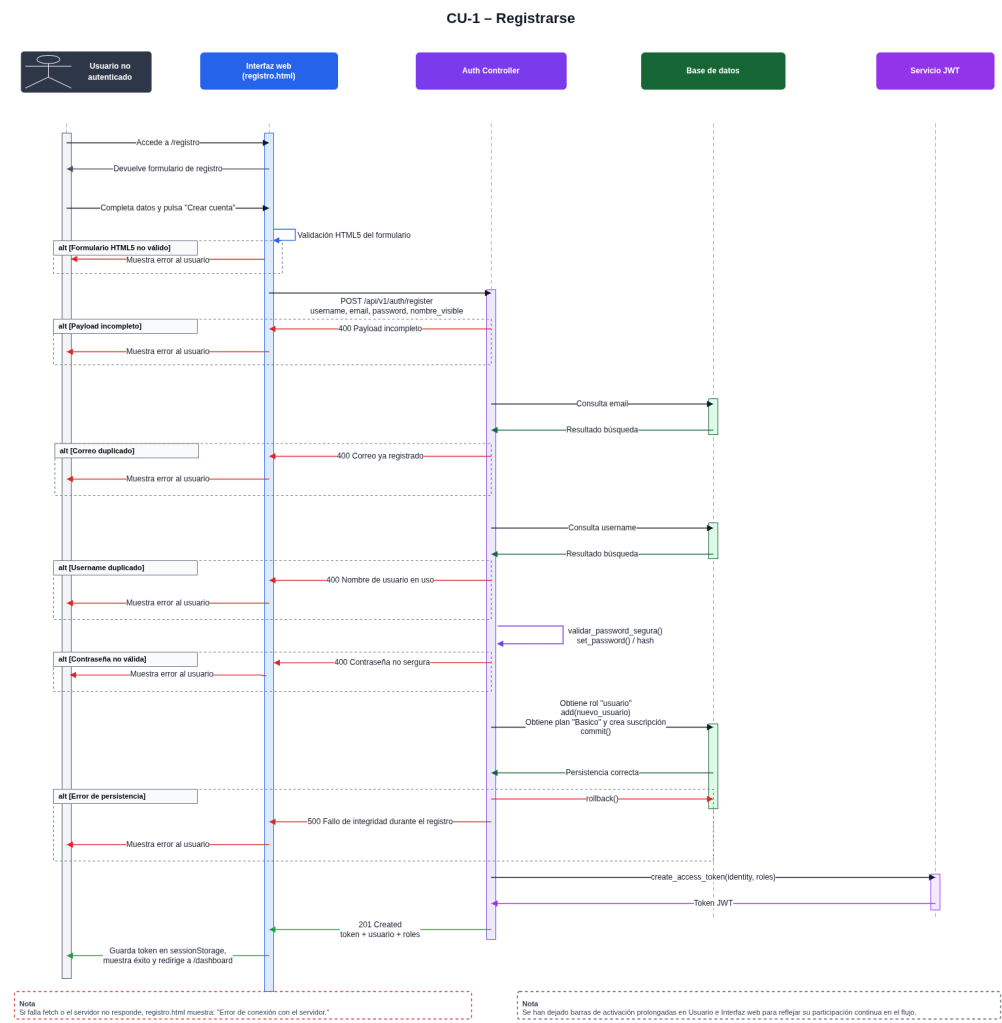


Figura C.6: Diagrama de secuencia del CU-1 Registrarse.

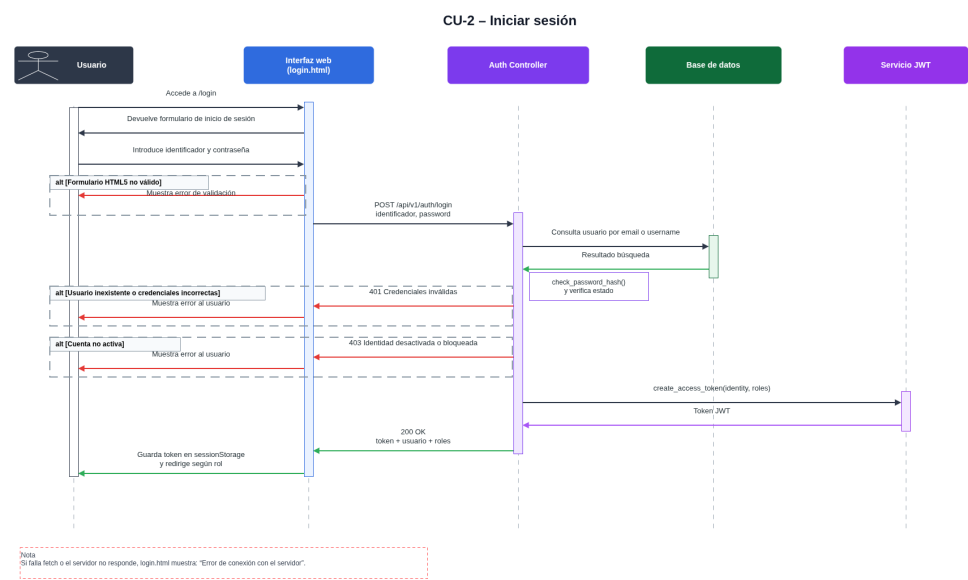


Figura C.7: Diagrama de secuencia del CU-2 Iniciar sesión.

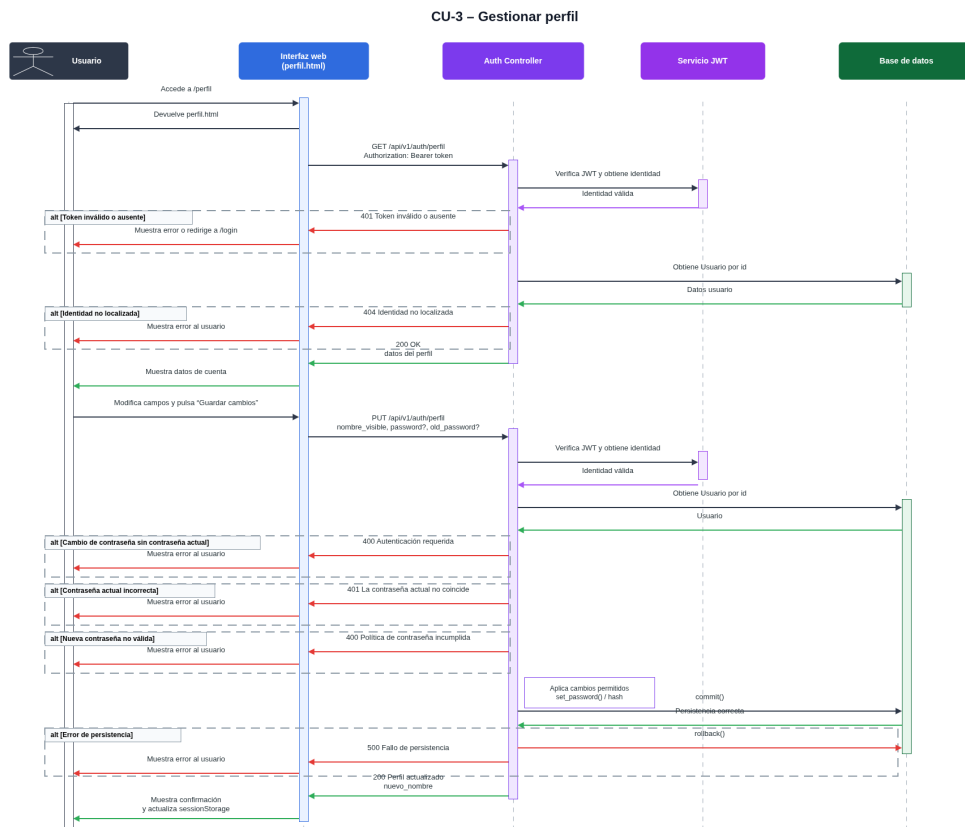


Figura C.8: Diagrama de secuencia del CU-3 Gestionar perfil.

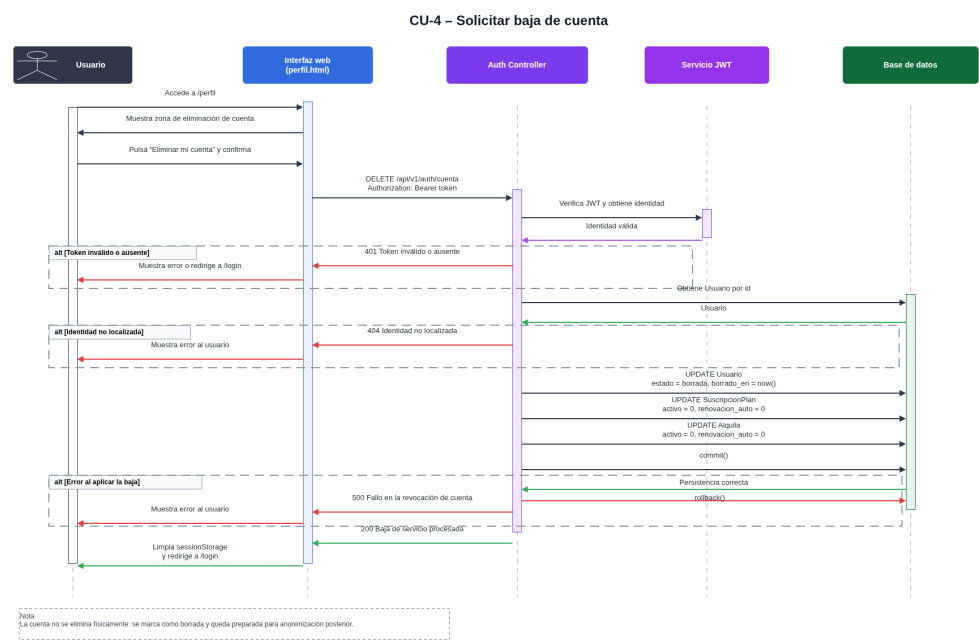


Figura C.9: Diagrama de secuencia del CU-4 Solicitar baja de cuenta.

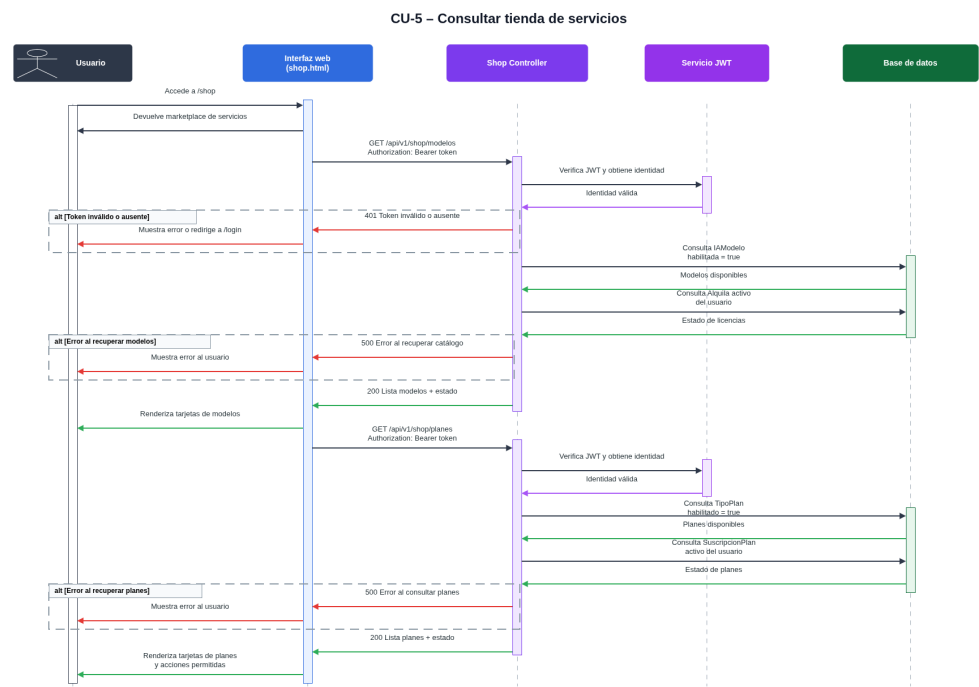


Figura C.10: Diagrama de secuencia del CU-5 Consultar tienda de servicios.

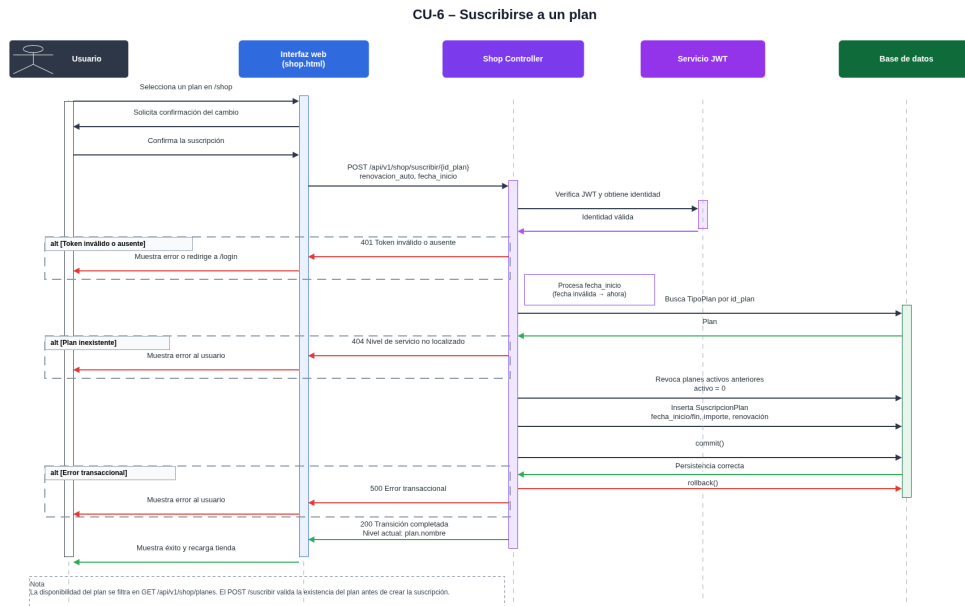


Figura C.11: Diagrama de secuencia del CU-6 Suscribirse a un plan.

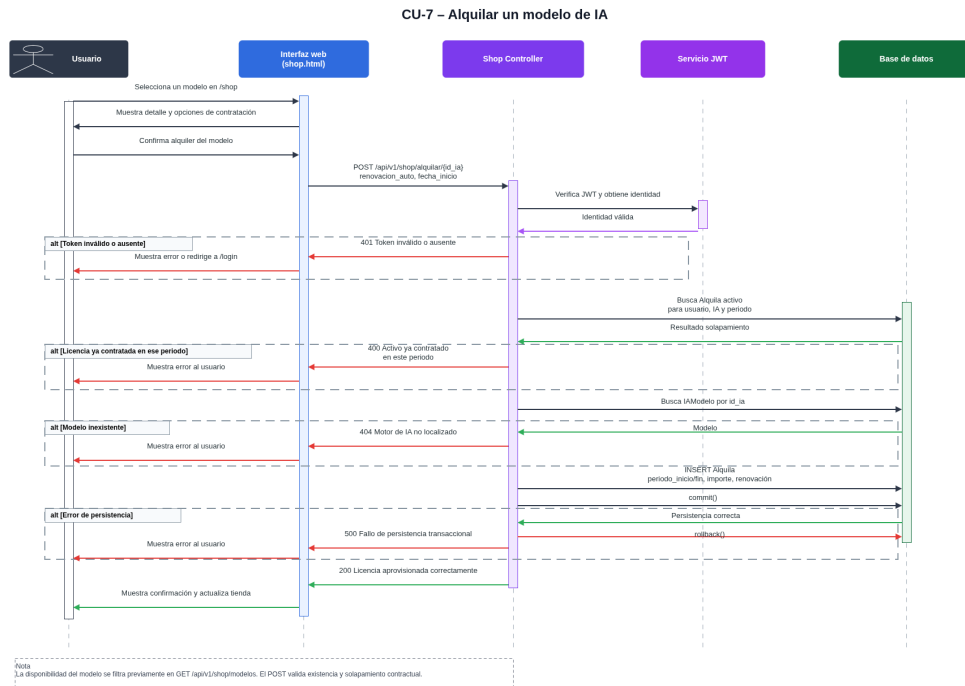


Figura C.12: Diagrama de secuencia del CU-7 Alquilar un modelo de IA.

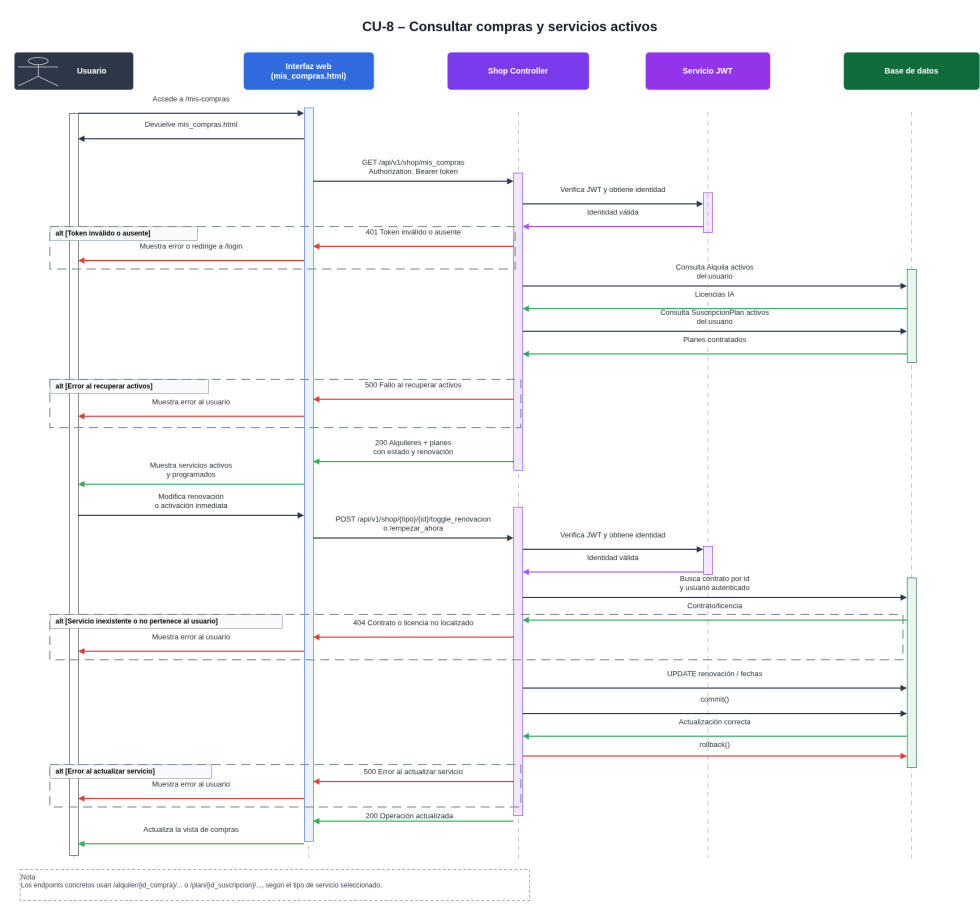


Figura C.13: Diagrama de secuencia del CU-8 Consultar compras y servicios activos.

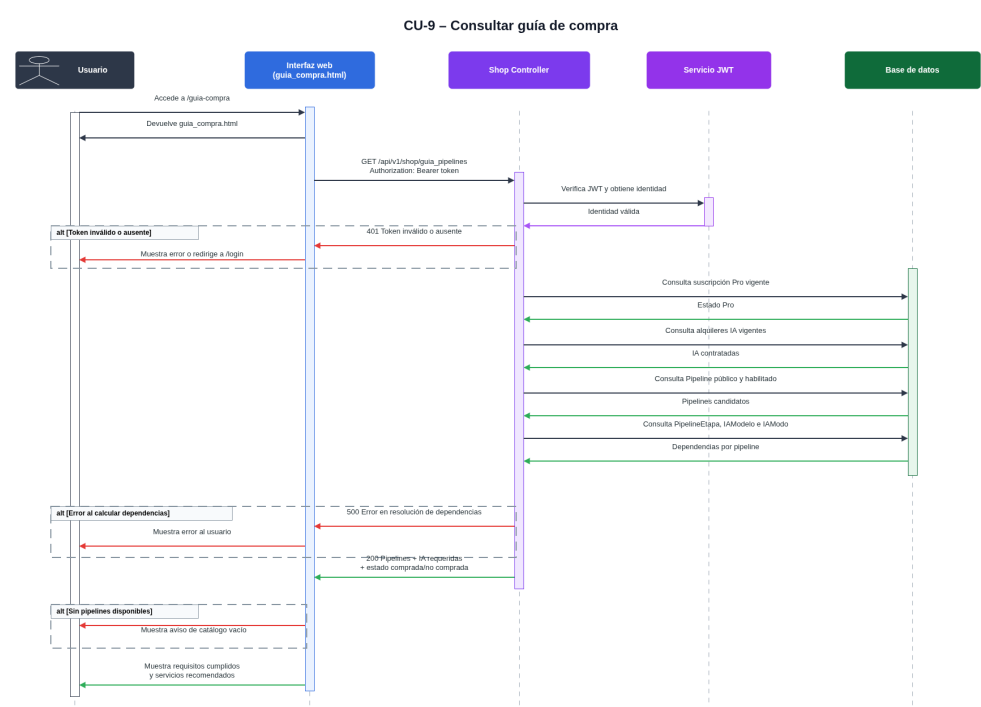


Figura C.14: Diagrama de secuencia del CU-9 Consultar guía de compra.

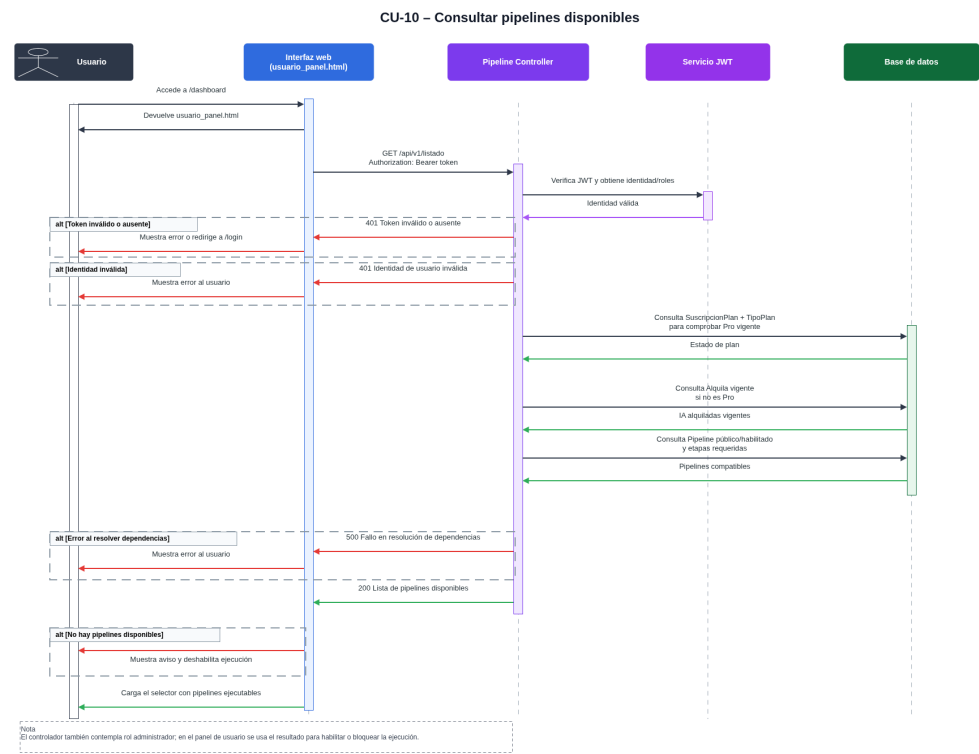


Figura C.15: Diagrama de secuencia del CU-10 Consultar pipelines disponibles.

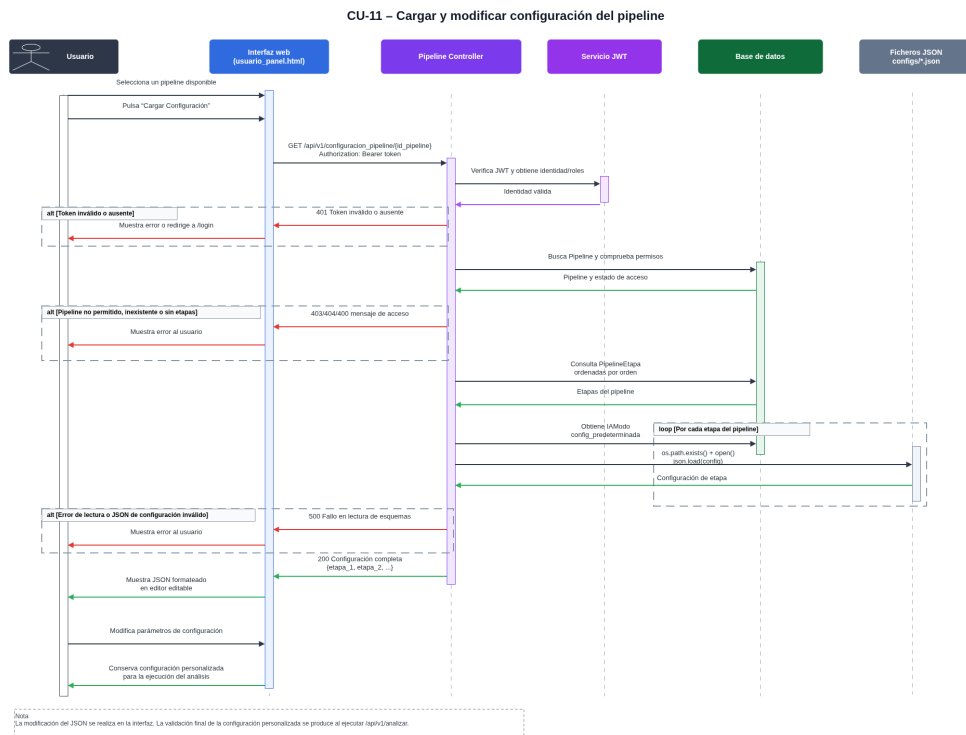


Figura C.16: Diagrama de secuencia del CU-11 Cargar y modificar configuración del pipeline.

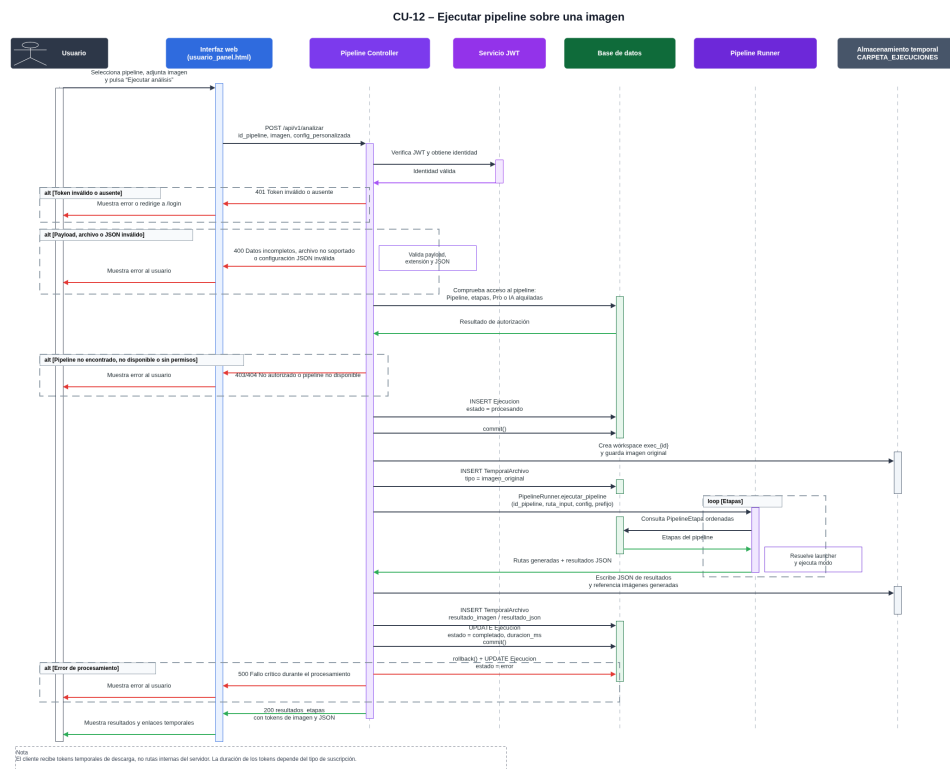


Figura C.17: Diagrama de secuencia del CU-12 Ejecutar pipeline sobre una imagen.

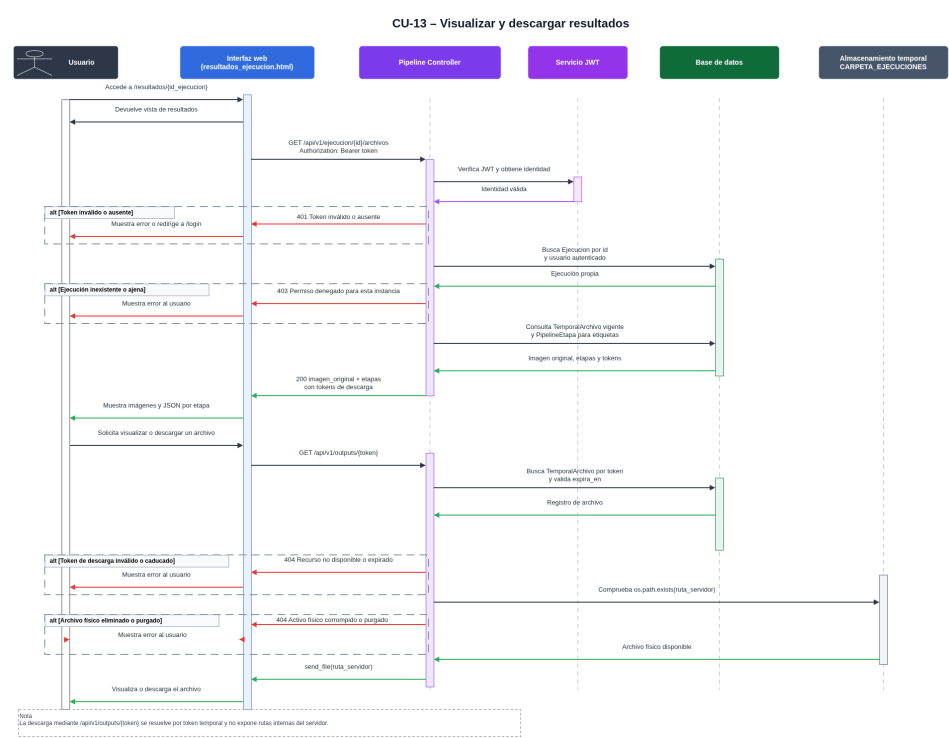


Figura C.18: Diagrama de secuencia del CU-13 Visualizar y descargar resultados.

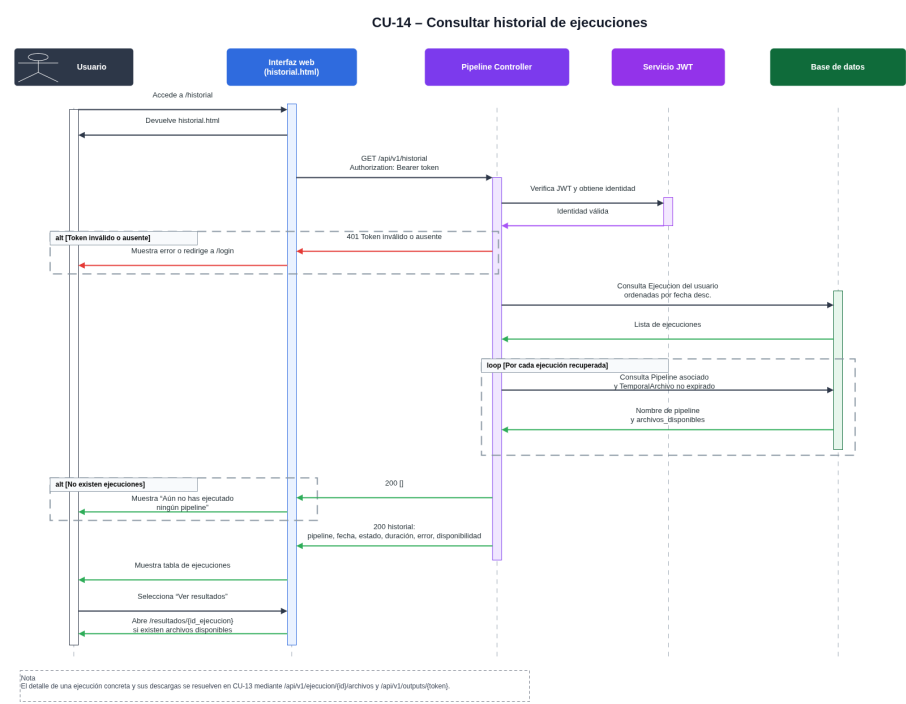


Figura C.19: Diagrama de secuencia del CU-14 Consultar historial de ejecuciones.

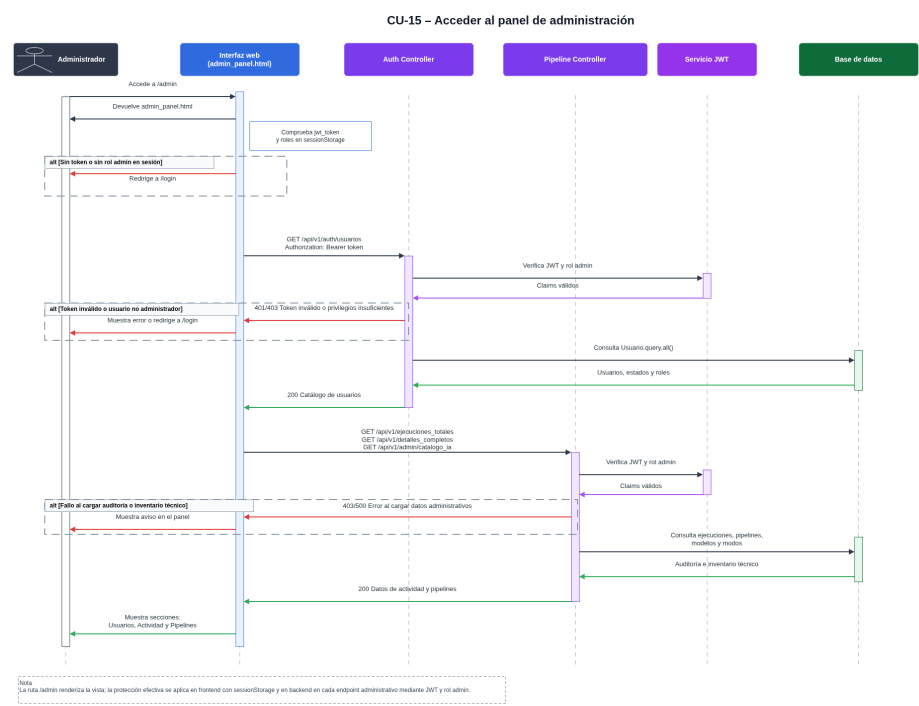


Figura C.20: Diagrama de secuencia del CU-15 Acceder al panel de administración.

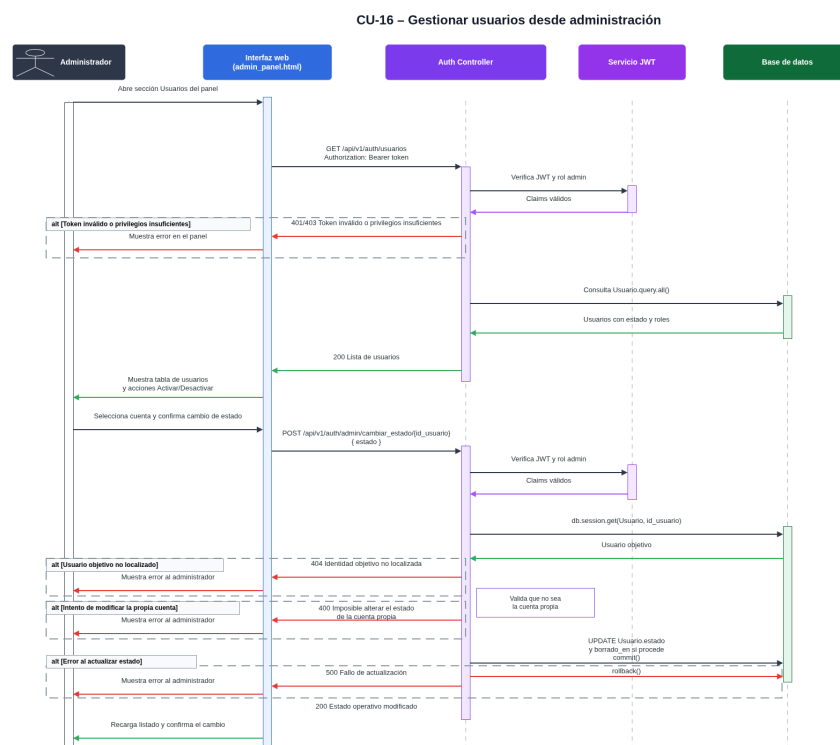


Figura C.21: Diagrama de secuencia del CU-16 Gestionar usuarios desde administración.

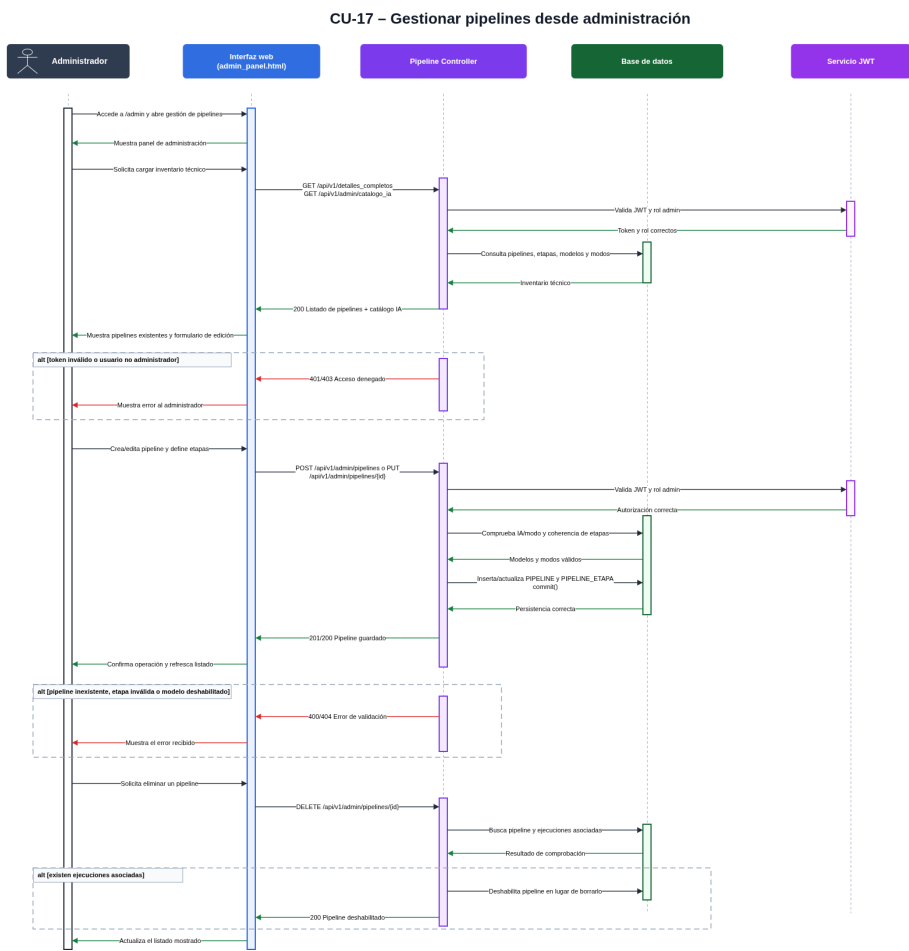


Figura C.22: Diagrama de secuencia del CU-17 Gestionar pipelines desde administración.

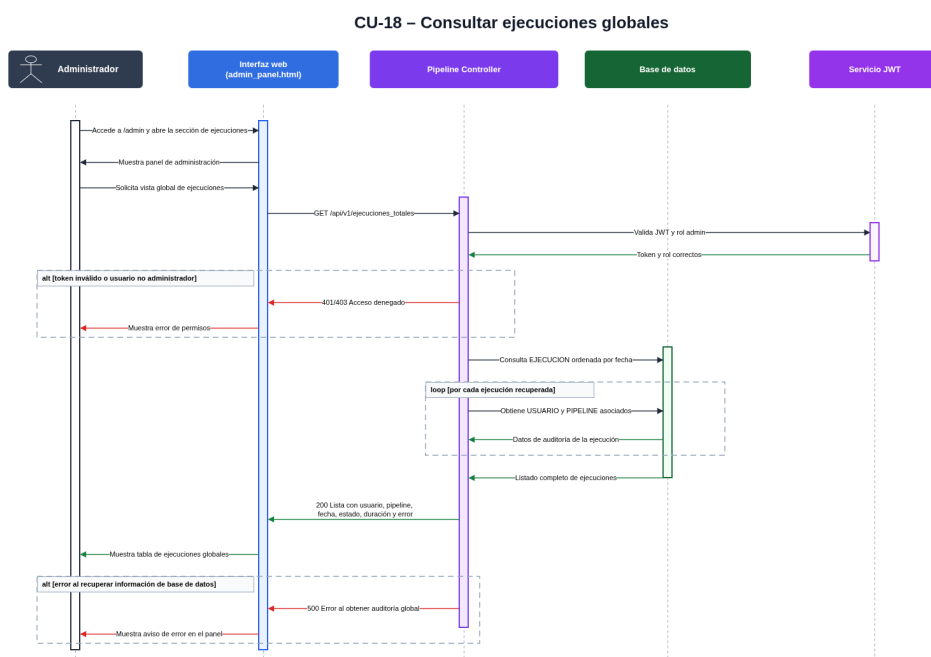


Figura C.23: Diagrama de secuencia del CU-18 Consultar ejecuciones globales.

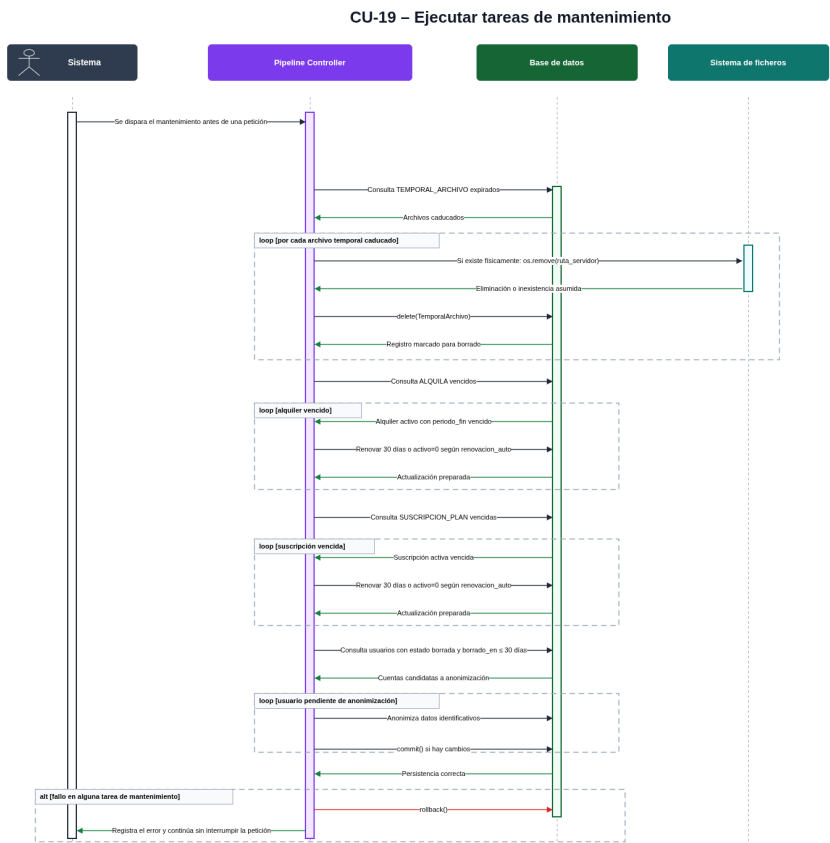


Figura C.24: Diagrama de secuencia del CU-19 Ejecutar tareas de mantenimiento.

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía

- [1] Python Cryptographic Authority. cryptography. <https://pypi.org/project/cryptography/>, 2026. [Online; accedido 22-mayo-2026].
- [2] SQLAlchemy Authors. Ssqlalchemy. <https://pypi.org/project/SQLAlchemy/>, 2026. [Online; accedido 22-mayo-2026].
- [3] The MediaPipe Authors. Mediapipe. <https://pypi.org/project/mediapipe/>, 2026. [Online; accedido 22-mayo-2026].
- [4] Creative Commons. Attribution 4.0 international cc by 4.0. <https://creativecommons.org/licenses/by/4.0/>, 2026. [Online; accedido 22-mayo-2026].
- [5] Flask-JWT-Extended contributors. Flask-jwt-extended. <https://pypi.org/project/Flask-JWT-Extended/>, 2026. [Online; accedido 22-mayo-2026].
- [6] Gunicorn contributors. Gunicorn. <https://pypi.org/project/gunicorn/>, 2026. [Online; accedido 22-mayo-2026].
- [7] Pillow contributors. Pillow. <https://pypi.org/project/Pillow/>, 2026. [Online; accedido 22-mayo-2026].
- [8] PyMySQL contributors. Pymysql. <https://pypi.org/project/PyMySQL/>, 2026. [Online; accedido 22-mayo-2026].
- [9] Tesoreria General de la Seguridad Social. Bases y tipos de cotizacion 2026, 2026. [Online; accedido 22-mayo-2026].

- [10] Ministerio de Trabajo y Economía Social. Actualización de las tablas salariales del xix convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública, 2026. [Online; accedido 22-mayo-2026].
- [11] Boletín Oficial del Estado. Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>, 2018. [Online; accedido 22-mayo-2026].
- [12] Boletín Oficial del Estado. Real decreto-ley 3/2026, de 3 de febrero, tarifa de primas para la cotización por accidentes de trabajo y enfermedades profesionales, 2026. [Online; accedido 22-mayo-2026].
- [13] NumPy Developers. Numpy. <https://pypi.org/project/numpy/>, 2026. [Online; accedido 22-mayo-2026].
- [14] Google AI Edge. Hand landmarks detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, 2026. [Online; accedido 22-mayo-2026].
- [15] Google AI Edge. Pose landmark detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker, 2026. [Online; accedido 22-mayo-2026].
- [16] Unión Europea. Reglamento ue 2016/679, artículo 17, derecho de supresión. <https://www.boe.es/doue/2016/119/L00001-00088.pdf>, 2016. [Online; accedido 22-mayo-2026].
- [17] Free Software Foundation. Gnu affero general public license. <https://www.gnu.org/licenses/agpl-3.0.en.html>, 2026. [Online; accedido 22-mayo-2026].
- [18] GitHub. Gnu affero general public license v3.0. <https://choosealicense.com/licenses/agpl-3.0/>, 2026. [Online; accedido 22-mayo-2026].
- [19] Google. Model card hand tracking with fairness. https://storage.googleapis.com/mediapipe-assets/Model%20Card%20Hand%20Tracking%20%28Lite_Full%29%20with%20Fairness%20Oct%202021.pdf, 2021. [Online; accedido 22-mayo-2026].
- [20] Saurabh Kumar. python-dotenv. <https://pypi.org/project/python-dotenv/>, 2026. [Online; accedido 22-mayo-2026].

- [21] OpenCV on Wheels contributors. opencv-python-headless. <https://pypi.org/project/opencv-python-headless/>, 2026. [Online; accedido 22-mayo-2026].
- [22] Pallets Projects. Flask. <https://pypi.org/project/Flask/>, 2026. [Online; accedido 22-mayo-2026].
- [23] Pallets Projects. Flask-sqlalchemy. <https://pypi.org/project/Flask-SQLAlchemy/>, 2026. [Online; accedido 22-mayo-2026].
- [24] Pallets Projects. Werkzeug. <https://pypi.org/project/Werkzeug/>, 2026. [Online; accedido 22-mayo-2026].
- [25] pytest-dev team. pytest. <https://pypi.org/project/pytest/>, 2026. [Online; accedido 22-mayo-2026].
- [26] PyTorch Team. torch. <https://pypi.org/project/torch/>, 2026. [Online; accedido 22-mayo-2026].
- [27] PyTorch Team. torchvision. <https://pypi.org/project/torchvision/>, 2026. [Online; accedido 22-mayo-2026].
- [28] Ultralytics. Ultralytics. <https://pypi.org/project/ultralytics/>, 2026. [Online; accedido 22-mayo-2026].
- [29] Ultralytics. Ultralytics licensing. <https://www.ultralytics.com/license>, 2026. [Online; accedido 22-mayo-2026].